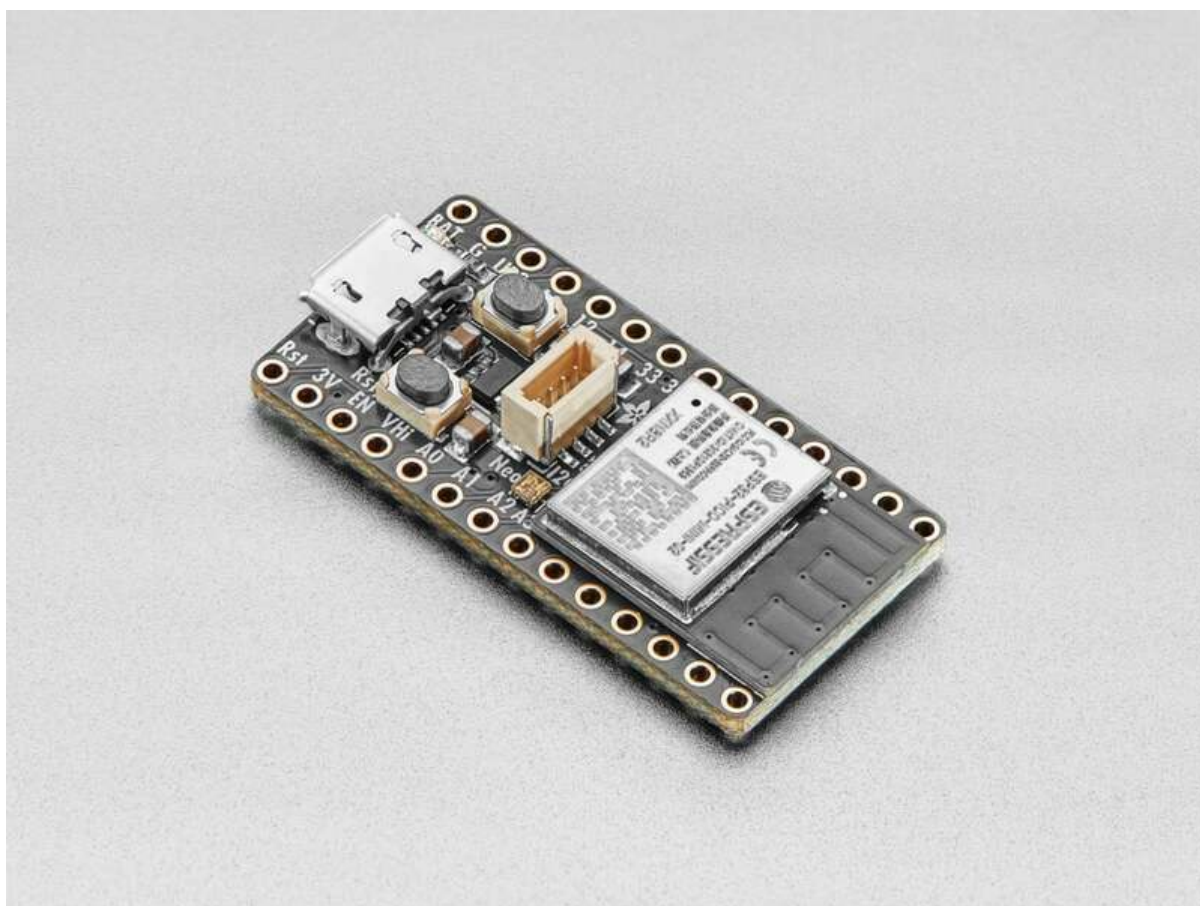




# Adafruit ItsyBitsy ESP32

Created by Liz Clark



<https://learn.adafruit.com/adafruit-itsybitsy-esp32>

Last updated on 2025-07-14 07:02:57 PM EDT

# Table of Contents

|                                                                                                                                                                                                                                                                                                               |    |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Overview                                                                                                                                                                                                                                                                                                      | 5  |
| Pinouts                                                                                                                                                                                                                                                                                                       | 8  |
| <ul style="list-style-type: none"><li>• Power Pins</li><li>• ESP32 Module</li><li>• Logic Pins</li><li>• Bottom Row</li><li>• Top Row</li><li>• STEMMA QT Connector</li><li>• NeoPixel LED</li><li>• Buttons</li><li>• Red LED</li><li>• CH9102 USB-to-Serial Converter</li><li>• w.FL Antenna Port</li></ul> |    |
| Low Power Use                                                                                                                                                                                                                                                                                                 | 15 |
| CircuitPython Setup                                                                                                                                                                                                                                                                                           | 19 |
| <ul style="list-style-type: none"><li>• Driver Install</li><li>• CircuitPython Download</li><li>• Connecting to the Web Flasher</li><li>• Erasing the Board Contents</li><li>• Programming the Board</li></ul>                                                                                                |    |
| Setting up Web Workflow                                                                                                                                                                                                                                                                                       | 24 |
| <ul style="list-style-type: none"><li>• Option 1: Create settings.toml file via REPL</li><li>• Option 2: Create settings.toml file using Thonny</li></ul>                                                                                                                                                     |    |
| Connecting via Web Browser                                                                                                                                                                                                                                                                                    | 30 |
| <ul style="list-style-type: none"><li>• Connect using MDNS Address</li><li>• Connect using IP Address</li></ul>                                                                                                                                                                                               |    |
| Using Web Workflow                                                                                                                                                                                                                                                                                            | 31 |
| <ul style="list-style-type: none"><li>• Welcome! Page</li><li>• Password Entry</li><li>• Serial Terminal</li><li>• File Browser</li></ul>                                                                                                                                                                     |    |
| NeoPixel Example                                                                                                                                                                                                                                                                                              | 34 |
| <ul style="list-style-type: none"><li>• Adding a Library File</li><li>• Edit code.py</li></ul>                                                                                                                                                                                                                |    |
| I2C Scan Test                                                                                                                                                                                                                                                                                                 | 38 |
| <ul style="list-style-type: none"><li>• Wiring</li><li>• Download the Project Bundle</li><li>• Upload the Project Bundle with Web Workflow</li></ul>                                                                                                                                                          |    |
| CircuitPython WiFi Test                                                                                                                                                                                                                                                                                       | 42 |
| <ul style="list-style-type: none"><li>• Download the Project Bundle</li><li>• Upload the Project Bundle with Web Workflow</li></ul>                                                                                                                                                                           |    |
| Arduino IDE Setup                                                                                                                                                                                                                                                                                             | 47 |

|                                                                                                                                                                                                                                                                                                                                                                                                                         |           |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>Blink</b>                                                                                                                                                                                                                                                                                                                                                                                                            | <b>51</b> |
| <hr/>                                                                                                                                                                                                                                                                                                                                                                                                                   |           |
| <ul style="list-style-type: none"><li>• <a href="#">Pre-Flight Check: Get Arduino IDE &amp; Hardware Set Up</a></li><li>• <a href="#">Start up Arduino IDE and Select Board/Port</a></li><li>• <a href="#">New Blink Sketch</a></li><li>• <a href="#">Verify (Compile) Sketch</a></li><li>• <a href="#">Upload Sketch</a></li><li>• <a href="#">Finally, a Blink!</a></li></ul>                                         |           |
| <b>I2C Scan Test</b>                                                                                                                                                                                                                                                                                                                                                                                                    | <b>58</b> |
| <hr/>                                                                                                                                                                                                                                                                                                                                                                                                                   |           |
| <ul style="list-style-type: none"><li>• <a href="#">Common I2C Connectivity Issues</a></li><li>• <a href="#">Perform an I2C scan!</a></li><li>• <a href="#">Wiring the MCP9808</a></li></ul>                                                                                                                                                                                                                            |           |
| <b>WiFi Test</b>                                                                                                                                                                                                                                                                                                                                                                                                        | <b>62</b> |
| <hr/>                                                                                                                                                                                                                                                                                                                                                                                                                   |           |
| <ul style="list-style-type: none"><li>• <a href="#">WiFi Connection Test</a></li><li>• <a href="#">Secure Connection Example</a></li></ul>                                                                                                                                                                                                                                                                              |           |
| <b>WipperSnapper Setup</b>                                                                                                                                                                                                                                                                                                                                                                                              | <b>69</b> |
| <hr/>                                                                                                                                                                                                                                                                                                                                                                                                                   |           |
| <ul style="list-style-type: none"><li>• <a href="#">What is WipperSnapper</a></li><li>• <a href="#">Sign up for Adafruit.io</a></li><li>• <a href="#">Add a New Device to Adafruit IO</a></li><li>• <a href="#">Feedback</a></li><li>• <a href="#">Troubleshooting</a></li><li>• <a href="#">"Uninstalling" WipperSnapper</a></li></ul>                                                                                 |           |
| <b>WipperSnapper Essentials</b>                                                                                                                                                                                                                                                                                                                                                                                         | <b>75</b> |
| <hr/>                                                                                                                                                                                                                                                                                                                                                                                                                   |           |
| <b>LED Blink</b>                                                                                                                                                                                                                                                                                                                                                                                                        | <b>76</b> |
| <hr/>                                                                                                                                                                                                                                                                                                                                                                                                                   |           |
| <ul style="list-style-type: none"><li>• <a href="#">Where is the LED on my board?</a></li><li>• <a href="#">Create a LED Component on Adafruit IO</a></li><li>• <a href="#">Usage</a></li></ul>                                                                                                                                                                                                                         |           |
| <b>Read a Push-button</b>                                                                                                                                                                                                                                                                                                                                                                                               | <b>80</b> |
| <hr/>                                                                                                                                                                                                                                                                                                                                                                                                                   |           |
| <ul style="list-style-type: none"><li>• <a href="#">Button Location</a></li><li>• <a href="#">Create a Push-button Component on Adafruit IO</a></li></ul>                                                                                                                                                                                                                                                               |           |
| <b>NeoPixel LED</b>                                                                                                                                                                                                                                                                                                                                                                                                     | <b>85</b> |
| <hr/>                                                                                                                                                                                                                                                                                                                                                                                                                   |           |
| <ul style="list-style-type: none"><li>• <a href="#">Where is the NeoPixel on my board?</a></li><li>• <a href="#">Create the NeoPixel Component</a></li><li>• <a href="#">Set the NeoPixel's RGB Color</a></li><li>• <a href="#">Set NeoPixel Brightness</a></li></ul>                                                                                                                                                   |           |
| <b>I2C Sensor</b>                                                                                                                                                                                                                                                                                                                                                                                                       | <b>91</b> |
| <hr/>                                                                                                                                                                                                                                                                                                                                                                                                                   |           |
| <ul style="list-style-type: none"><li>• <a href="#">Parts</a></li><li>• <a href="#">Wiring</a></li><li>• <a href="#">Add an MCP9808 Component</a></li><li>• <a href="#">Read I2C Sensor Values</a></li></ul>                                                                                                                                                                                                            |           |
| <b>Analog Input</b>                                                                                                                                                                                                                                                                                                                                                                                                     | <b>96</b> |
| <hr/>                                                                                                                                                                                                                                                                                                                                                                                                                   |           |
| <ul style="list-style-type: none"><li>• <a href="#">Analog to Digital Converter (ADC)</a></li><li>• <a href="#">Potentiometers</a></li><li>• <a href="#">Hardware</a></li><li>• <a href="#">Wire Up the Potentiometer</a></li><li>• <a href="#">Create a Potentiometer Component on Adafruit IO</a></li><li>• <a href="#">Read Analog Pin Values</a></li><li>• <a href="#">Read Analog Pin Voltage Values</a></li></ul> |           |

## Factory Reset

103

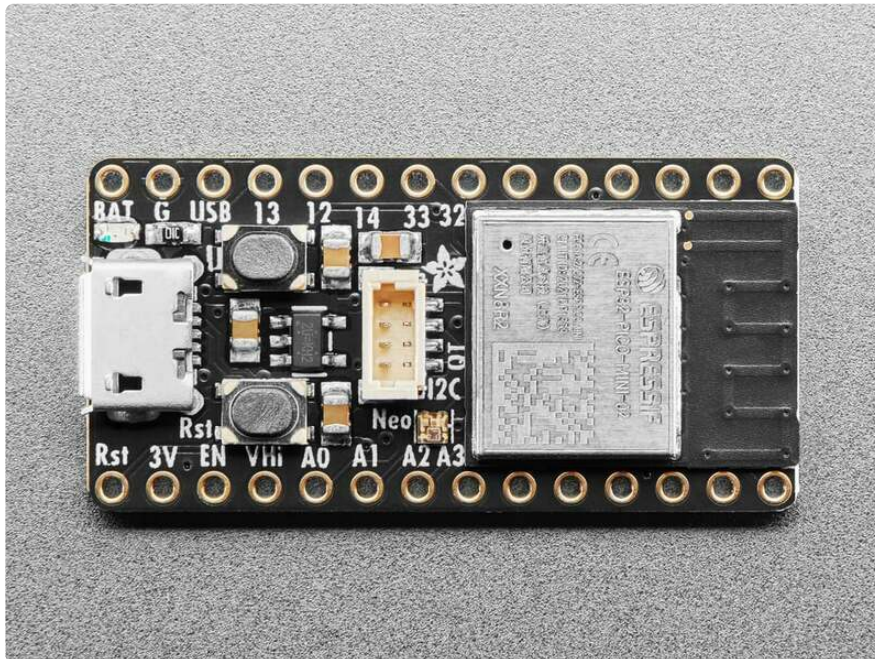
- [Factory Reset Example Code](#)
- [Factory Reset .bin](#)
- [The WebSerial ESPTool Method](#)
- [The esptool Method \(for advanced users\)](#)
- [Reset the board](#)
- [Older Versions of Chrome](#)

## Downloads

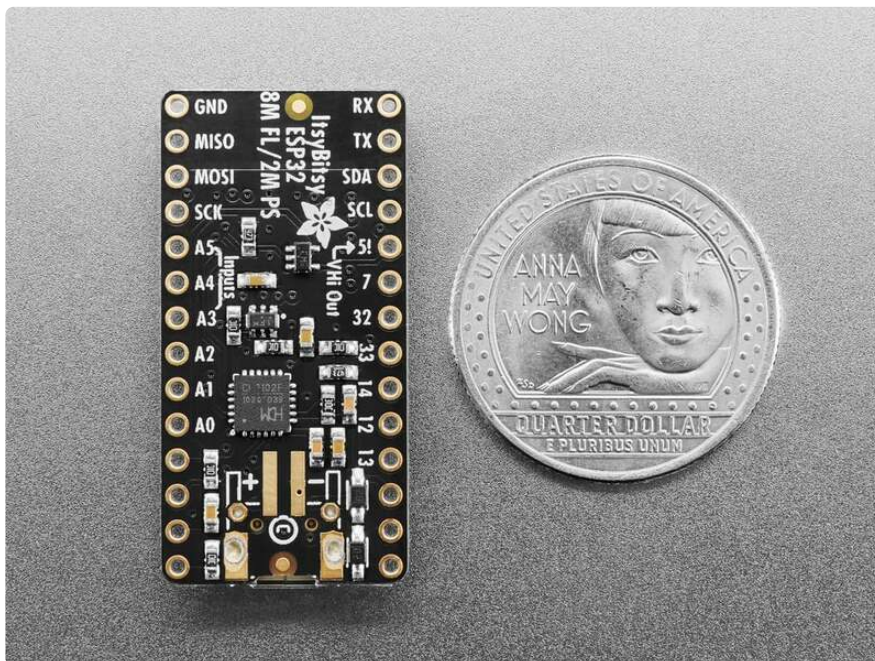
112

- [Files](#)
- [Schematic and Fab Print](#)
- [3D Model](#)

# Overview



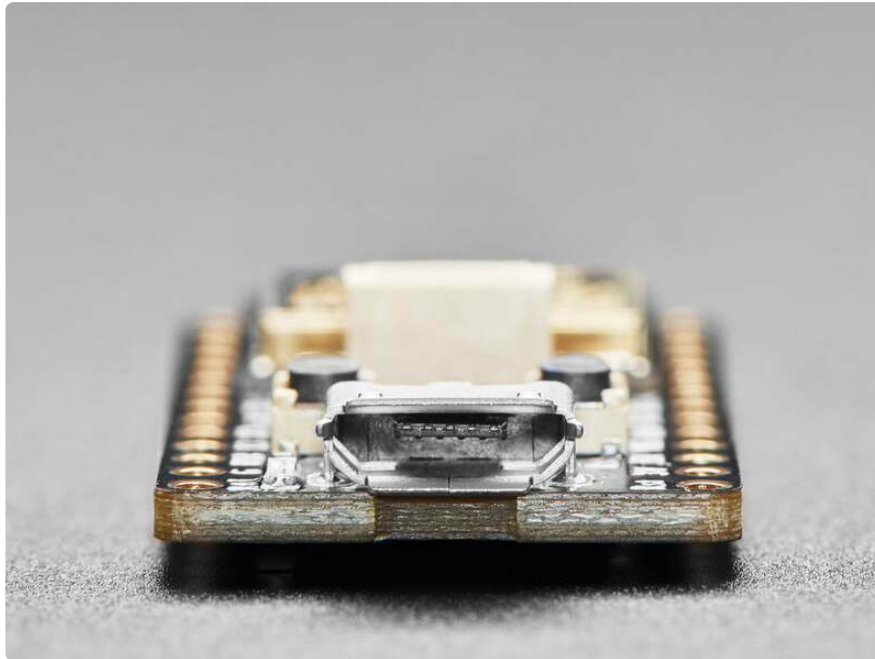
What's smaller than a Feather but larger than a Trinket? It's an **Adafruit ItsyBitsy ESP32**, a powerful processor PCB with a plethora of pins! It features the ESP32 Pico module, an FCC-certified module that contains an **ESP32 chip with dual-core 240MHz Tensilica processor, WiFi, and Bluetooth classic + BLE**, configured with **8 MB of Flash memory**, and **2 MB of PSRAM**.



We've added some handy accessories like a USB to serial converter chip, power regulator, USB Micro B, buttons, NeoPixel, and STEMMA QT I2C connector to outfit

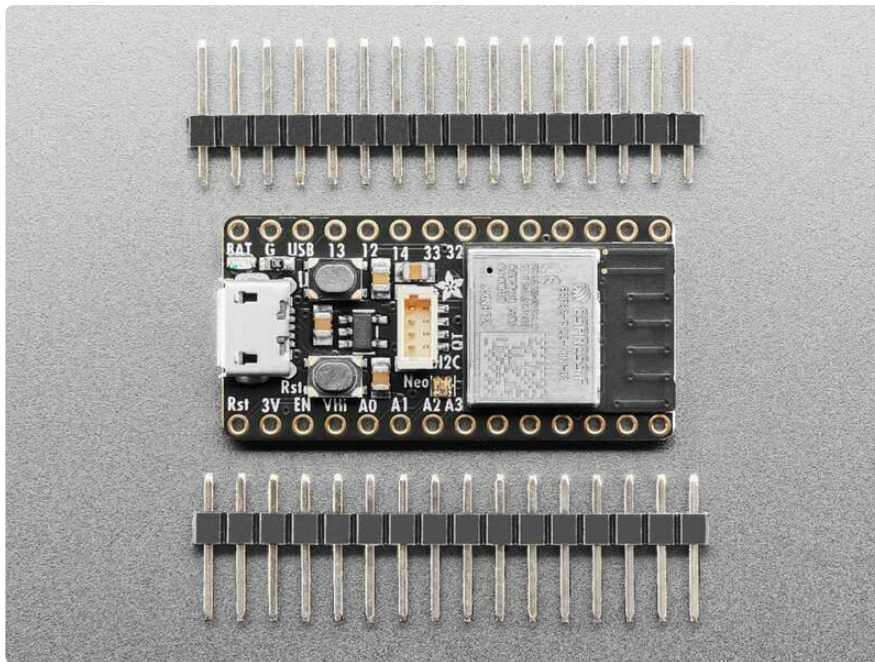


this superhero chip for any task you want to throw it at. This is also an **ultra low power ESP32 board** with a **deep sleep current consumption of 10uA!**



At the core of the Itsy is the ESP32 (PICO ECO V3) chip, which is a single 2.4 GHz Wi-Fi and Bluetooth combo chip designed with TSMC's 40 nm low-power technology. The ESP32 PICO, in particular, integrates all peripheral components seamlessly, including a crystal oscillator, flash, PSRAM, filter capacitors, and RF matching links in one single package. This makes it perfect for stuffing into a small space as the ItsyBitsy.

**Please note**, like other ESP32 modules, the ItsyBitsy ESP32 does not have native USB support - instead, there's a USB to serial converter chip. This means it cannot act like a USB keyboard or mouse, but it does have Bluetooth BLE and BT classic, so you can use it wirelessly.



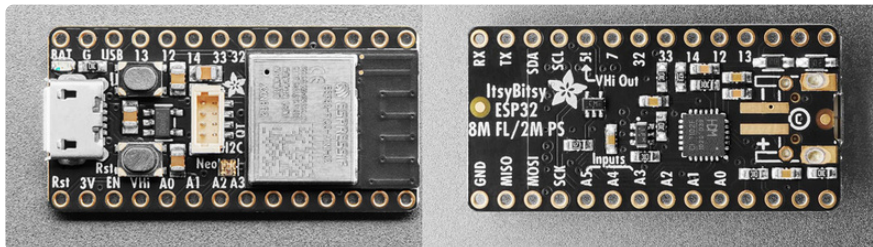
Here are some of the updates you can look forward to when using ItsyBitsy ESP32:

- [Same size and form-factor as other ItsyBitsy mainboards \(https://adafru.it/IIa\)](https://adafru.it/IIa) - with a similar, but not identical, pinout (there are no pins at the end of the board like most other Itsy's due to the radio antenna being there).
- **USB Micro B** - To maintain compatibility with the rest of the ItsyBitsy's.
- **ESP32 V2 03 Dual Core 240MHz Xtensa processor** - the ESP32 you know and love, with the latest engineering fixes. Massive user base and thousands of existing projects and libraries to use.
- WiFi, Bluetooth LE, and BT Classic for any IoT project usage
- **8 MB Flash & 2 MB PSRAM**
- USB to Serial converter built-in with high-speed UART for debugging and uploading, auto-reset circuit works perfectly with any ESP32 uploading tool.
- Can be used with **Arduino IDE, CircuitPython** or **MicroPython**.
- **Built-in RGB NeoPixel LED** with power control to reduce quiescent power in deep sleep.
- **Built-in Red LED** on pin D13.
- **5V level-shifted output on D5**, perfect for driving NeoPixels or other devices that want a 5V logic signal.
- Battery input pads on the underside with diode protection for external battery packs up to 6V input
- **20 General Purpose "IO" pins:**
  - 20 Pads expose pins from the ESP32:
    - 3 are analog input only (A3, A4, A5)
    - 1 is digital output only (5) with 5V level shifted up.

- 13 x 12-bit analog inputs (A0, A1, A2, A3, A4, A5, D12, D13, D14, SDA, SCL, D32, D33).
  - Dual 8-bit analog output DACs on A0/A1
  - PWM outputs on any pin.
  - I2C port with STEMMA QT plug-n-play connector - a second I2C port can be defined on any other pins.
  - Hardware UART in addition to the USB-serial UART.
  - Hardware SPI on the high speed SPI. peripheral pins - a second SPI port can be defined on any other pins.
  - Hardware I2S on any pins.
  - 8 x Capacitive Touch with no additional components required.
- 3.3V regulator with **600mA peak output** (<https://adafru.it/NC4>).
  - Light sleep at 4mA, deep sleep at ~10uA.
  - **Reset switch** for starting your project code over, for entering bootloader mode or for user reading.
  - **User switch** on pin 35.
  - **Really small**

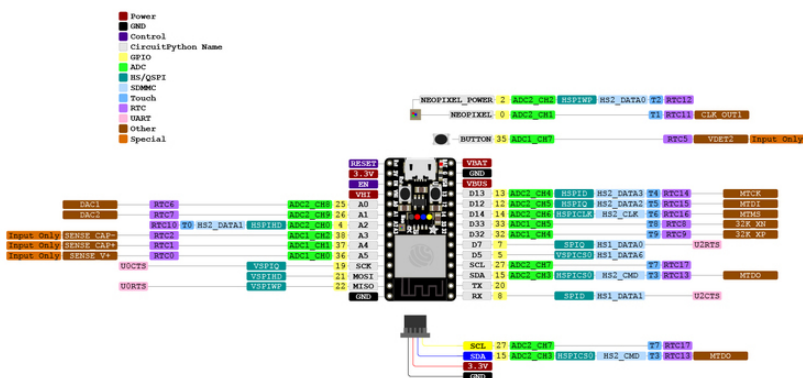
Each order comes with one assembled and tested ItsyBitsy ESP32 with headers that can be soldered in for use with a breadboard.

## Pinouts



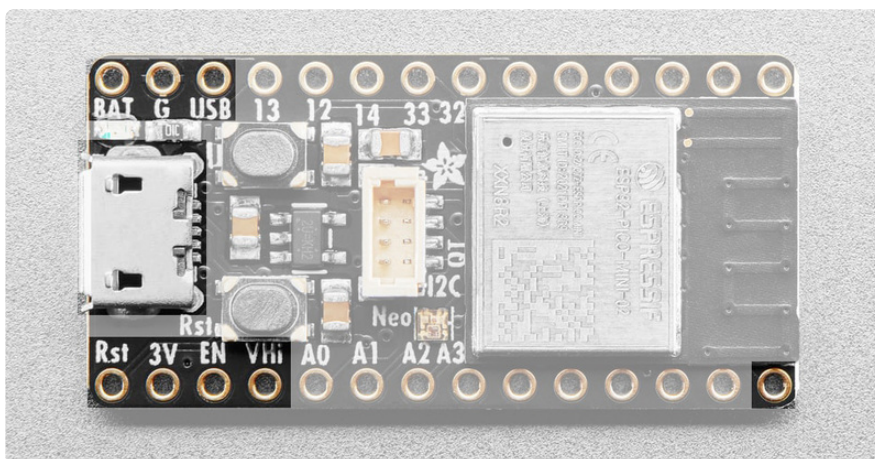
The Adafruit ItsyBitsy ESP32 is jam-packed with pins, connectors and buttons. This page covers all the features!





PrettyPins PDF [on GitHub](https://adafru.it/19jb) (<https://adafru.it/19jb>).

## Power Pins



The ItsyBitsy ESP32 has a Micro B USB connector on the left in the image above. This connector is used to power and program the board. Use a Micro B USB cable to plug the ItsyBitsy into your computer.

Along with the Micro B USB connector, the ItsyBitsy has eight power-related pins.

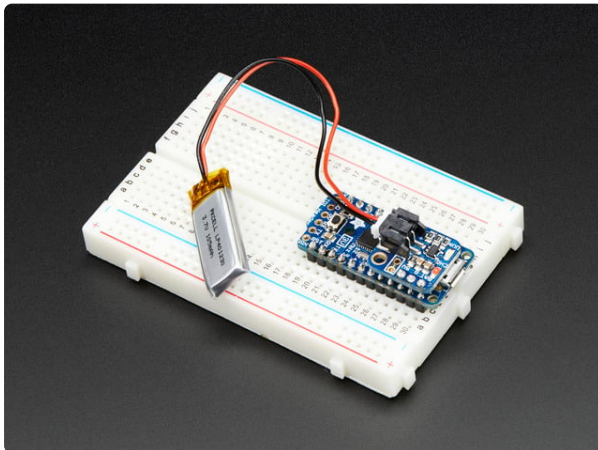
Along the top edge:

- **BAT** - battery input for an alternative power source to USB, the voltage can only be from **3.5V to 6VDC**
- **G** - This is ground for power and data. There is a second ground pin available at the end of the bottom edge.

- **USB** - This is the same pin as the Micro B USB connector's 5V USB power pin. This should be used as an output to get 5V power from the USB port.

You can always put any voltage from **3.5V to 6VDC** into **BAT** and the circuitry will switch between **BAT** and **USB** dynamically for you. That means you can have a Battery backup that only gets enabled when USB is disconnected.

If you want to add rechargeable power, a LiPoly backpack can be soldered into these three pins that will let you have a battery that is automatically recharged whenever USB is plugged in, then switches to LiPoly when on the go:



#### [Adafruit Lilon/LiPoly Backpack Add-On for Pro Trinket/ItsyBitsy](https://www.adafruit.com/product/2124)

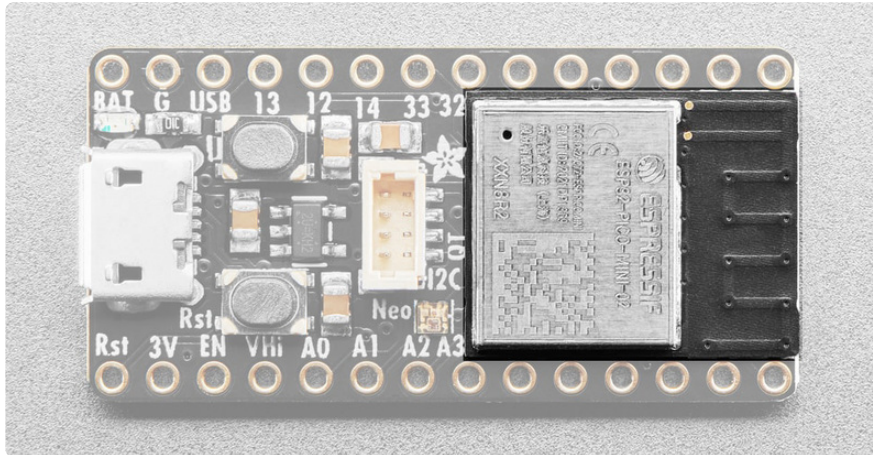
If you have an ItsyBitsy or Pro Trinket you probably know it's the perfect little size for a portable project. This LiPoly backpack makes it really easy to do! Instead of wiring 2...

<https://www.adafruit.com/product/2124>

Along the bottom edge:

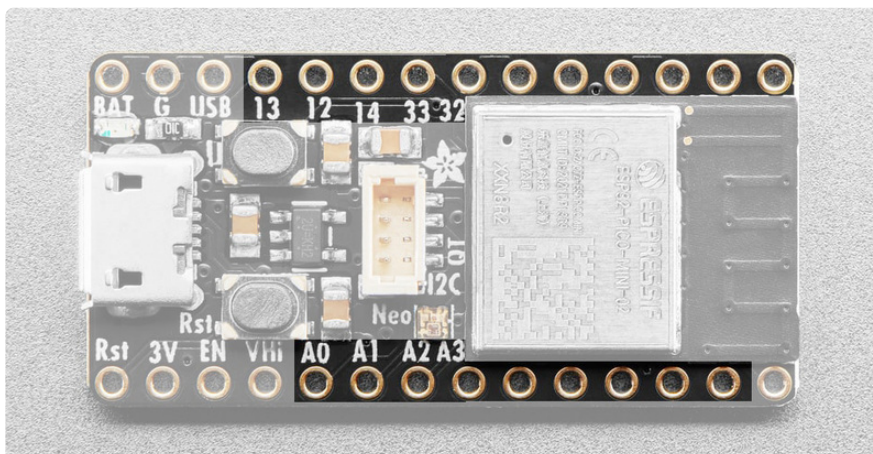
- **Rst** - This is the reset pin. Tie to ground to manually reset the board.
- **3.3V** - This pin is the regulated output from the onboard regulator. You can draw 500mA whether powered by USB or battery.
- **EN** - This pin is connected to the regulator enable. It will let you shut off power whether powered from **BAT**tery or **USB**
- **VHi** - This is a special pin! It is a dual-Schottky diode connected output from **BAT** and **USB**. This means this will always have the higher-of-the-two voltages, but will always have power output. The voltage will be about 5VDC when powered by USB, but can range from 3.5-6VDC when powered from battery. It's not regulated, but it is high-current, great for driving servos and NeoPixels.
- **GND** - This is ground for power and data.

# ESP32 Module



The processor on the ItsyBitsy ESP32 is an **ESP32 V2 03 Dual Core 240MHz Xtensa**. It's the ESP32 you know and love, with the latest engineering fixes. It has a massive user base and thousands of existing projects and libraries to use. WiFi, Bluetooth LE, and BT Classic for any IoT project usage with 8 MB Flash and 2 MB PSRAM.

## Logic Pins



The ItsyBitsy ESP32 has 20 general purpose "IO" pins. You can have a PWM output on any pin, as well as hardware I2S on any pin.

## Bottom Row

- **A0/GPIO25** - This is DAC1 (8-bit analog output). It uses ADC2.
- **A1/GPIO26** - This is also DAC2 (8-bit analog output). It uses ADC2.
- **A2/GPIO4** - It uses ADC2. One of the capacitive touch pins.
- **A3/GPIO38** - It is input/ADC only. It uses ADC1.
- **A4/GPIO37** - It is input/ADC only. It uses ADC1.
- **A5/GPIO36** - It is input/ADC only. It uses ADC2.
- **SCK/GPIO19** - This is the SPI clock pin.
- **MOSI/GPIO21** - This is the SPI Microcontroller Out / Serial In (MOSI) pin.

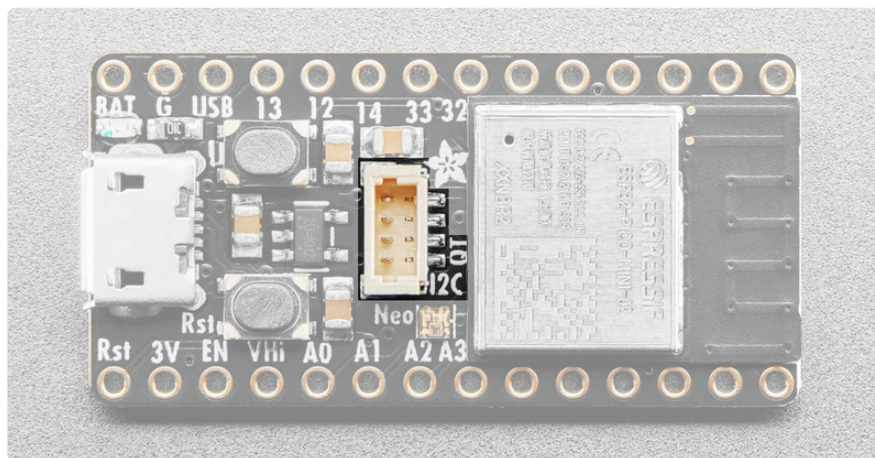
- **MISO/GPIO22** - This is the SPI Microcontroller In / Serial Out (MISO) pin.

## Top Row

- **D13/GPIO13** - It uses ADC2. This pin is shared with the built-in LED. One of the capacitive touch pins.
- **D12/GPIO12** - It uses ADC2. One of the capacitive touch pins.
- **D14/GPIO14** - It uses ADC2. One of the capacitive touch pins.
- **D33/GPIO33** - It uses ADC1. One of the capacitive touch pins.
- **D32/GPIO32** - It uses ADC1. One of the capacitive touch pins.
- **D7/GPIO7** - Digital input/output only.
- **D5/GPIO5** - Digital output only. This is a special pin that is level-shifted up to **Vhi** voltage, so it's perfect for driving NeoPixels that want a ~5V logic level input.
- **SCL/GPIO27** - This is the I2C clock pin. One of the capacitive touch pins.
- **SDA/GPIO15** - This is the I2C data pin. One of the capacitive touch pins.
- **TX/GPIO20** - This is the UART TX (transmit) pin. Connect to the RX pin found on a breakout or device. This is separate than the 'debug UART' which is connected to the USB-to-Serial converter, so it will not interfere during upload. In Arduino, use `Serial1`. In CircuitPython, use `board.TX`.
- **RX/GPIO8** - This is the UART RX (receive) pin. Connect to the TX pin found on a breakout or device. This is separate than the 'debug UART' which is connected to the USB-to-Serial converter, so it will not interfere during upload. In Arduino, use `Serial1`. In CircuitPython, use `board.RX`.

Note you cannot read analog inputs on **ADC2** once WiFi has started, as it is shared with the WiFi hardware.

## STEMMA QT Connector

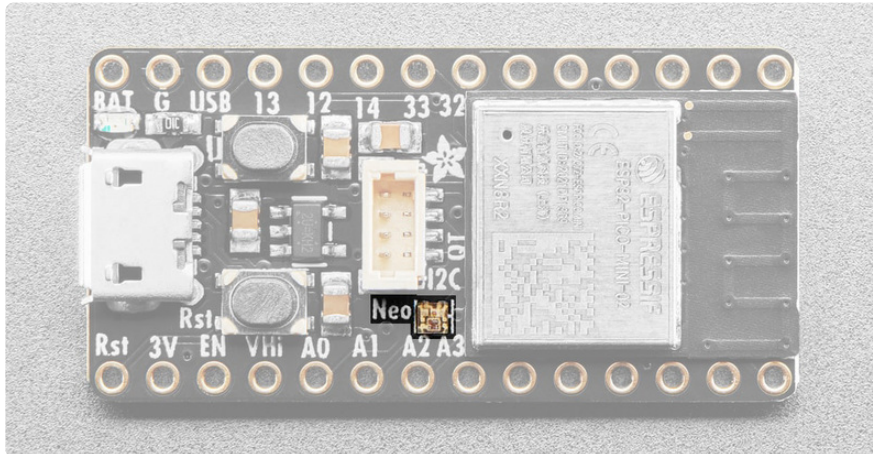


There is a 4-pin **STEMMA QT connector** in the center of the board. The I2C has pullups to 3.3V power.



The I2C pins are connected to GPIO15 (SDA) and GPIO27 (SCL). In Arduino, you can use the STEMMA connector with `Wire`. In CircuitPython, you can use the STEMMA connector with `board.SCL` and `board.SDA`, `board.I2C()` or `board.STEMMA_I2C()`.

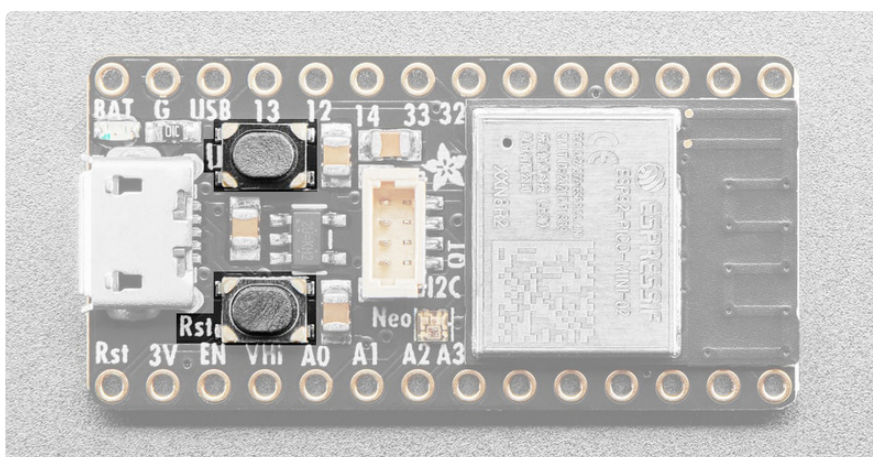
## NeoPixel LED



In the center of the board, to the left of the ESP32 module, is the **NeoPixel**. This addressable RGB LED works both as a status LED (in CircuitPython and the bootloader), and can be controlled with code. It is connected to GPIO0 and is available in CircuitPython as `board.NEOPIXEL` and in Arduino as `PIN_NEOPIXEL`.

There is a NeoPixel power pin on GPIO2 that is necessary for the NeoPixel to work. This is so it can be fully de-powered for low power usage. It is available in CircuitPython as `board.NEOPIXEL_POWER` and in Arduino as `NEOPIXEL_POWER`.

## Buttons



There are two buttons on the ItsyBitsy ESP32:

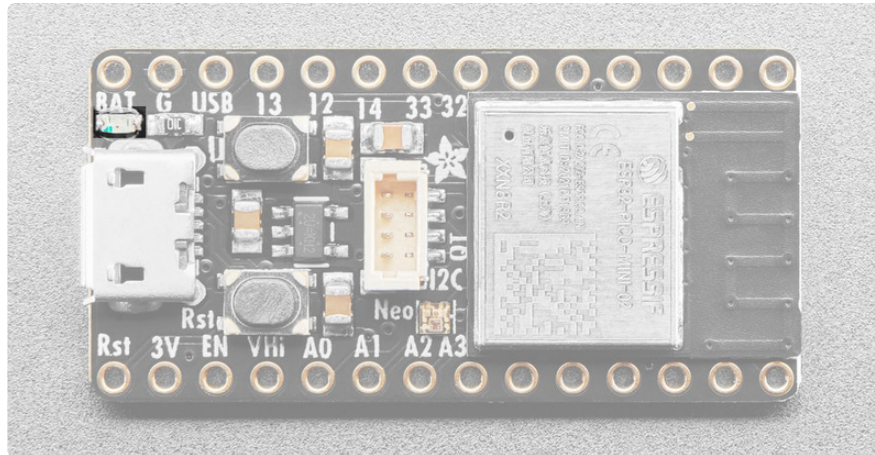
- **User Switch** - This is a user readable button switch to use as an input, labeled **U** on the silk. You can read it on pin GPIO 35. Note this is an input-only pin. There



is a pull-up on board. It is available in CircuitPython as `board.BUTTON`, and in Arduino as `BUTTON`.

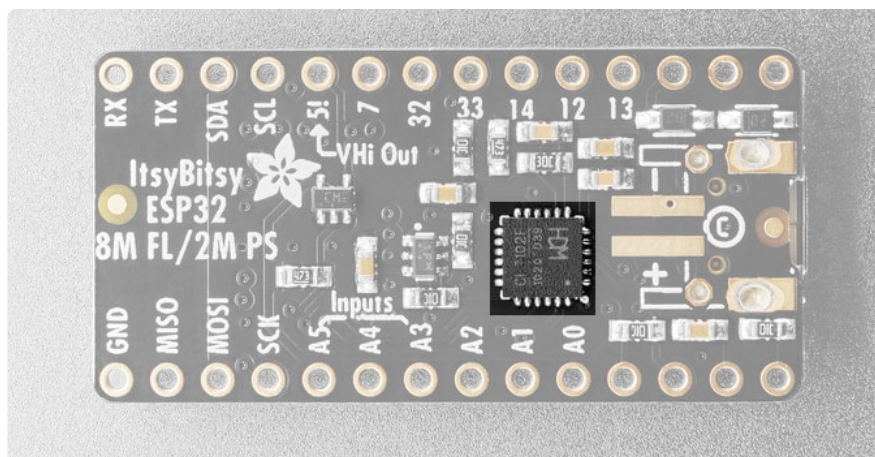
- **Reset** - The reset button, labeled **Rst** on the silk, is used to reboot the board.

## Red LED



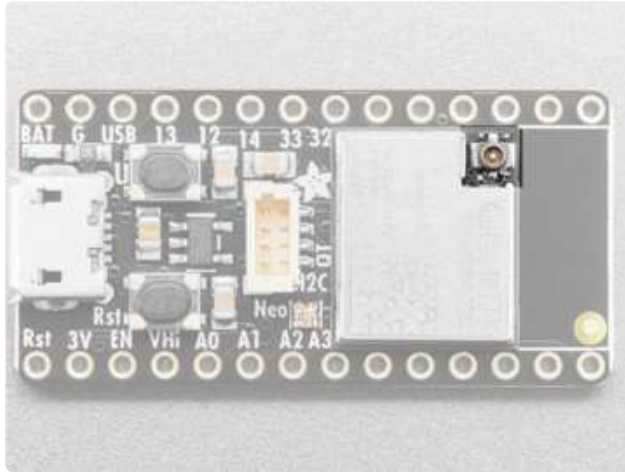
The onboard red LED is located directly below the BAT pin on the front of the board. It is connected to GPIO13 and can be controlled in code like any LED, set high to turn on. It is available in CircuitPython as `board.LED`, and Arduino as `LED_BUILTIN`.

## CH9102 USB-to-Serial Converter



The **CH9102F USB to serial converter** communicates between the ESP32 and your computer over USB. It can handle 50bps to 4Mbps max rate for debugging and uploading, with auto-reset circuit that works perfectly with any ESP32 uploading tool. Sometimes these chips require drivers to be installed on your computer's operating system. We have a [Learn Guide detailing how to install the driver for the CH9102F](https://adafruit.it/19jc) (<https://adafruit.it/19jc>).

## w.FL Antenna Port



The [Adafruit ItsyBitsy ESP32 with w.FL Antenna](http://adafru.it/5890) (<http://adafru.it/5890>) comes with, you guessed it, a w.FL antenna port! **This is the only version that has the port!**

**It does NOT come with an antenna, you must purchase one separately.** Consider the [WiFi Antenna with w.FL](http://adafru.it/5445) (<http://adafru.it/5445>) or a [RP-SMA to w.FL adapter](http://adafru.it/5444) (<http://adafru.it/5444>).

---

## Low Power Use

This microcontroller board can be used for low power usage thanks to the ESP32's multiple sleep modes.

There are three basic operating states to Espressif chips: normal, light sleep and deep sleep.

**Normal power** usage is as you expect: you can use the chip and run code as you like - connecting to WiFi, reading sensors, etc.

**Light sleep** is sort of a 'hibernation' - power usage is minimal and WiFi is disconnected, but the internal clock and memory is kept. That means you can wake up where you left off, in the middle of the code as desired. You'll still need to re-initialize any external hardware that got disconnected, and WiFi, but it's often faster than waking from a deep sleep

**Deep sleep** is the lowest power but the tradeoff is that all memory and state is lost - the only thing that's running is the real time clock that can wake the chip up. When woken up, the chip starts as if it was physically reset - from the beginning of the code. This can be beneficial if you want to have a fresh start each time

A rough guideline is:

- Normal power: 100mA+ can be as much power as need and spike during WiFi connection
- Light sleep: 2mA assuming all external hardware is de-powered
- Deep sleep: 100uA assuming all external hardware is de-powered

The Adafruit ItsyBitsy ESP32 has a **NEOPIXEL\_POWER** pin that controls power to the NeoPixel LED. This pin is automatically pulled HIGH in both CircuitPython and Arduino. Disabling this pin by setting it to an output and LOW allows you to drop the power draw.

Here's a generic sketch we use for all our boards that has a macro-defined section for enabling and disabling all external powered elements. For example, if there's a power pin for NeoPixels, I2C port, TFT, etc...we turn that off before going into light or deep sleep! This will minimize power usage

```
// SPDX-FileCopyrightText: 2022 Limor Fried for Adafruit Industries
//
// SPDX-License-Identifier: MIT

#include <Adafruit_NeoPixel.h>

// While we wait for Feather ESP32 V2 to get added to the Espressif BSP,
// we have to select PICO D4 and UNCOMMENT this line!
// #define ADAFRUIT_FEATHER_ESP32_V2

// then these pins will be defined for us
#if defined(ADAFRUIT_FEATHER_ESP32_V2) or defined(ARDUINO_ADAFRUIT_ITSYBITSY_ESP32)
#define PIN_NEOPIXEL_0
#define NEOPIXEL_I2C_POWER 2
#endif

#if defined(PIN_NEOPIXEL)
  Adafruit_NeoPixel pixel(1, PIN_NEOPIXEL, NEO_GRB + NEO_KHZ800);
#endif

void setup() {
  Serial.begin(115200);

  // Turn on any internal power switches for TFT, NeoPixels, I2C, etc!
  enableInternalPower();
}

void loop() {
  LEDon();
  delay(1000);

  disableInternalPower();
  LEDoff();
  esp_sleep_enable_timer_wakeup(1000000); // 1 sec
  esp_light_sleep_start();
  // we'll wake from light sleep here

  // wake up 1 second later and then go into deep sleep
  esp_sleep_enable_timer_wakeup(1000000); // 1 sec
  esp_deep_sleep_start();
  // we never reach here
}

void LEDon() {
#if defined(PIN_NEOPIXEL)
  pixel.begin(); // INITIALIZE NeoPixel
  pixel.setBrightness(20); // not so bright
  pixel.setPixelColor(0, 0xFFFFFF);
  pixel.show();
#endif
}
```

```

void LEDoff() {
  #if defined(PIN_NEOPIXEL)
    pixel.setPixelColor(0, 0x0);
    pixel.show();
  #endif
}

void enableInternalPower() {
  #if defined(NEOPIXEL_POWER)
    pinMode(NEOPIXEL_POWER, OUTPUT);
    digitalWrite(NEOPIXEL_POWER, HIGH);
  #endif

  #if defined(NEOPIXEL_I2C_POWER)
    pinMode(NEOPIXEL_I2C_POWER, OUTPUT);
    digitalWrite(NEOPIXEL_I2C_POWER, HIGH);
  #endif

  #if defined(ARDUINO_ADAFRUIT_FEATHER_ESP32S2)
    // turn on the I2C power by setting pin to opposite of 'rest state'
    pinMode(PIN_I2C_POWER, INPUT);
    delay(1);
    bool polarity = digitalRead(PIN_I2C_POWER);
    pinMode(PIN_I2C_POWER, OUTPUT);
    digitalWrite(PIN_I2C_POWER, !polarity);
    pinMode(NEOPIXEL_POWER, OUTPUT);
    digitalWrite(NEOPIXEL_POWER, HIGH);
  #endif
}

void disableInternalPower() {
  #if defined(NEOPIXEL_POWER)
    pinMode(NEOPIXEL_POWER, OUTPUT);
    digitalWrite(NEOPIXEL_POWER, LOW);
  #endif

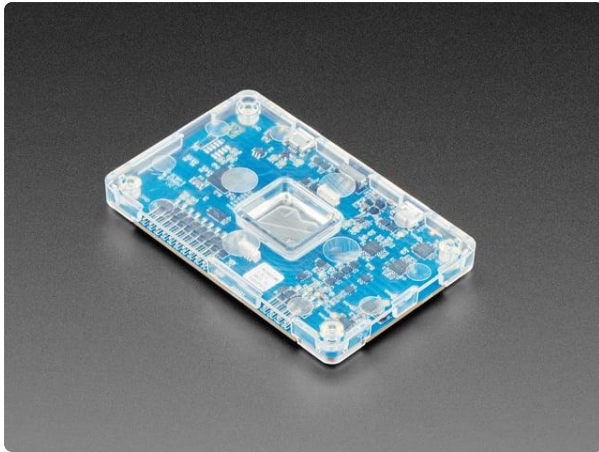
  #if defined(NEOPIXEL_I2C_POWER)
    pinMode(NEOPIXEL_I2C_POWER, OUTPUT);
    digitalWrite(NEOPIXEL_I2C_POWER, LOW);
  #endif

  #if defined(ARDUINO_ADAFRUIT_FEATHER_ESP32S2)
    // turn off the I2C power by setting pin to rest state (off)
    pinMode(PIN_I2C_POWER, INPUT);
    pinMode(NEOPIXEL_POWER, OUTPUT);
    digitalWrite(NEOPIXEL_POWER, LOW);
  #endif
}

```

The best way to really test power draw is with a specialty power meter such as the Nordic PPK 2





## Nordic nRF-PPK2 - Power Profiler Kit II

The Power Profiler Kit II is a standalone unit, which can measure and optionally supply currents all the way from sub-uA and as high as 1A on all Nordic DKs, in...  
<https://www.adafruit.com/product/5048>

When running the above code and monitoring with a PPK, you'll get a graph like this:



The big pulse is normal mode, you can see the ESP32 booting up, loading code, and then pausing 1 second. Then there's a big drop for one sec to light sleep, and finally one more 1 second pause at deep sleep.

## Power Draw for ItsyBitsy ESP32

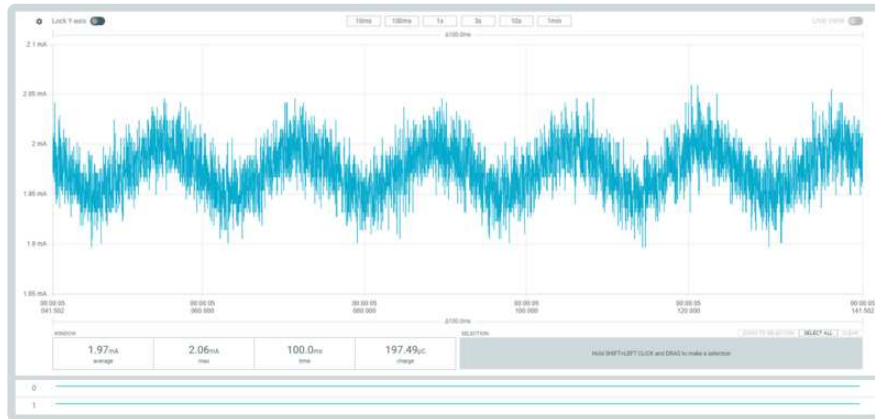
The following graphs show the power draw for the ItsyBitsy ESP32 in normal power mode, light sleep mode, and deep sleep mode.

### Normal Power Mode



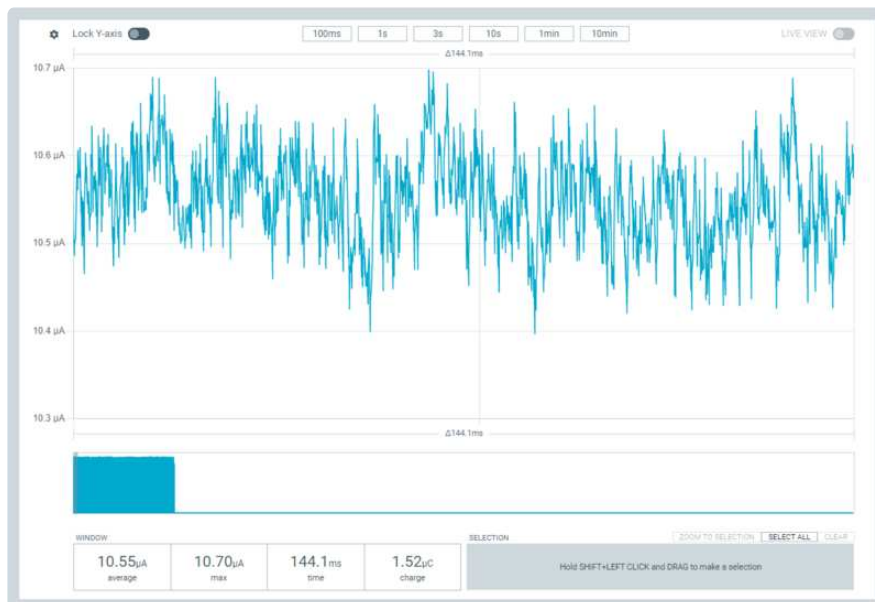
The power draw, running normally (without WiFi), is 41.9mA.

## Light Sleep Mode



The power draw in light sleep mode is 1.97mA.

## Deep Sleep Mode



The power draw in deep sleep mode is 10.55uA.

---

## CircuitPython Setup

[CircuitPython](https://adafru.it/tB7) (<https://adafru.it/tB7>) is a derivative of [MicroPython](https://adafru.it/BeZ) (<https://adafru.it/BeZ>) designed to simplify experimentation and education on low-cost microcontrollers. It makes it easier than ever to get prototyping by requiring no upfront desktop software downloads. ESP32 CircuitPython firmware is uploaded to the board via the USB serial port.

Follow this step-by-step to get CircuitPython running on your board.

## Driver Install

If this is your first time using an ESP32 board on Windows or MacOS, you may need to install the USB to serial drivers. There are two options for the USB to serial chipset on your board. If you are unsure which one you have, install both drivers.

For instructions and more information regarding the CH9102F chip and driver install, please visit the [How To Install Drivers for WCH USB to Serial Chips guide \(https://adafruit.it/-f8\)](https://adafruit.it/-f8).

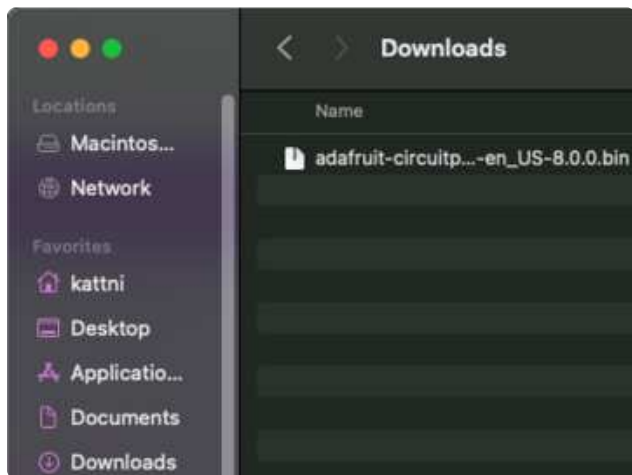
For driver downloads for the CP2104 and CP2012N, please visit the [Silicon Labs Downloads page \(https://adafruit.it/11em\)](https://adafruit.it/11em).

For those running Linux, the drivers are already included.

## CircuitPython Download

Download the latest version of  
CircuitPython for this board via  
circuitpython.org

<https://adafruit.it/19jd>



Click the link above to download the latest CircuitPython .bin file.

Save it wherever is convenient for you.

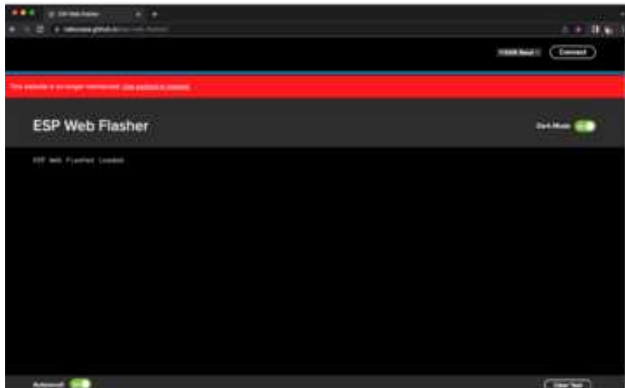
## Connecting to the Web Flasher

To begin, **plug your board into your computer via USB**, using a known-good data-sync cable, directly, or via an adapter if needed.

You will have to use the Chrome or a Chromium-based browser to install CircuitPython. [For example, Edge and Opera are Chromium based \(https://adafruit.it/10BL\)](https://adafruit.it/10BL).

Safari and Firefox, etc are not supported - [they have not implemented Web Serial \(https://adafruit.it/10BM\)](https://adafruit.it/10BM)!

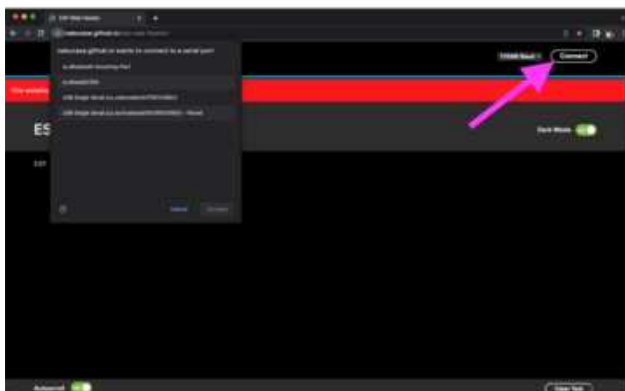
In the **Chrome** browser visit [https://adafruit.github.io/Adafruit\\_WebSerial\\_ESPTool/](https://adafruit.github.io/Adafruit_WebSerial_ESPTool/) (<https://adafru.it/PMB>)



The main page of the ESP Web Flasher should look something like this.

Note: The site now displays an alert that it is no longer maintained, and suggests using a different option. The ESP Web Flasher has still proven to be more consistent and easier to use, so it is highly suggested that you continue with this version.

You should remove all other **USB** devices so **only the target board is attached**. This eliminates confusion over multiple ports!

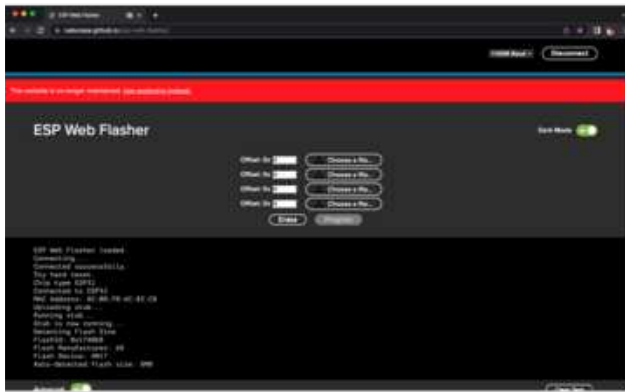


Press the **Connect** button in the top right of the web browser. You will get a pop up asking you to select the COM or Serial port. Look for **USB Single Serial**.

On some systems, such as MacOS, there may be additional system ports that appear in the list (as shown in the image).

```
ESP Web Flasher loaded.
Connecting...
Connected successfully.
Try hard reset.
Chip type ESP32
Connected to ESP32
MAC Address: AC:0B:FB:6C:83:C8
Uploading stub...
Running stub...
Stub is now running...
Detecting Flash Size
FlashId: 0x174068
Flash Manufacturer: 68
Flash Device: 4017
Auto-detected Flash size: 8MB
```

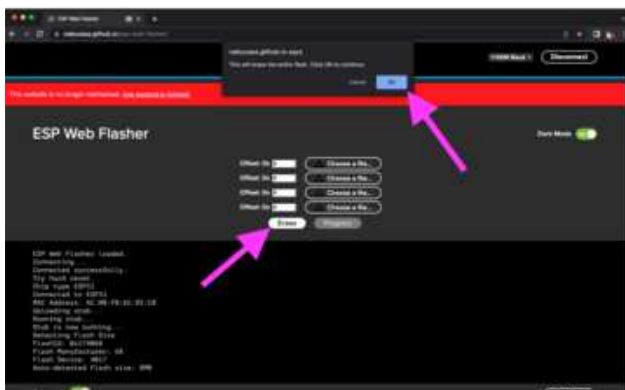
The Javascript code will now try to connect to the board. It may timeout for a bit until it succeeds. On success, you will see that it is **Connected** and will print out a unique **MAC address** identifying the board along with other information that was detected.



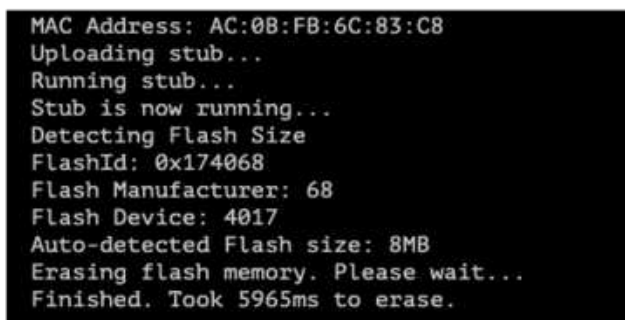
Once you have successfully connected, the command toolbar will appear.

## Erasing the Board Contents

If you would like to erase the entire flash area so that you can start with a clean slate, you can use the erase feature. We recommend doing this every time before installing or updating CircuitPython.



To erase the contents, click the **Erase** button. You will be prompted as to whether you want to continue. Click **OK** to continue. If you do not wish to continue, click Cancel.



You'll see "Erasing flash memory. Please wait..." This will eventually be followed by "Finished." and the amount of time it took to erase.

**Do not disconnect!** Immediately continue on to Programming the Board.

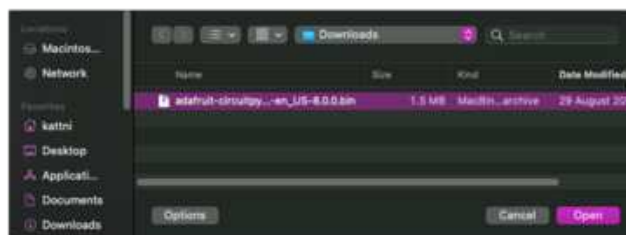


Do not disconnect after erasing! You should immediately continue on to programming your board. If you do not, you may end up with your board in a bad state that makes it more difficult to continue. You can avoid this!

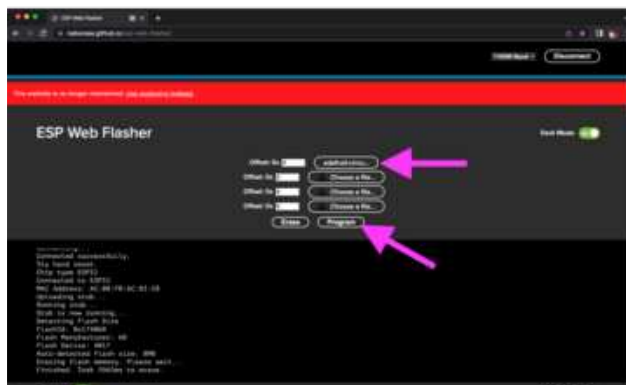
## Programming the Board



You can click on **Choose a file...** from any of the available buttons. It will only attempt to program buttons with a file and a unique location. Select the **.bin** file you downloaded at the beginning of this page from the file chooser dialogue.



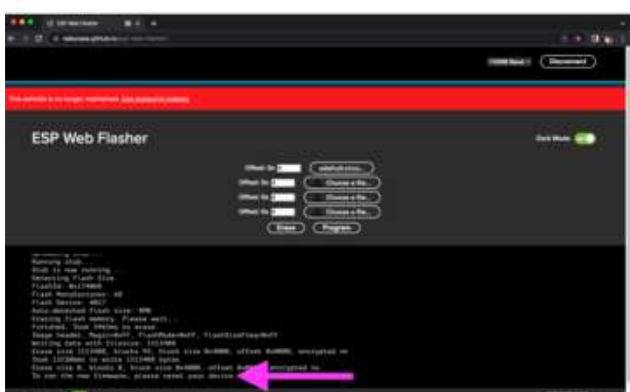
Verify that the **Offset** box next to the file location you used is 0x0. The offset defaults to 0x0, so unless you changed it manually, it should be good to go.



Once you choose a file, the button text will change to match your filename. You can then click the **Program** button to start flashing.



A progress bar will appear and after a minute or two, you will have written the firmware.



You've now successfully programmed CircuitPython onto your board! As suggested in the output, press reset to run the new firmware.

As the ESP32 does not have native USB, no USB drive will show up on your computer when you reset. With CircuitPython firmware loaded, the REPL can be accessed over a serial/COM port.

For more details on installation, how to configure your ESP32, and info on getting started with CircuitPython on your ESP32 using the Web Workflow, check out the [CircuitPython on ESP32 Quick Start guide \(https://adafruit.it/10JF\)](https://adafruit.it/10JF).

## Setting up Web Workflow

Web workflow is enabled when a file named **settings.toml** is added to the root folder of the CircuitPython file system. This file contains local wifi info and other settings. [More info here \(https://adafruit.it/10Bp\)](https://adafruit.it/10Bp).

Now, normally creating a file is super easy when the board shows up as a disk drive...but as mentioned before, that's not possible on the original ESP32 because it does not have native USB so it cannot show up as a disk drive. So we have to be a little more creative!

To start out, the minimal contents of this file are:

```
CIRCUITPY_WIFI_SSID = "wifissid"
CIRCUITPY_WIFI_PASSWORD = "wifipassword"
CIRCUITPY_WEB_API_PASSWORD= "webpassword"
```

with each item updated with local specifics.

- **wifissid** - replace with local wifi network name
- **wifipassword** - replace with local wifi network password
- **webpassword** - used when connecting to the board via web browser, set to whatever

There are a couple of options for creating this file.

Creating the settings.toml file will also enable the wifi hardware, which may be off by default.

## Option 1: Create **settings.toml** file via REPL

The **settings.toml** file is simple enough that it can be created with a few simple Python commands directly via the REPL.

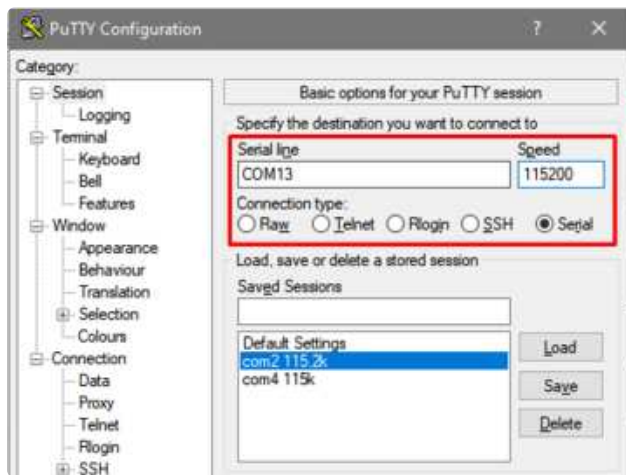
What is the REPL?

<https://adafru.it/Awz>

Here are the basic commands to use. **Note these need to be updated with local wifi specifics. Note also where double quotes and single quotes are used.**

```
f = open('settings.toml', 'w')
f.write('CIRCUITPY_WIFI_SSID = "wifissid"\n')
f.write('CIRCUITPY_WIFI_PASSWORD = "wifipassword"\n')
f.write('CIRCUITPY_WEB_API_PASSWORD = "webpassword"\n')
f.close()
```

- Replace **wifissid** with the name of your local wifi network
- Replace **wifipassword** with your local wifi password
- The other password, **webpassword**, is used when you access the board via a web browser. Set this to whatever you want.



If you have a terminal program like PuTTY, minicom, screen or similar and you know how to use them - connect to the ESP32 board at the correct port name and 115200 baud

Once connected, you can press the **Reset** button to kick the firmware, then hit return a few times to get to the REPL prompt. Now you can copy and paste the lines above (with your correct SSID/password!

Don't forget, ESP32 does not support 5 GHz networks, so use your 2.4 GHz SSID if you have two.

```

Board ID:adafruit_feather_esp32_v2
UID:C45752EBF2CA

Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.
code.py output:
Hello World!

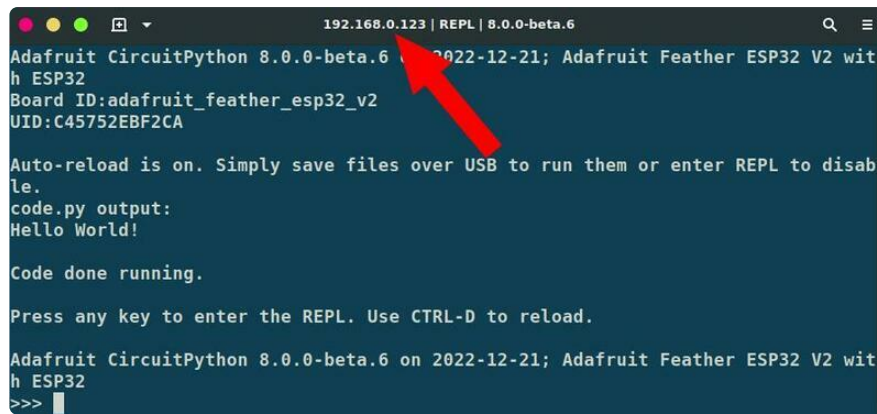
Code done running.

Press any key to enter the REPL. Use CTRL-D to reload.

Adafruit CircuitPython 8.0.0-beta.6 on 2022-12-21; Adafruit Feather ESP32 V2 with ESP32
>>> f = open('settings.toml', 'w')
>>> f.write('CIRCUITPY_WIFI_SSID = " " \n')
31
>>> f.write('CIRCUITPY_WIFI_PASSWORD = " " \n')
37
>>> f.write('CIRCUITPY_WEB_API_PASSWORD = "esp32" \n')
37
>>> f.close()
>>>

```

Now press the **Reset** button again, you will see the ESP32 reboot. This time, it should get an IP address. If you have a fairly smart terminal program, the IP address will appear in the title bar.



```
Adafruit CircuitPython 8.0.0-beta.6 on 2022-12-21; Adafruit Feather ESP32 V2 with ESP32
Board ID:adafruit_feather_esp32_v2
UID:C45752EBF2CA

Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.
code.py output:
Hello World!

Code done running.

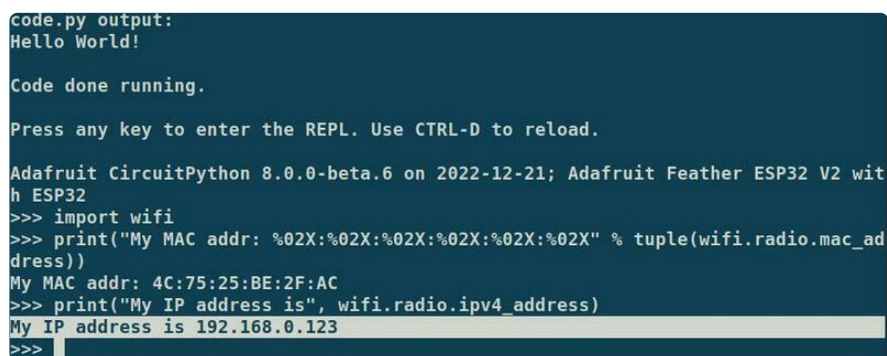
Press any key to enter the REPL. Use CTRL-D to reload.

Adafruit CircuitPython 8.0.0-beta.6 on 2022-12-21; Adafruit Feather ESP32 V2 with ESP32
>>>
```

Alternatively, or if you want more deets - you can always query the board over the REPL to ask it the MAC address and IP address:

```
import wifi

print("My MAC addr: %02X:%02X:%02X:%02X:%02X:%02X" % tuple(wifi.radio.mac_address))
print("My IP address is", wifi.radio.ipv4_address)
```



```
code.py output:
Hello World!

Code done running.

Press any key to enter the REPL. Use CTRL-D to reload.

Adafruit CircuitPython 8.0.0-beta.6 on 2022-12-21; Adafruit Feather ESP32 V2 with ESP32
>>> import wifi
>>> print("My MAC addr: %02X:%02X:%02X:%02X:%02X:%02X" % tuple(wifi.radio.mac_address))
My MAC addr: 4C:75:25:BE:2F:AC
>>> print("My IP address is", wifi.radio.ipv4_address)
My IP address is 192.168.0.123
>>>
```

If that's not working either, try asking what SSID's it thinks it's seeing to make sure you have no typos!

```
import wifi

print("Available WiFi networks:")
for n in wifi.radio.start_scanning_networks():
    print("\t%s\t\tRSSI: %d\tChannel: %d" % (str(n.ssid, "utf-8"), n.rssi, n.channel))
wifi.radio.stop_scanning_networks()
```

Note that since this code has an indented part in it, you shouldn't just paste it in directly into the REPL because it won't handle the tab/spaces correctly. Instead, once you hit **Return** to get to the REPL, type in **Control-E** to enter paste mode. Then paste in the text and type **Control-D** to finish and run



```
code.py output:
Hello World!

Code done running.

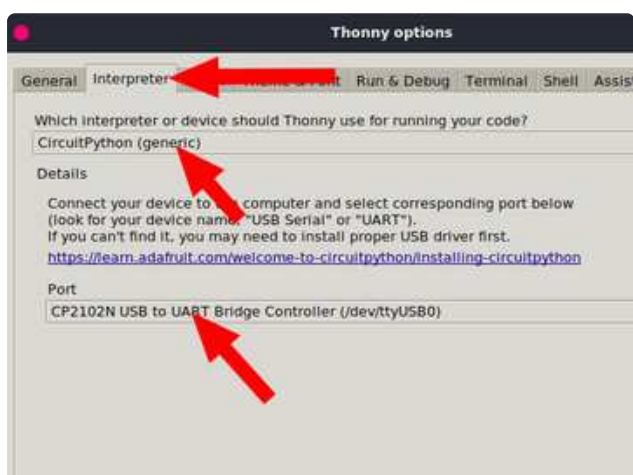
Press any key to enter the REPL. Use CTRL-D to reload.

Adafruit CircuitPython 8.0.0-alpha.1-147-g05d2013b4 on 2022-08-18; Adafruit Feather ESP32 V2 with ESP32
>>>
paste mode; Ctrl-C to cancel, Ctrl-D to finish
=== import wifi
===
=== print("Available WiFi networks:")
=== for n in wifi.radio.start_scanning_networks():
===     print("\t%s\t\tRSSI: %d\tChannel: %d" % (str(n.ssid, "utf-8"), n.rssi, n
=== .channel))
=== wifi.radio.stop_scanning_networks()
===
Available WiFi networks:
adafruit                RSSI: -58          Channel: 6
patrick's Network       RSSI: -76          Channel: 6
Fios-N2VnJ              RSSI: -70          Channel: 1
VVCBR                  RSSI: -88          Channel: 1
ohana                   RSSI: -93          Channel: 1
MySpectrumWiFi73-2G     RSSI: -54          Channel: 11
Sally                   RSSI: -82          Channel: 11
Patrick                 RSSI: -86          Channel: 11
Sonos                   RSSI: -92          Channel: 11
SNET                    RSSI: -93          Channel: 11
Verizon_9DWHL4          RSSI: -96          Channel: 11
linksys_SES_2868        RSSI: -74          Channel: 3
Meatlocker              RSSI: -85          Channel: 5
Fios-K57GI              RSSI: -73          Channel: 7
Fios-K57GI              RSSI: -88          Channel: 7
>>>
```

Check that your SSID appears properly - if not, maybe its 5 GHz, maybe its not typed correctly, etc!

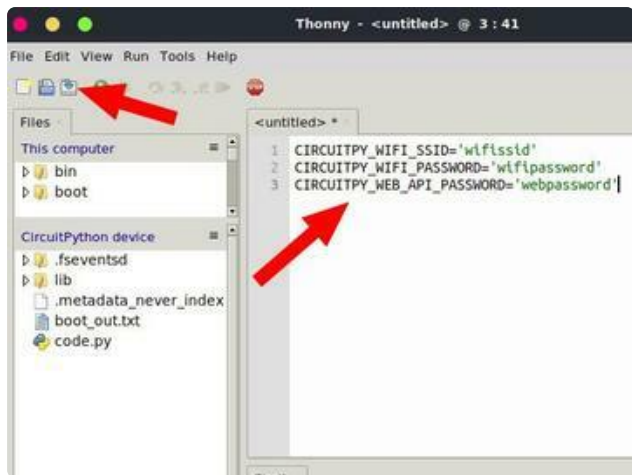
## Option 2: Create **settings.toml** file using Thonny

This option requires installing additional software - the [Thonny Python IDE \(https://adafru.it/Qb6\)](https://adafru.it/Qb6). Thonny provides file access and a text editor which makes creating the **settings.toml** file a little more streamlined than using direct REPL commands. **This technique requires Thonny 4.0.0 or later.**



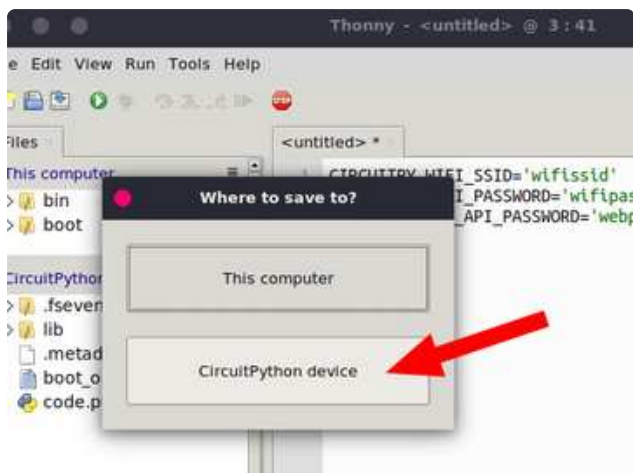
In Thonny, open the **Tools -> Options** dialog and select the **Interpreter** tab.

Set interpreter to **CircuitPython (generic)** and the COM port as needed.

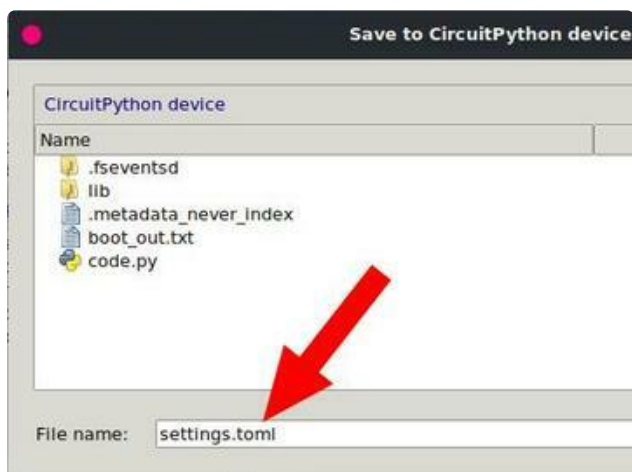


Enter the file contents in the editor window. Update `wifissid` etc. as needed.

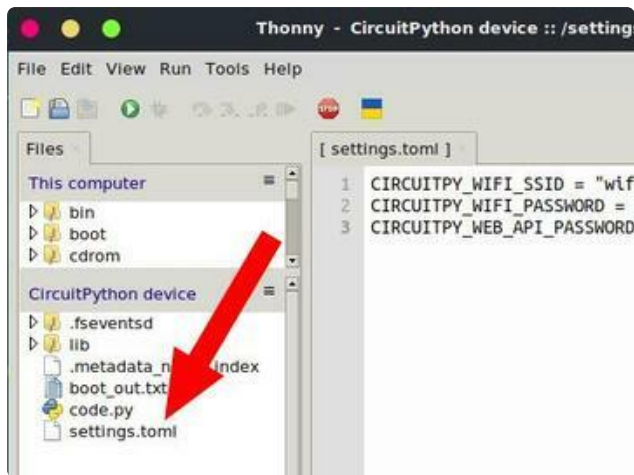
Click save.



Select **CircuitPython Device**



Save the file as **settings.toml**.



Now the **settings.toml** file shows up on the CircuitPython device.

---

## Connecting via Web Browser

Once the web workflow is enabled and the board connects to the local wifi network with an IP address, the board can be accessed by opening a web browser and entering the board's network address. But what is the address to enter?

### Connect using MDNS Address

The CircuitPython web workflow uses [MDNS \(https://adafru.it/10BW\)](https://adafru.it/10BW) so that connecting to the board can be as simple as using the address **circuitpython.local**. Try using that first and if it works, great. Open a web browser and navigate to:

```
http://circuitpython.local
```

Or just click the button below:

**circuitpython.local**

<https://adafru.it/10BX>

### Connect using IP Address

If MDNS does not work for some reason, the fallback approach is to use the actual IP address, like **192.168.0.121**, the CircuitPython board was assigned when it connected to the local wifi network.

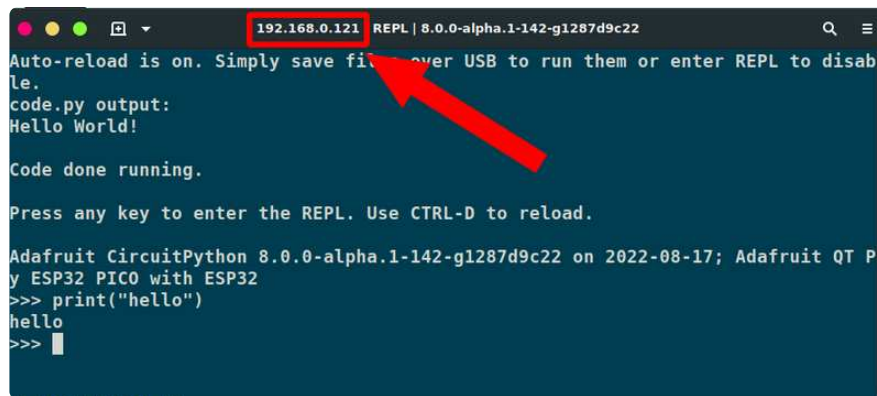
```
http://192.168.0.121
```

There are several options for determining this address.

## Terminal Window Title Bar

The web workflow serial output includes some special escape sequences that are intended to update the window title bar of the terminal program being used.

For example, here's what a terminal window running screen to connect to the board looks like:

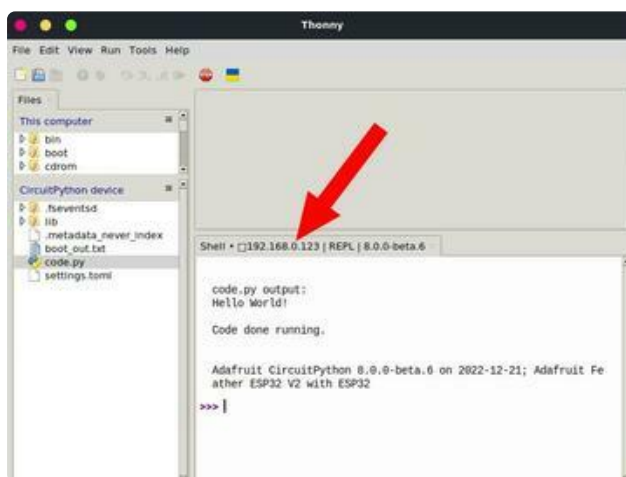


## Via REPL

The IP address can be obtained fairly easily via the REPL with a few commands:

```
>>> import wifi
>>> wifi.radio.ipv4_address
192.168.0.121
>>>
```

## Serial output in Thonny



In Thonny, look for the IP address in the banner text of the serial output in the Shell window.

Note the **settings.toml** file has been added to the CircuitPython device.

# Using Web Workflow

## Welcome! Page

This is the initial page seen when first connecting.



Launch a web browser and navigate to **circuitpython.local** OR the numeric IP address like **192.168.0.121**



The browser should find the board and show the **Welcome!** screen.

## Password Entry

When first accessing things from the **Welcome!** page, a password must be entered.



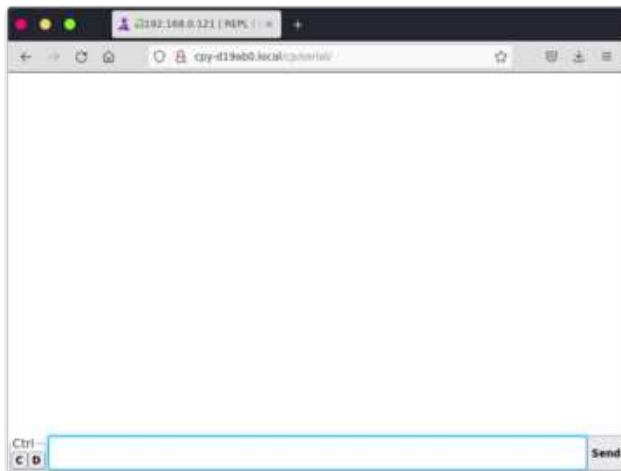
Enter the password that was setup for **CIRCUITPY\_WEB\_API\_PASSWORD** in the **settings.toml** file.

Leave Username blank.

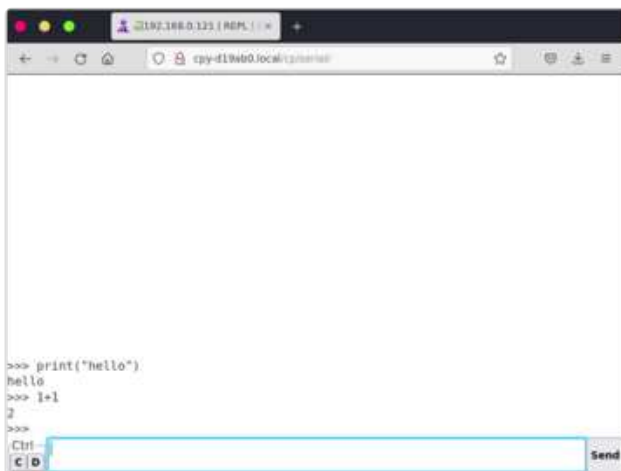
## Serial Terminal

From the Welcome! page, click the **serial terminal** link to access the serial output as well as REPL for entering commands.





The serial window initially looks blank.

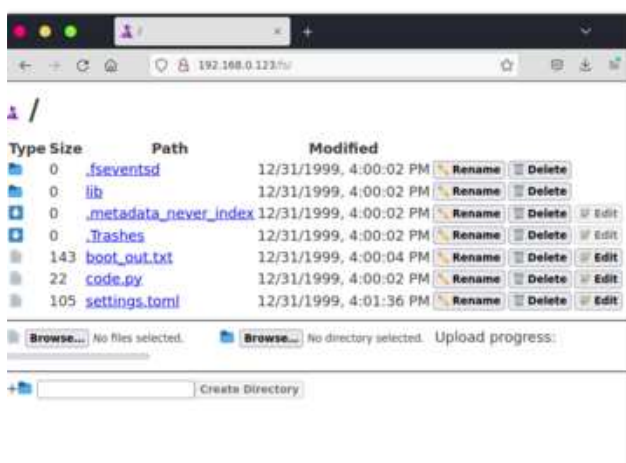


Commands can be entered in the input field at the bottom.

The results will be shown above and scroll up.

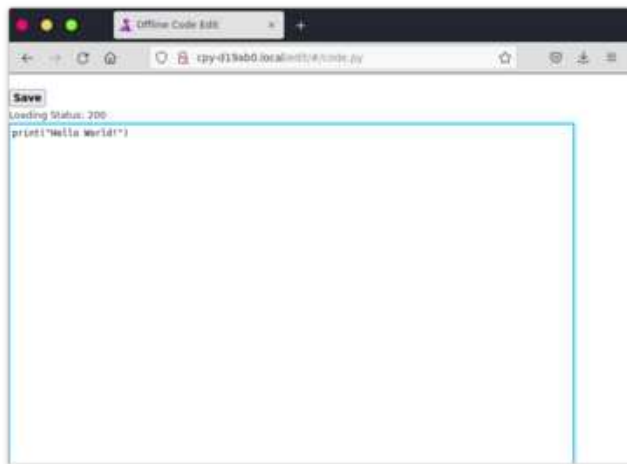
## File Browser

From the Welcome! page, click the **file browser** link on the Welcome! page to access files and folders.

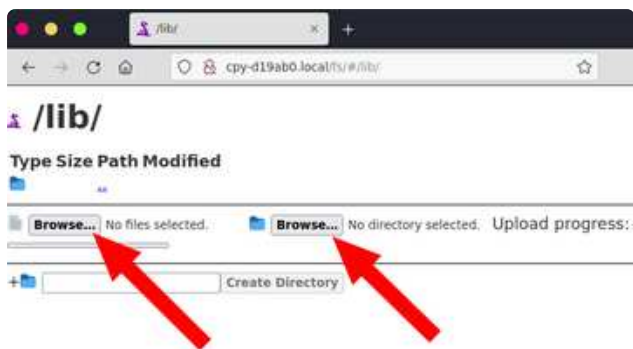


The files and folders should show up.

There are buttons for various actions.



For example, click the **Edit** button for **code.py** bring up a simple editor to allow changing the contents.



Library files and folders from the [Bundle \(https://adafru.it/ENC\)](https://adafru.it/ENC) can be uploaded to the **/lib** folder using the **Browse...** buttons.

There's a separate button for files and folders (directories).

---

## NeoPixel Example

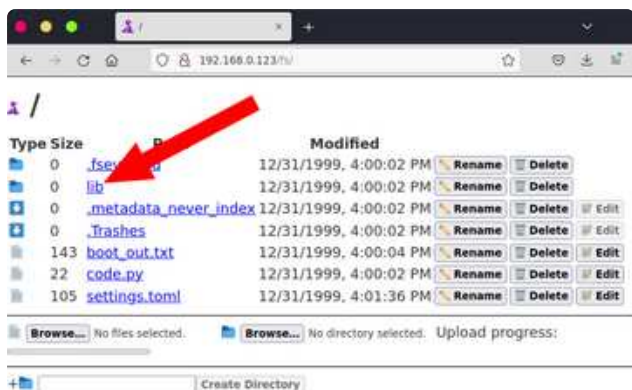
Here's a simple example showing how to blink the on board NeoPixel of either the Feather ESP32 V2 or the QT PY ESP32 Pico. It covers adding a library file, **neopixel.mpy**, as well as editing **code.py** directly in the browser.

### Adding a Library File

Follow these steps to add **neopixel.mpy** to the **/lib** folder.

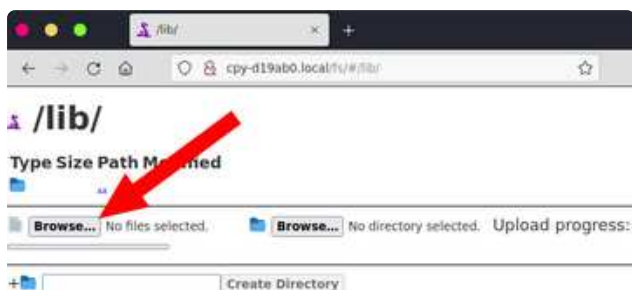


From the main Welcome! screen, click on the **file browser** link.

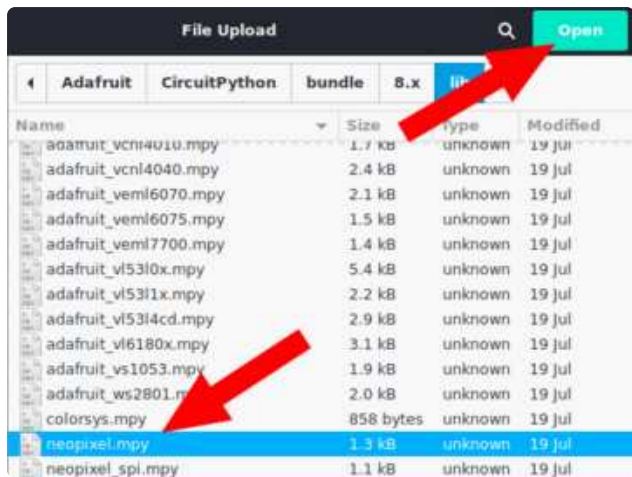


Now click the **lib** folder link to navigate into the **/lib** folder.

We need to navigate first, since uploaded files will go into the current directory location.

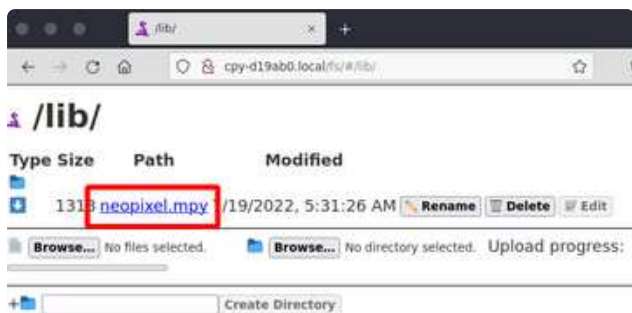


Now click the first **Browse...** button (for files) to bring up a file explorer window.



Navigate to where you've downloaded the [CircuitPython Library Bundle](https://adafru.it/ENC) (<https://adafru.it/ENC>) and select the **neopixel.mpy** file.

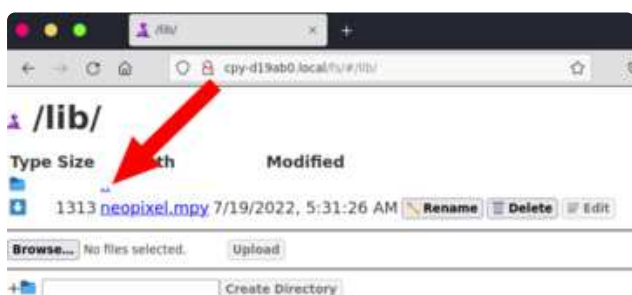
Then click **OK**, or **Open**, or whatever your file window shows.



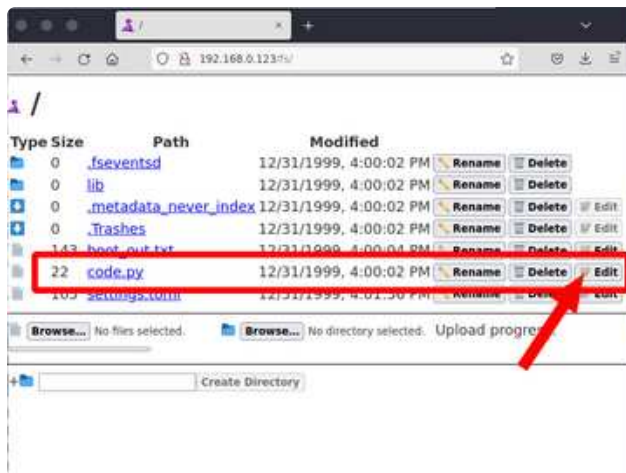
After uploading, the **neopixel.mpy** file will be shown in the **/lib** folder.

## Edit code.py

Follow these steps to write a simple NeoPixel blink example and save it as **code.py** so it will run automatically.



From the previous step, click the two little **..** to go back up one level in the folders.



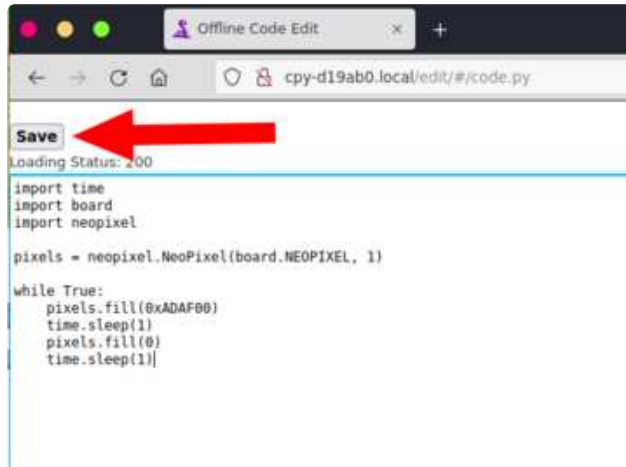
Now click the **Edit** button on the `code.py` line.

Here's the simple example code used:

```
import time
import board
import neopixel

pixels = neopixel.NeoPixel(board.NEOPIXEL, 1)

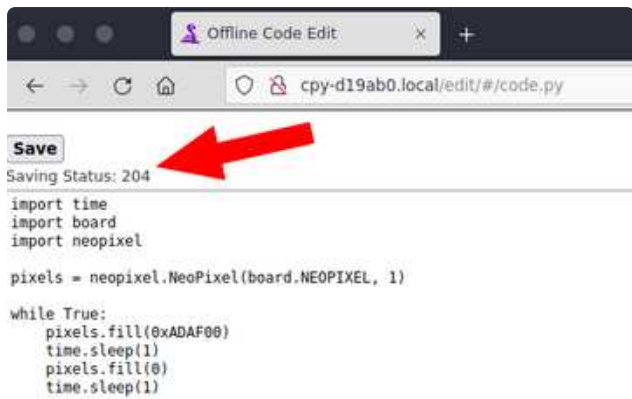
while True:
    pixels.fill(0xADAF00)
    time.sleep(1)
    pixels.fill(0)
    time.sleep(1)
```



Replace the contents of `code.py` with the code listing shown (can copy from above).

Make sure it matches what is shown - no indent errors, etc. Then click the **Save** button.





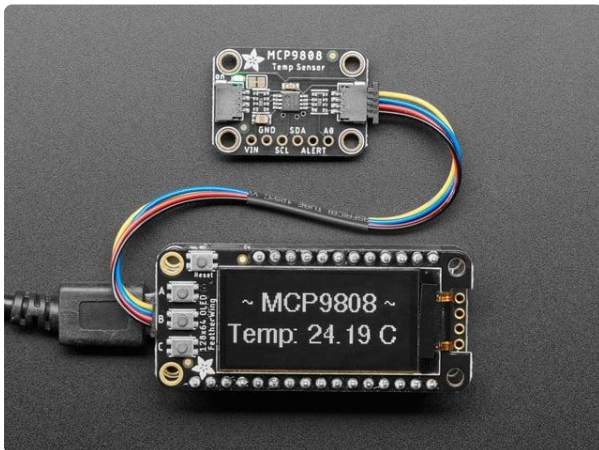
The page will remain, but the status code should change.

At this point the code should be running and the onboard NeoPixel should be blinking.

There may be a small delay after hitting the Save button before the code could actually run.

## I2C Scan Test

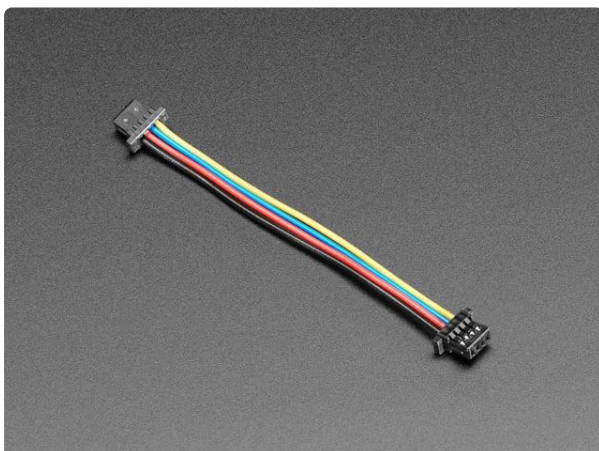
Here's a simple example showing how to scan for any I2C devices that are connected to the ItsyBitsy ESP32 using [Web Workflow](https://adafruit.it/10BX) (<https://adafruit.it/10BX>). You'll connect an MCP9808 I2C sensor to the ItsyBitsy ESP32 for testing.



### [Adafruit MCP9808 High Accuracy I2C Temperature Sensor Breakout](https://www.adafruit.com/product/5027)

The MCP9808 digital temperature sensor is one of the more accurate/precise we've ever seen, with a typical accuracy of  $\pm 0.25^{\circ}\text{C}$  over the sensor's  $-40^{\circ}\text{C}$  to...

<https://www.adafruit.com/product/5027>

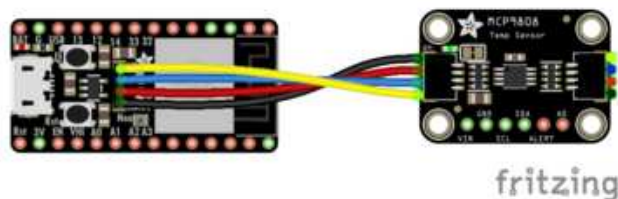


### [STEMMA QT / Qwiic JST SH 4-Pin Cable - 50mm Long](https://www.adafruit.com/product/4399)

This 4-wire cable is 50mm / 1.9" long and fitted with JST SH female 4-pin connectors on both ends. Compared with the chunkier JST PH these are 1mm pitch instead of 2mm, but...

<https://www.adafruit.com/product/4399>

## Wiring



Plug a **STEMMA QT** to **STEMMA QT** cable into the port on the **ItsyBitsy ESP32** and a port on the **MCP9808** breakout.

## Download the Project Bundle

In the example below, click the **Download Project Bundle** button to download the necessary Python file in a zip file. Then, extract the contents of the zip file. For this example, no additional libraries are needed.

```
# SPDX-FileCopyrightText: 2017 Limor Fried for Adafruit Industries
#
# SPDX-License-Identifier: MIT

# pylint: disable=broad-except, eval-used, unused-import

"""CircuitPython I2C Device Address Scan"""
import time
import board
import busio

# List of potential I2C busses
ALL_I2C = ("board.I2C()", "board.STEMMA_I2C()", "busio.I2C(board.GP1, board.GP0)")

# Determine which busses are valid
found_i2c = []
for name in ALL_I2C:
    try:
        print("Checking {}...".format(name), end="")
        bus = eval(name)
        bus.unlock()
        found_i2c.append((name, bus))
        print("ADDED.")
    except Exception as e:
        print("SKIPPED:", e)

# Scan valid busses
if len(found_i2c):
    print("-" * 40)
    print("I2C SCAN")
    print("-" * 40)
    while True:
        for bus_info in found_i2c:
            name = bus_info[0]
            bus = bus_info[1]

            while not bus.try_lock():
                pass
```

```

        print(
            name,
            "addresses found:",
            [hex(device_address) for device_address in bus.scan()],
        )

        bus.unlock()

        time.sleep(2)
    else:
        print("No valid I2C bus found.")

```

## Upload the Project Bundle with Web Workflow

Follow these steps to add the **code.py** file from the Project Bundle to the ItsyBitsy ESP32 with [Web Workflow](https://adafru.it/10BX) (<https://adafru.it/10BX>).

### Welcome!

Welcome to CircuitPython's Web API. Go to the [file browser](#) to work with files in the CIRCUITPYTHON drive. Go to the [full code editor](#) for a more enriching experience which requires an active internet connection. Go to the [serial terminal](#) to see code output and interact with the REPL. Make sure you've set CIRCUITPYTHON\_WEB\_API\_PASSWORD="somepassword" in /settings.toml. Provide the password when the browser prompts for it. Leave the username blank.

#### Device Info:

Board:  
Adafruit ItsyBitsy ESP32  
Version:  
9.0.0-beta.2-1-g721d4deca5  
Hostname:  
cay-96bc79  
IP:  
192.168.1.192

From the main Welcome! screen, click on the **file browser** link.



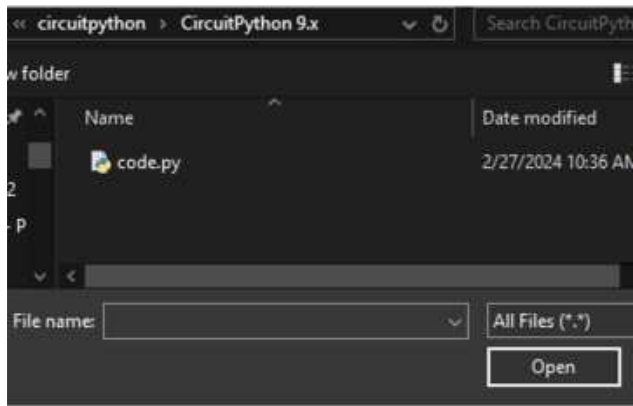
| Type          | Size   | Path          | Modified               |                                                                    |
|---------------|--------|---------------|------------------------|--------------------------------------------------------------------|
| lib           | 0 B    | lib           | 12/31/1999, 7:00:02 PM | <a href="#">Rename</a> <a href="#">Delete</a>                      |
| sd            | 0 B    | sd            | 12/31/1999, 7:00:02 PM | <a href="#">Rename</a> <a href="#">Delete</a>                      |
| boot_out.txt  | 155 B  | boot_out.txt  | 12/31/1999, 7:00:06 PM | <a href="#">Rename</a> <a href="#">Delete</a> <a href="#">Edit</a> |
| code.py       | 1.0 kB | code.py       | 2/27/2024, 10:05:26 AM | <a href="#">Rename</a> <a href="#">Delete</a> <a href="#">Edit</a> |
| settings.toml | 119 B  | settings.toml | 12/31/1999, 7:05:44 PM | <a href="#">Rename</a> <a href="#">Delete</a> <a href="#">Edit</a> |

[Browse...](#) No files selected.

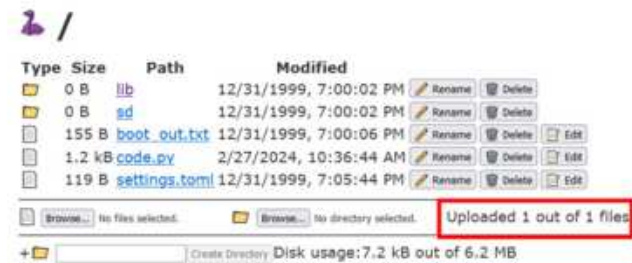
[Browse...](#) No directory selected.

+ [Create Directory](#) Disk usage: 52.2 kB out of 6.2 MB

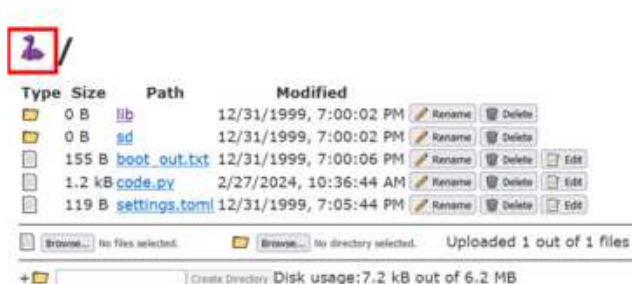
Click the first **Browse...** button (for files) to bring up a file explorer window.



Navigate to where you downloaded the Project Bundle and select the **code.py** file. Click **Open** to upload the file to the device.



When the file has successfully uploaded, you'll see "**Uploaded 1 out of 1 files**" appear below the directory listing.



Your code will immediately start running. To see the serial output, you can head back to the main Welcome! screen by clicking on **Blinka** (the purple snake).

## 👤 Welcome!

Welcome to CircuitPython's Web API. Go to the [file browser](#) to work with files in the CIRCUITPY drive. Go to the [serial terminal](#) for a more enriching experience which requires an active internet connection. Go to the [serial terminal](#) to see code output and interact with the REPL. Make sure you've set CIRCUITPY\_USERNAME='somepass@adafruit.com' in /settings.toml. Provide the password when the browser prompts for it. **Leave the username blank.**

### Device Info:

Board:  
Adafruit ItsyBitsy ESP32  
Version:  
9.0.0-beta.2-1-g721d4eca5  
Hostname:  
cpx-95bc79  
IP:  
192.168.1.182

On the Welcome! screen, click on the **serial terminal** link. This will take you to the serial terminal window.

```
board.I2C() addresses found: ['0x18']  
board.I2C() addresses found: ['0x18']  
board.I2C() addresses found: ['0x18']  
board.I2C() addresses found: ['0x18']  
board.I2C() addresses found: ['0x18']  
board.I2C() addresses found: ['0x18']
```



On the serial terminal screen, you should see an I2C device at address **0x18**. This means that the I2C scan detected the MCP9808 I2C device

---

## CircuitPython WiFi Test

Here's an example testing WiFi connectivity with the ItsyBitsy ESP32 with [Web Workflow \(https://adafru.it/10BX\)](https://adafru.it/10BX).

### Download the Project Bundle

In the example below, click the **Download Project Bundle** button to download the necessary library and Python files in a zip file. Then, extract the contents of the zip file.

```
# SPDX-FileCopyrightText: 2017 Limor Fried for Adafruit Industries  
#  
# SPDX-License-Identifier: MIT  
  
import os  
import ipaddress  
import ssl  
import wifi  
import socketpool  
import adafruit_requests  
  
# URLs to fetch from  
TEXT_URL = "http://wifitest.adafruit.com/testwifi/index.html"  
JSON_QUOTES_URL = "https://www.adafruit.com/api/quotes.php"
```

```

JSON_STARS_URL = "https://api.github.com/repos/adafruit/circuitpython"

print("ItsyBitsy ESP32 WebClient Test")

print(f"My MAC address: {[hex(i) for i in wifi.radio.mac_address]}")

print("Available WiFi networks:")
for network in wifi.radio.start_scanning_networks():
    print("\t%s\t\tRSSI: %d\tChannel: %d" % (str(network.ssid, "utf-8"),
                                             network.rssi, network.channel))
wifi.radio.stop_scanning_networks()

print(f"Connecting to {os.getenv('CIRCUITPY_WIFI_SSID')}")
wifi.radio.connect(os.getenv("CIRCUITPY_WIFI_SSID"),
os.getenv("CIRCUITPY_WIFI_PASSWORD"))
print(f"Connected to {os.getenv('CIRCUITPY_WIFI_SSID')}")
print(f"My IP address: {wifi.radio.ipv4_address}")

ping_ip = ipaddress.IPv4Address("8.8.8.8")
ping = wifi.radio.ping(ip=ping_ip)

# retry once if timed out
if ping is None:
    ping = wifi.radio.ping(ip=ping_ip)

if ping is None:
    print("Couldn't ping 'google.com' successfully")
else:
    # convert s to ms
    print(f"Pinging 'google.com' took: {ping * 1000} ms")

pool = socketpool.SocketPool(wifi.radio)
requests = adafruit_requests.Session(pool, ssl.create_default_context())

print(f"Fetching text from {TEXT_URL}")
response = requests.get(TEXT_URL)
print("-" * 40)
print(response.text)
print("-" * 40)

print(f"Fetching json from {JSON_QUOTES_URL}")
response = requests.get(JSON_QUOTES_URL)
print("-" * 40)
print(response.json())
print("-" * 40)

print()

print(f"Fetching and parsing json from {JSON_STARS_URL}")
response = requests.get(JSON_STARS_URL)
print("-" * 40)
print(f"CircuitPython GitHub Stars: {response.json()['stargazers_count']}")
print("-" * 40)

print("Done")

```

## Upload the Project Bundle with Web Workflow

Follow these steps to add the `/lib` folder and `code.py` file from the Project Bundle to the ItsyBitsy ESP32 with [Web Workflow](https://adafru.it/10BX) (<https://adafru.it/10BX>).



## Welcome!

Welcome to CircuitPython's Web API. Go to the **file browser** to work with files in the CIRCUITPY drive. Go to the **full code editor** for a more enriching experience (which requires an active internet connection). Go to the **serial terminal** to see code output and interact with the REPL. Make sure you've set CIRCUITPY\_WEB\_API\_PASSWORD="somepassword" in /settings.toml. Provide the password when the browser prompts for it. **Leave the username blank.**

### Device Info:

Board: [Adafruit ItsyBitsy ESP32](#)  
Version: 9.0.0-beta.2-1-g721d4deca5  
Hostname: [cpx-96bc79](#)  
IP: [192.168.1.192](#)

From the main Welcome! screen, click on the **file browser** link.

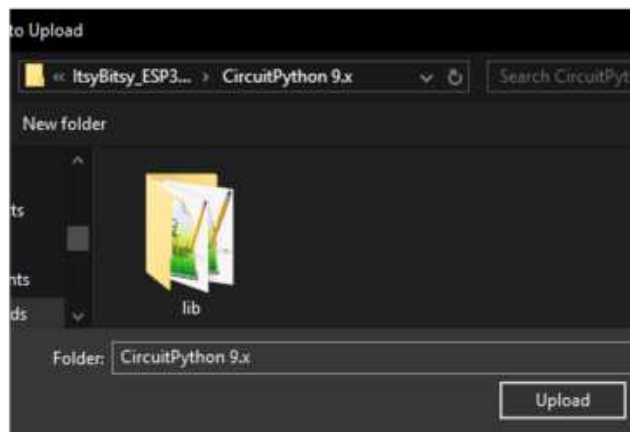


| Type   | Size                          | Path                   | Modified               |                        |                      |
|--------|-------------------------------|------------------------|------------------------|------------------------|----------------------|
| 0 B    | <a href="#">lib</a>           | 12/31/1999, 7:00:02 PM | <a href="#">Rename</a> | <a href="#">Delete</a> |                      |
| 0 B    | <a href="#">sd</a>            | 12/31/1999, 7:00:02 PM | <a href="#">Rename</a> | <a href="#">Delete</a> |                      |
| 155 B  | <a href="#">boot_out.txt</a>  | 12/31/1999, 7:00:06 PM | <a href="#">Rename</a> | <a href="#">Delete</a> | <a href="#">Edit</a> |
| 1.0 kB | <a href="#">code.py</a>       | 2/27/2024, 10:05:26 AM | <a href="#">Rename</a> | <a href="#">Delete</a> | <a href="#">Edit</a> |
| 119 B  | <a href="#">settings.toml</a> | 12/31/1999, 7:05:44 PM | <a href="#">Rename</a> | <a href="#">Delete</a> | <a href="#">Edit</a> |

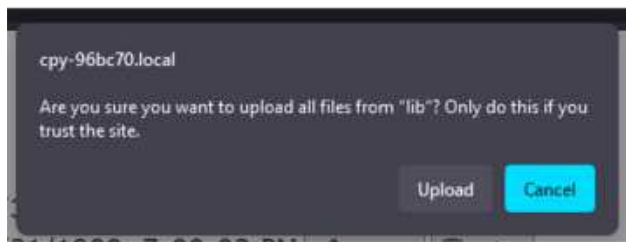
[Browse...](#) No files selected. [Browse...](#) No directory selected.

+  Create Directory Disk usage: 52.2 kB out of 6.2 MB

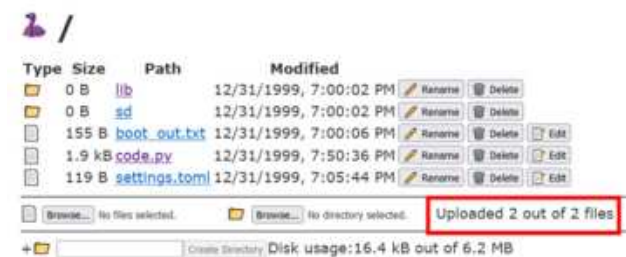
Click the second **Browse...** button (for folders) to bring up a file explorer window.



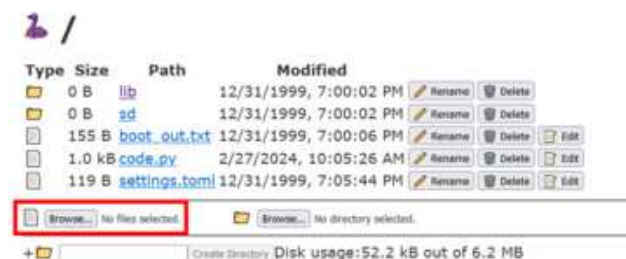
Navigate to where you downloaded the Project Bundle and select the **/lib** folder. Click **Upload** to upload the folder to the device.



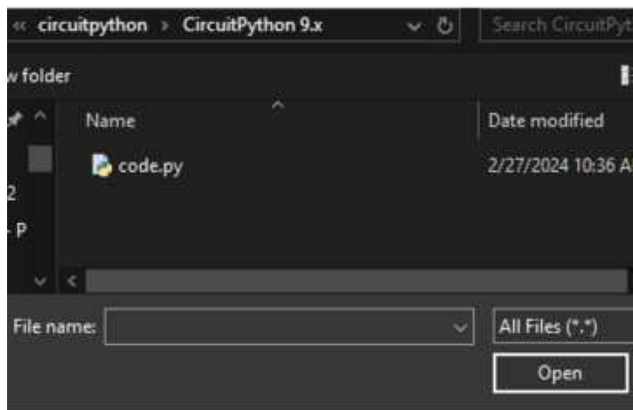
You'll be prompted in the browser to confirm that you want to upload the folder. Click **Upload** to proceed.



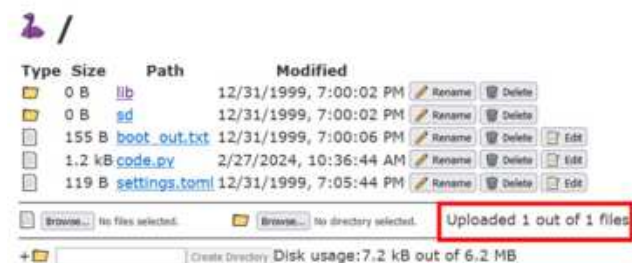
When the `/lib` folder has been successfully uploaded, you will see a message below the file directory that says "**Uploaded 2 of 2**".



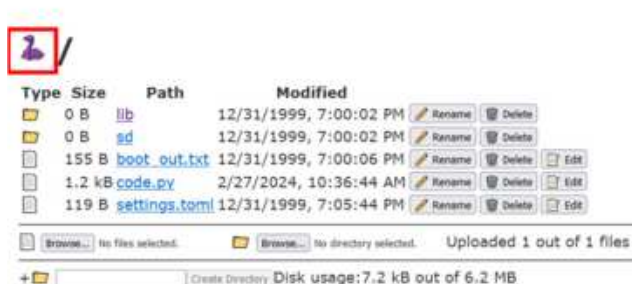
Now you'll upload the `code.py` file. Click the first **Browse...** button (for files) to bring up a file explorer window.



Navigate to where you downloaded the Project Bundle and select the **code.py** file. Click **Open** to upload the file to the device.



When the file has successfully uploaded, you'll see "**Uploaded 1 out of 1 files**" appear below the directory listing.



Your code will immediately start running. To see the serial output, you can head back to the main Welcome! screen by clicking on **Blinka** (the purple snake).

## Welcome!

Welcome to CircuitPython's Web API. Go to the [file browser](#) to work with files in the CIRCUITPY drive. Go to the [serial terminal](#) to see code output and interact with the REPL. Make sure you've set CIRCUITPY\_USERNAME='cups4000' in /settings.toml. Provide the password when the browser prompts for it. **Leave the username blank.**

### Device Info:

Board:  
Adafruit ItsyBitsy ESP32  
Version:  
9.0.0-beta.2-1-g721d4deca5  
Hostname:  
cpr-95bc79  
IP:  
192.168.1.182

On the Welcome! screen, click on the **serial terminal** link. This will take you to the serial terminal window.



On the serial terminal screen, click the small **D** button to send a Ctrl+D message to the ItsyBitsy ESP32. This will give it a soft reboot to re-run the code. When the code runs, you'll see the WiFi test output.

## Arduino IDE Setup

You need to install the right USB-to-serial driver for your chip in addition to the Arduino IDE. If you are unsure which is the right one, install both!

### Install Arduino IDE

The first thing you will need to do is to download the latest release of the Arduino IDE. You will need to be using **version 1.8** or higher for this guide

**Arduino IDE Download**

<https://adafru.it/f1P>

### Install CP2104 / CP2102N USB Driver

The USB-to-Serial converter that talks to the ESP32 chip itself will need a driver on your computer's operating system. The driver is available for Mac and Windows. It is already built into Linux.

Click here to download the CP2104 USB Driver

<https://adafru.it/vrf>

## Install CH9102 / CH34X USB Driver

Newer ESP32 boards have a different USB-to-serial converter that talks to the chip itself, and will need a driver on your computer's operating system. The driver is available for Mac and Windows. It is already built into Linux.

If you would like more detail, check out [the guide on installing these drivers](https://adafru.it/-f8) (<https://adafru.it/-f8>).

Click here to download the Windows driver

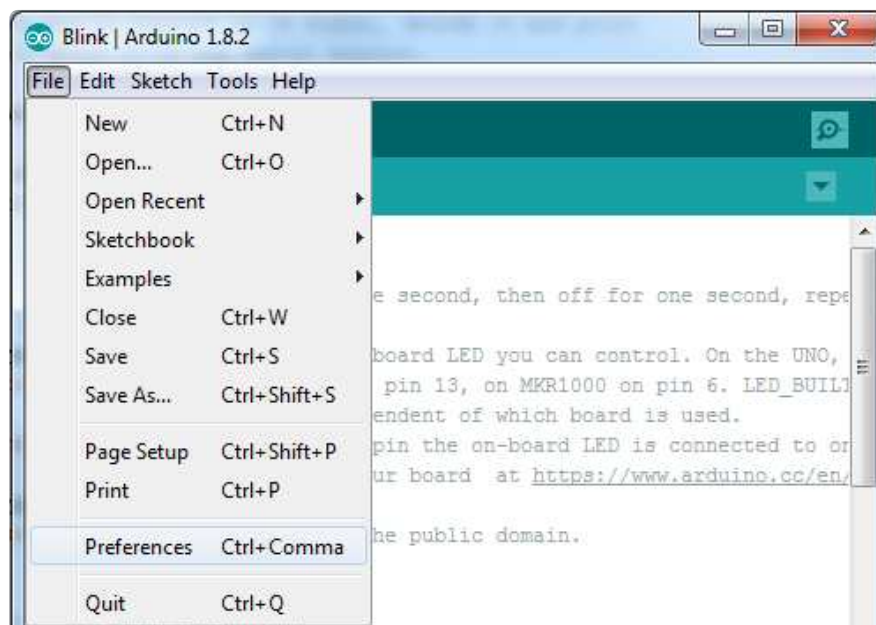
<https://adafru.it/-f9>

Click here to download the Mac driver

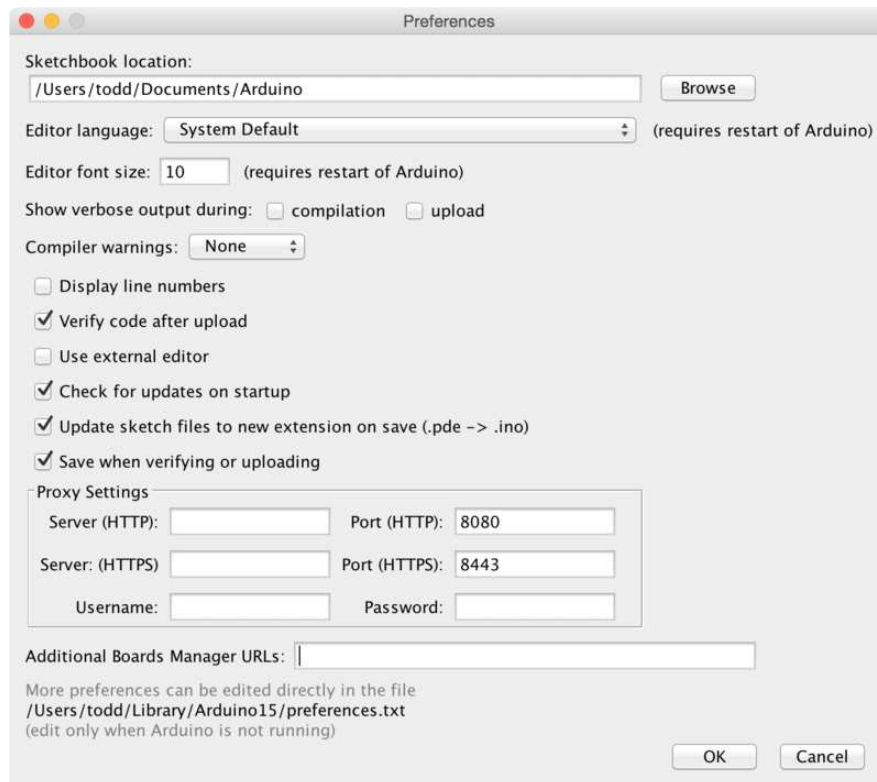
<https://adafru.it/-ed>

## Install ESP32 Board Support Package

After you have downloaded and installed **the latest version of Arduino IDE**, you will need to start the IDE and navigate to the **Preferences** menu. You can access it from the **File** menu in Windows or Linux, or the **Arduino** menu on OS X.



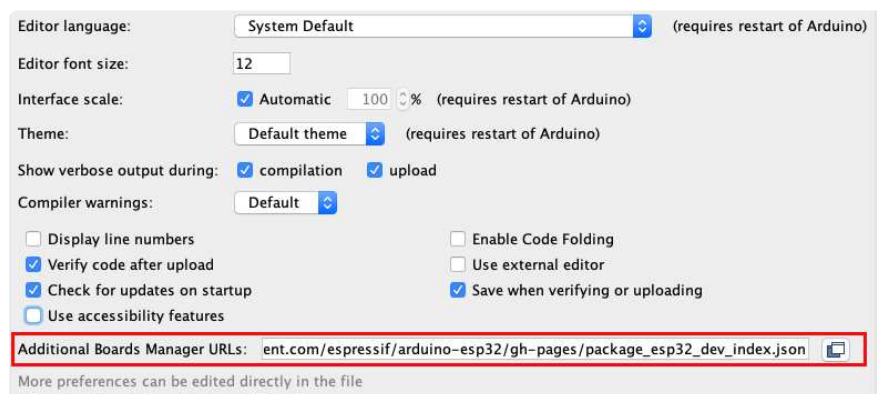
A dialog will pop up just like the one shown below.



We will be adding a URL to the new **Additional Boards Manager URLs** option. The list of URLs is comma separated, and you will only have to add each URL once. New Adafruit boards and updates to existing boards will automatically be picked up by the Board Manager each time it is opened. The URLs point to index files that the Board Manager uses to build the list of available & installed boards.

To find the most up to date list of URLs you can add, you can visit the list of [third party board URLs on the Arduino IDE wiki \(https://adafru.it/f7U\)](https://adafru.it/f7U). We will only need to add one URL to the IDE in this example, but **you can add multiple URLs by separating them with commas**. Copy and paste the link below into the **Additional Boards Manager URLs** option in the Arduino IDE preferences.

[https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package\\_esp32\\_dev\\_index.json](https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_dev_index.json)

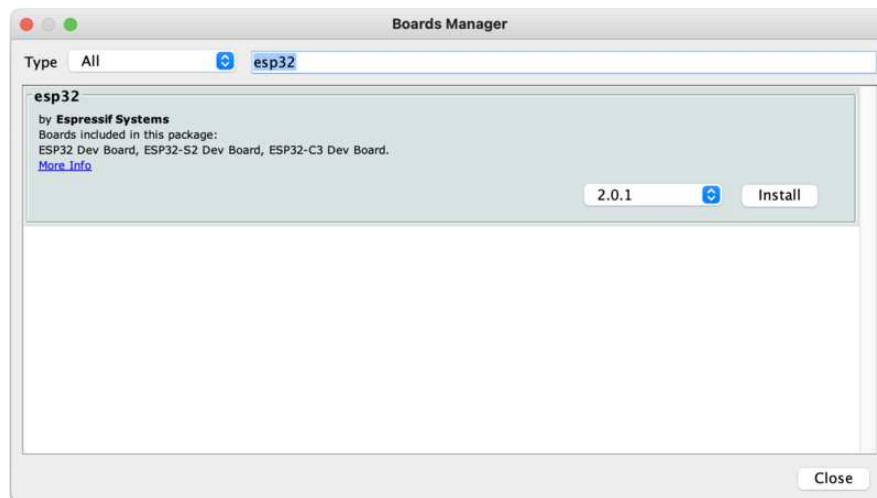




If you have multiple boards you want to support, say ESP8266 and Adafruit, have both URLs in the text box separated by a comma (,)

Once done click **OK** to save the new preference settings.

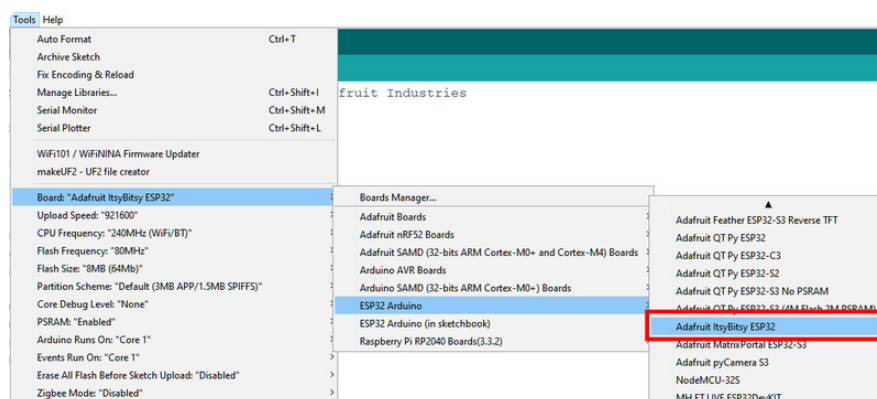
The next step is to actually install the Board Support Package (BSP). Go to the **Tools → Board → Board Manager** submenu. A dialog should come up with various BSPs. Search for **esp32**.



Click the **Install** button and wait for it to finish. Once it is finished, you can close the dialog.

In the **Tools → Board** submenu you should see **ESP32 Arduino** and in that dropdown it should contain the ESP32 boards along with all the latest ESP32 boards.

Look for the board called Adafruit ItsyBitsy ESP32.



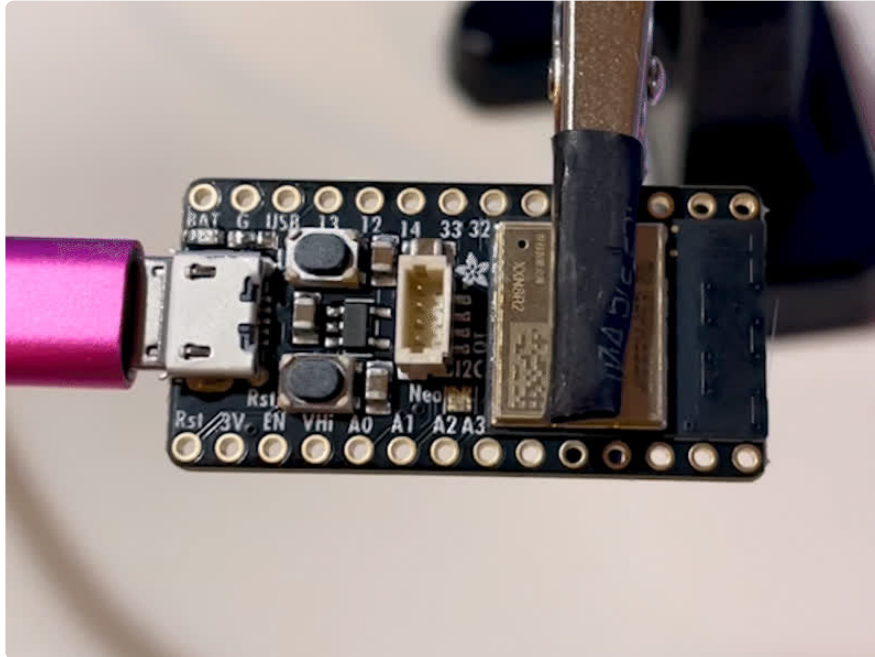
The upload speed can be changed: faster speed makes uploads take less time but sometimes can cause upload issues. **921600** should work fine, but if you're having issues, you can drop down lower.

---

# Blink

The first and most basic program you can upload to your Arduino is the classic Blink sketch. This takes something on the board and makes it, well, blink! On and off. It's a great way to make sure everything is working and you're uploading your sketch to the right board and right configuration.

When all else fails, you can always come back to Blink!



## Pre-Flight Check: Get Arduino IDE & Hardware Set Up

This lesson assumes you have Arduino IDE set up. This is a generalized checklist, some elements may not apply to your hardware. If you haven't yet, check the previous steps in the guide to make sure you:

- **Install the very latest Arduino IDE for Desktop** (not all boards are supported by the Web IDE so we don't recommend it)
- **Install any board support packages (BSP) required for your hardware.** Some boards are built in defaults on the IDE, but lots are not! You may need to install plug-in support which is called the BSP.
- **Get a Data/Sync USB cable for connecting your hardware.** A significant amount of problems folks have stem from not having a USB cable with data pins. Yes, these cursed cables roam the land, making your life hard. If you find a USB cable that doesn't work for data/sync, throw it away immediately! There is no need to keep it around, cables are very inexpensive these days.

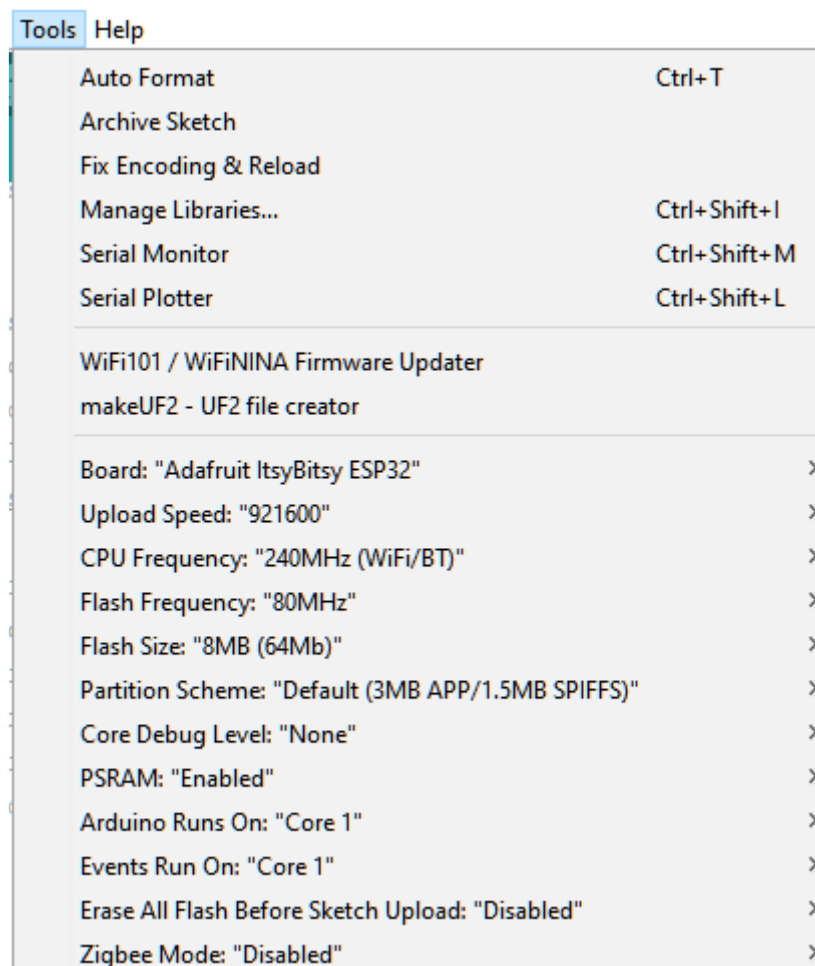
- **Install any drivers required** - If you have a board with a FTDI or CP210x chip, you may need to get separate drivers. If your board has native USB, it probably doesn't need anything. After installing, reboot to make sure the driver sinks in.
- **Connect the board to your computer.** If your board has a power LED, make sure it's lit. Is there a power switch? Make sure its turned On!

The ItsyBitsy ESP32 has no power LED or power switch.

## Start up Arduino IDE and Select Board/Port

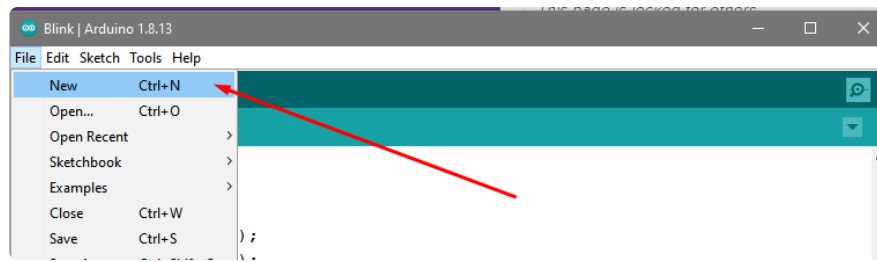
OK now you are prepared! Open the Arduino IDE on your computer. Now you have to tell the IDE what board you are using, and how you want to connect to it.

In the IDE find the **Tools** menu. You will use this to select the board. If you switch boards, you must switch the selection! So always double-check before you upload code in a new session.



## New Blink Sketch

OK lets make a new blink sketch! From the **File** menu, select **New**



Then in the new window, copy and paste this text:

```
int led = LED_BUILTIN;

void setup() {
  // Some boards work best if we also make a serial connection
  Serial.begin(115200);

  // set LED to be an output pin
  pinMode(led, OUTPUT);
}

void loop() {
  // Say hi!
  Serial.println("Hello!");

  digitalWrite(led, HIGH); // turn the LED on (HIGH is the voltage level)
  delay(500);              // wait for a half second
  digitalWrite(led, LOW);  // turn the LED off by making the voltage LOW
  delay(500);              // wait for a half second
}
```

Note that in this example, we are not only blinking the LED but also printing to the Serial monitor, think of it as a little bonus to test the serial connection.

One note you'll see is that we reference the LED with the constant `LED_BUILTIN` rather than a number. That's because, historically, the built in LED was on pin 13 for Arduinos. But in the decades since, boards don't always have a pin 13, or maybe it could not be used for an LED. So the LED could have moved to another pin. It's best to use `LED_BUILTIN` so you don't get the pin number confused!

The red LED on the ItsyBitsy ESP32 is available as `LED_BUILTIN`, as well as `13`.

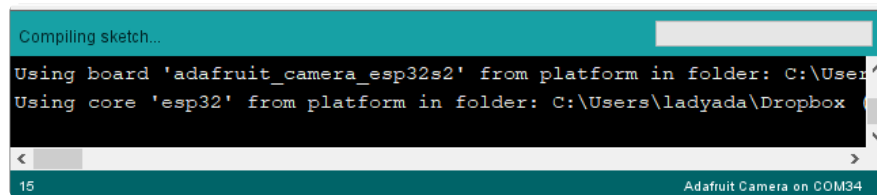
## Verify (Compile) Sketch

OK now you can click the Verify button to convert the sketch into binary data to be uploaded to the board.

Note that Verifying a sketch is the same as Compiling a sketch - so we will use the words interchangeably

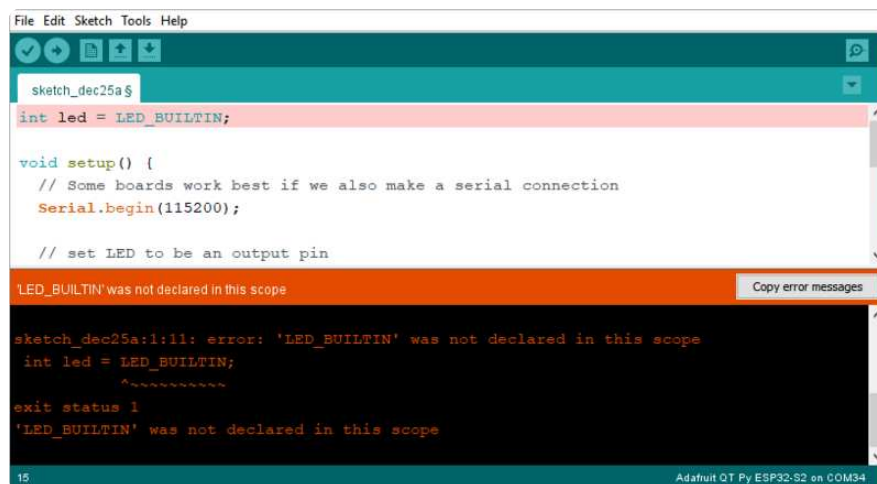


During verification/compilation, the computer will do a bunch of work to collect all the libraries and code and the results will appear in the bottom window of the IDE

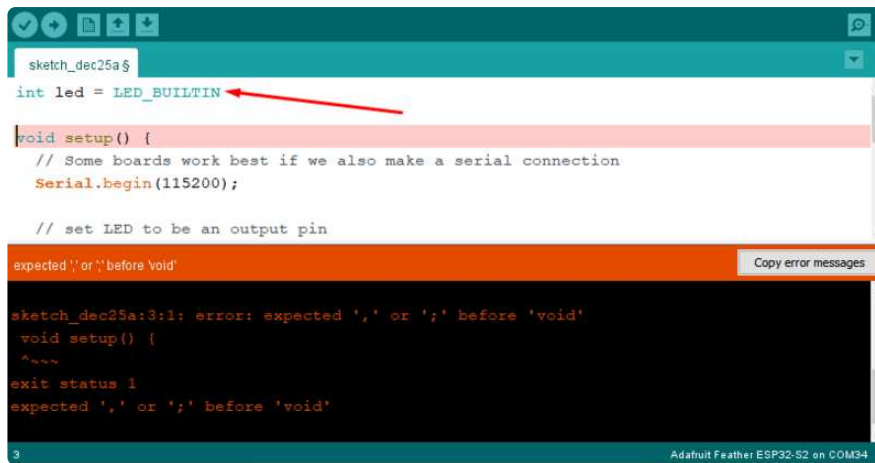


If something went wrong with compilation, you will get red warning/error text in the bottom window letting you know what the error was. It will also highlight the line with an error

For example, here I had the wrong board selected - and the selected board does not have a built in LED!



Here's another common error, in my haste I forgot to add a `;` at the end of a line. The compiler warns me that it's looking for one - note that the error is actually a few lines up!



```
sketch_dec25a$
int led = LED_BUILTIN;

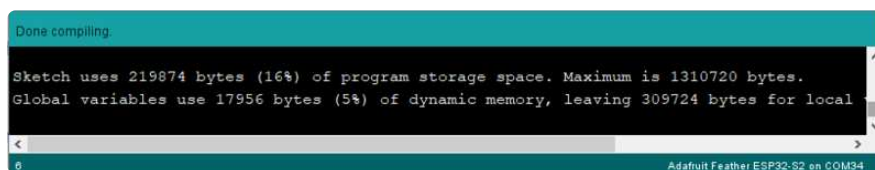
void setup() {
  // Some boards work best if we also make a serial connection
  Serial.begin(115200);

  // set LED to be an output pin
}

error: expected \',\' or \';\' before \'void\'
sketch_dec25a:3:1: error: expected \',\' or \';\' before \'void\'
void setup() {
^~~~~
exit status 1
expected \',\' or \';\' before \'void\'
```

Turning on detailed compilation warnings and output can be very helpful sometimes - It's in Preferences under "Show Verbose Output During:" and check the Compilation button. If you ever need to get help from others, be sure to do this and then provide all the text that is output. It can assist in determining what happened!

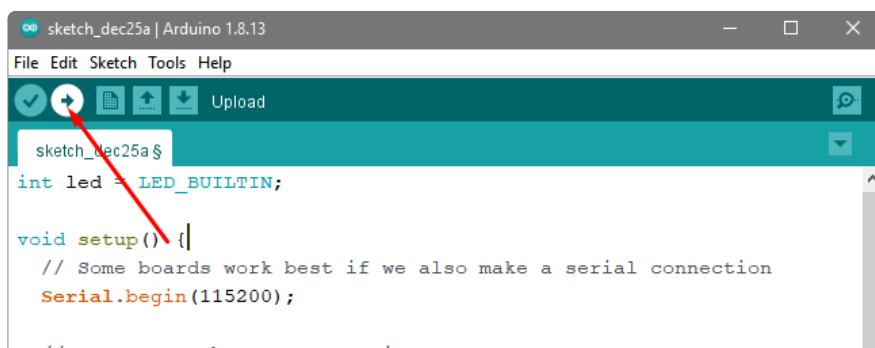
On success you will see something like this white text output and the message **Done compiling.** in the message area.



```
Done compiling.
Sketch uses 219874 bytes (16%) of program storage space. Maximum is 1310720 bytes.
Global variables use 17956 bytes (5%) of dynamic memory, leaving 309724 bytes for local
```

## Upload Sketch

Once the code is verified/compiling cleanly you can upload it to your board. Click the **Upload** button



```
sketch_dec25a | Arduino 1.8.13
File Edit Sketch Tools Help
Upload
sketch_dec25a$
int led = LED_BUILTIN;

void setup() {
  // Some boards work best if we also make a serial connection
  Serial.begin(115200);

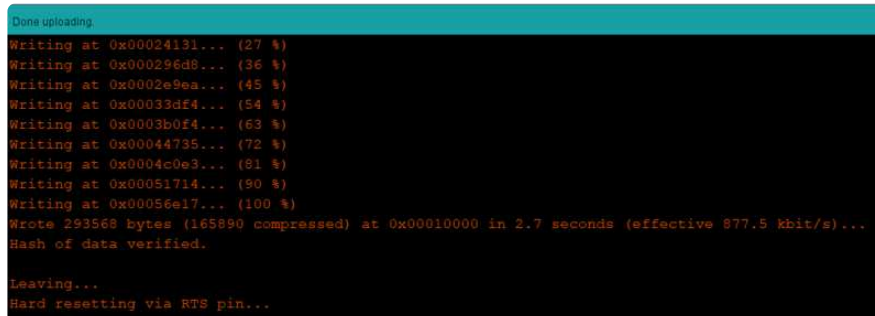
  // set LED to be an output pin
}
```

The IDE will try to compile the sketch again for good measure, then it will try to connect to the board and upload a the file.



This is actually one of the hardest parts for beginners because it's where a lot of things can go wrong.

However, let's start with what it looks like on success! Here's what your board upload process looks like when it goes right:



```
Done uploading.
Writing at 0x00024131... (27 %)
Writing at 0x000296d8... (36 %)
Writing at 0x0002e9ea... (45 %)
Writing at 0x00033df4... (54 %)
Writing at 0x0003b0f4... (63 %)
Writing at 0x00044735... (72 %)
Writing at 0x0004c0e3... (81 %)
Writing at 0x00051714... (90 %)
Writing at 0x00056e17... (100 %)
Wrote 293568 bytes (165890 compressed) at 0x00010000 in 2.7 seconds (effective 877.5 kbit/s)...
Hash of data verified.
Leaving...
Hard resetting via RTS pin...
```

Often times you will get a warning like this, which is kind of vague:

No device found on COM66 (or whatever port is selected)  
An error occurred while uploading the sketch



```
An error occurred while uploading the sketch
Sketch uses 11228 bytes (1%) of program storage space. Maximum is 1032192 bytes.
No device found on COM66
An error occurred while uploading the sketch
```

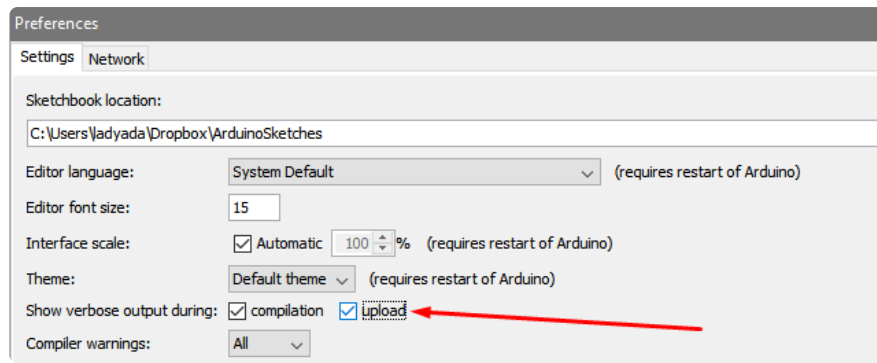
This could be a few things.

First up, **check again that you have the correct board selected!** Many electronics boards have very similar names or look, and often times folks grab a board different from what they thought.

Second, **make sure you selected the right port!** If you have the wrong port or no port selected, Arduino doesn't know where to look for your board.

If both of those are correct, the next step is to enable verbose upload messages.

Before continuing we really really suggest turning on **Verbose Upload** messages, it will help in this process because you will be able to see what the IDE is trying to do. It's a checkbox in the **Preferences** menu.



Now you can try uploading again!

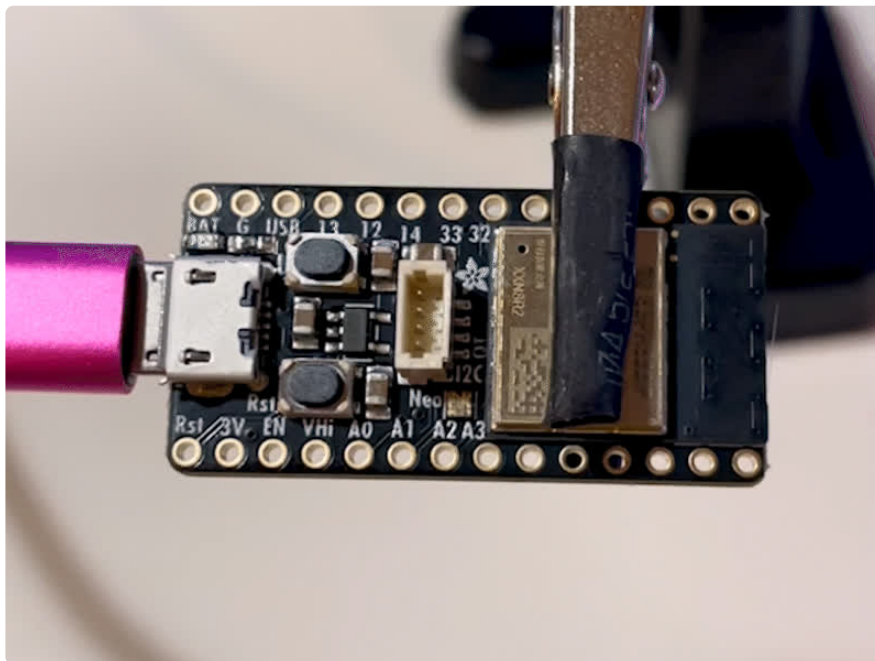


This time, you should have success!

After uploading this way, be sure to **click the reset button** - it sort of makes sure that the board got a good reset and will come back to life nicely.

## Finally, a Blink!

OK it was a journey but now we're here and you can enjoy your blinking LED. Next up, try to change the delay between blinks and re-upload. It's a good way to make sure your upload process is smooth and practiced.



---

# I2C Scan Test

A lot of sensors, displays, and devices can connect over I2C. I2C is a 2-wire 'bus' that allows multiple devices to all connect on one set of pins so it's very convenient for wiring!

When using your board, you'll probably want to connect up I2C devices, and it can be a little tricky the first time. The best way to debug I2C is go through a checklist and then perform an I2C scan

## Common I2C Connectivity Issues

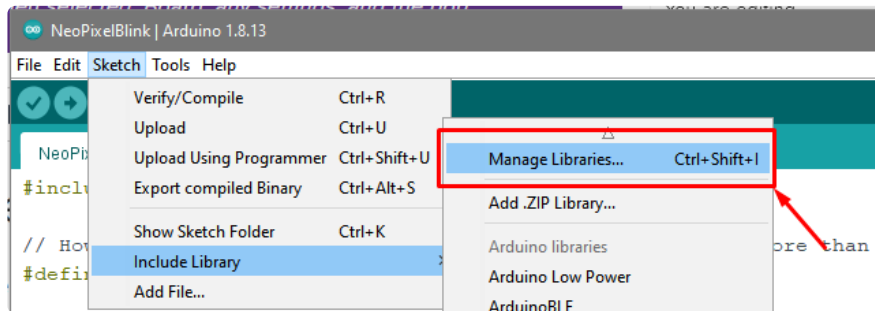
- **Have you connected four wires (at a minimum) for each I2C device?** Power the device with whatever is the logic level of your microcontroller board (probably 3.3V), then a ground wire, and a SCL clock wire, and and a SDA data wire.
- **If you're using a STEMMA QT board - check if the power LED is lit.** It's usually a green LED to the left side of the board.
- **Does the STEMMA QT/I2C port have switchable power or pullups?** To reduce power, some boards have the ability to cut power to I2C devices or the pullup resistors. Check the documentation if you have to do something special to turn on the power or pullups.
- **If you are using a DIY I2C device, do you have pullup resistors?** Many boards do not have pullup resistors built in and they are required! We suggest any common 2.2K to 10K resistors. You'll need two: one each connects from SDA to positive power, and SCL to positive power. Again, positive power (a.k.a VCC, VDD or V+) is often 3.3V
- **Do you have an address collision?** You can only have one board per address. So you cannot, say, connect two AHT20's to one I2C port because they have the same address and will interfere. Check the sensor or documentation for the address. Sometimes there are ways to adjust the address.
- **Does your board have multiple I2C ports?** Historically, boards only came with one. But nowadays you can have two or even three! This can help solve the "hey, but what if I want two devices with the same address" problem: just put one on each bus.
- **Are you hot-plugging devices?** I2C does not support dynamic re-connection, you cannot connect and disconnect sensors as you please. They should all be connected on boot and not change. ([Only exception is if you're using a hot-plug assistant but that'll cost you \(http://adafru.it/5159\)](http://adafru.it/5159)).
- **Are you keeping the total bus length reasonable?** I2C was designed for maybe 6" max length. We like to push that with plug-n-play cables, but really please keep them as short as possible! ([Only exception is if you're using an active bus extender \(http://adafru.it/4756\)](http://adafru.it/4756)).

The ItsyBitsy ESP32 does **not** have I2C pullup resistors - make sure any sensor you attach to the I2C port has resistors or add them yourself.

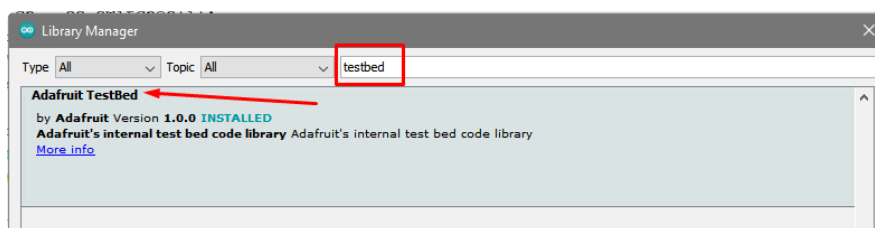
## Perform an I2C scan!

### Install TestBed Library

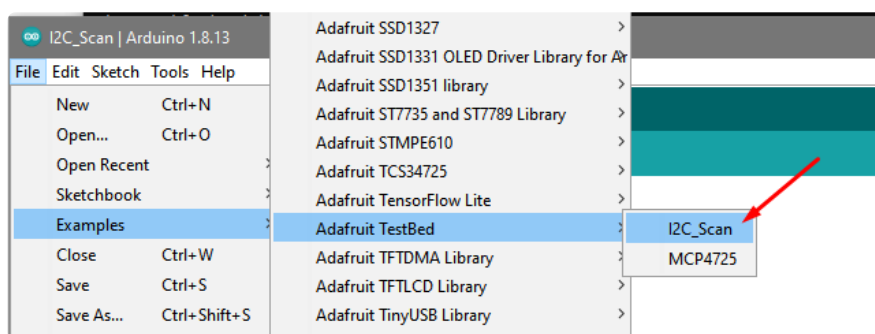
To scan I2C, the Adafruit TestBed library is used. This library and example just makes the scan a little easier to run because it takes care of some of the basics. You will need to add support by installing the library. Good news: it is very easy to do it. Go to the **Arduino Library Manager**.



Search for **TestBed** and install the **Adafruit TestBed** library



Now open up the I2C Scan example



```
// SPDX-FileCopyrightText: 2023 Carter Nelson for Adafruit Industries
//
// SPDX-License-Identifier: MIT
// -----
// i2c_scanner
//
// Modified from https://playground.arduino.cc/Main/I2cScanner/
// -----

#include <Wire.h>
```

```

// Set I2C bus to use: Wire, Wire1, etc.
#define WIRE Wire

void setup() {
  WIRE.begin();

  Serial.begin(9600);
  while (!Serial)
    delay(10);
  Serial.println("\nI2C Scanner");
}

void loop() {
  byte error, address;
  int nDevices;

  Serial.println("Scanning...");

  nDevices = 0;
  for(address = 1; address < 127; address++ )
  {
    // The i2c_scanner uses the return value of
    // the Write.endTransmission to see if
    // a device did acknowledge to the address.
    WIRE.beginTransmission(address);
    error = WIRE.endTransmission();

    if (error == 0)
    {
      Serial.print("I2C device found at address 0x");
      if (address<16)
        Serial.print("0");
      Serial.print(address,HEX);
      Serial.println("  !");

      nDevices++;
    }
    else if (error==4)
    {
      Serial.print("Unknown error at address 0x");
      if (address<16)
        Serial.print("0");
      Serial.println(address,HEX);
    }
  }
  if (nDevices == 0)
    Serial.println("No I2C devices found\n");
  else
    Serial.println("done\n");

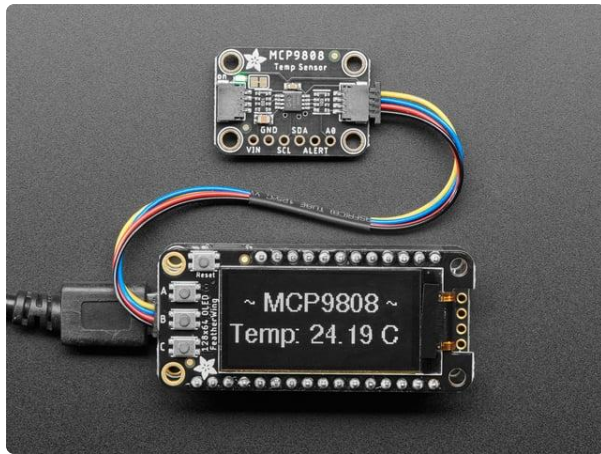
  delay(5000);           // wait 5 seconds for next scan
}

```

## Wire up I2C device

While the examples here will be using the [Adafruit MCP9808](http://adafru.it/5027) (<http://adafru.it/5027>), a high accuracy temperature sensor, the overall process is the same for just about any I2C sensor or device.

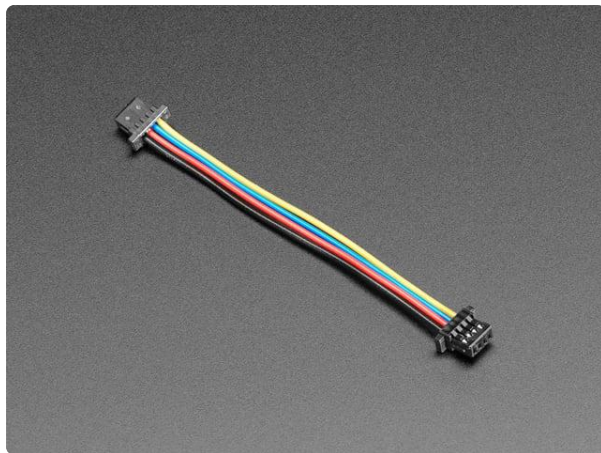
The first thing you'll want to do is get the sensor connected so your board has I2C to talk to.



## Adafruit MCP9808 High Accuracy I2C Temperature Sensor Breakout

The MCP9808 digital temperature sensor is one of the more accurate/precise we've ever seen, with a typical accuracy of  $\pm 0.25^{\circ}\text{C}$  over the sensor's  $-40^{\circ}\text{C}$  to...

<https://www.adafruit.com/product/5027>



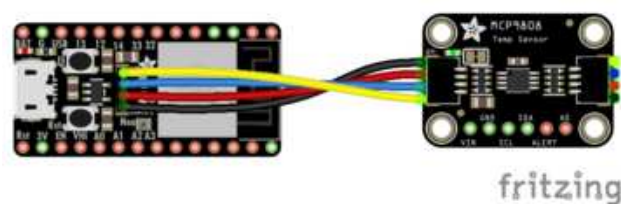
## STEMMA QT / Qwiic JST SH 4-Pin Cable - 50mm Long

This 4-wire cable is 50mm / 1.9" long and fitted with JST SH female 4-pin connectors on both ends. Compared with the chunkier JST PH these are 1mm pitch instead of 2mm, but...

<https://www.adafruit.com/product/4399>

## Wiring the MCP9808

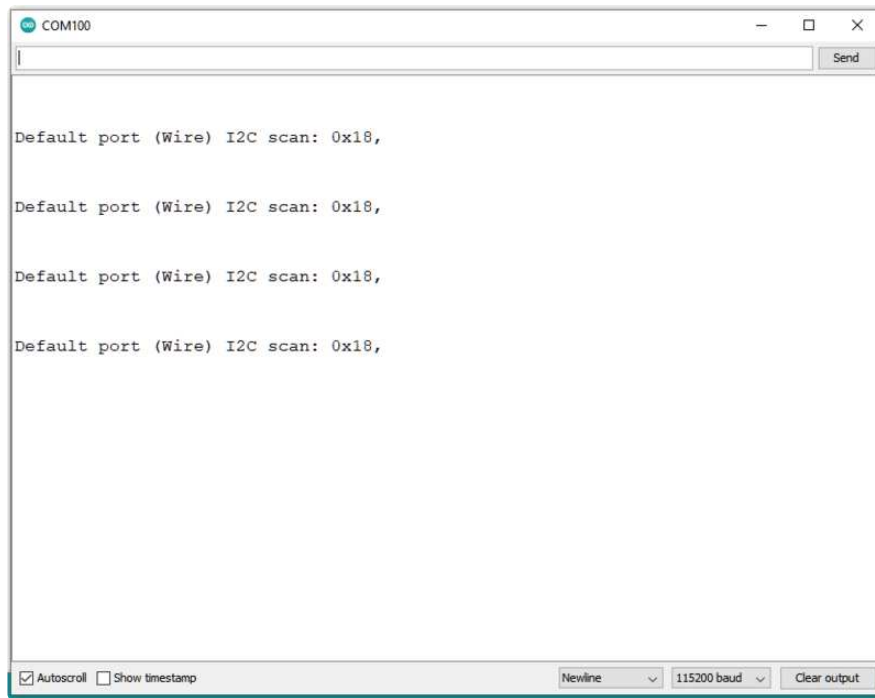
The MCP9808 comes with a STEMMA QT connector, which makes wiring it up quite simple and solder-free.



Plug a **STEMMA QT** to **STEMMA QT** cable into the port on the ItsyBitsy ESP32 and a port on the MCP9808 breakout.

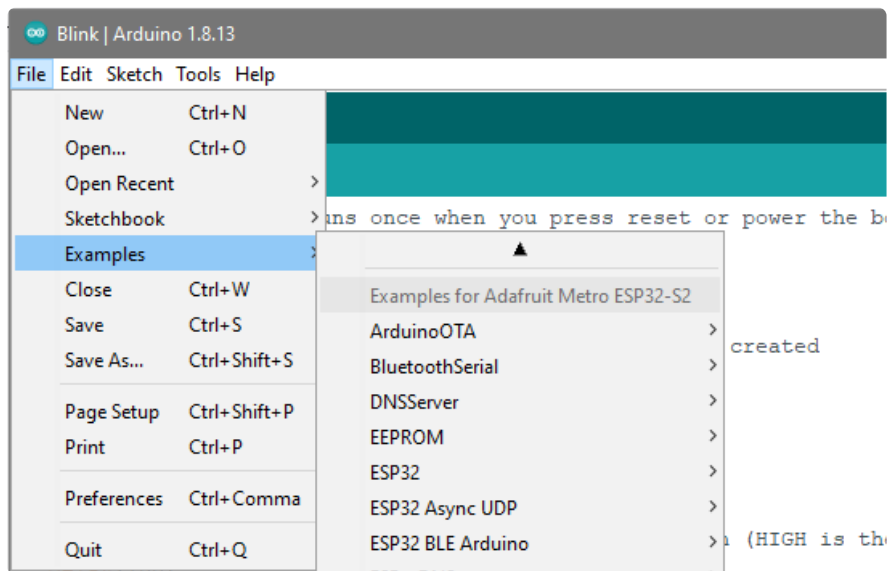
Now upload the scanning sketch to your microcontroller and open the serial port to see the output. You should see something like this:





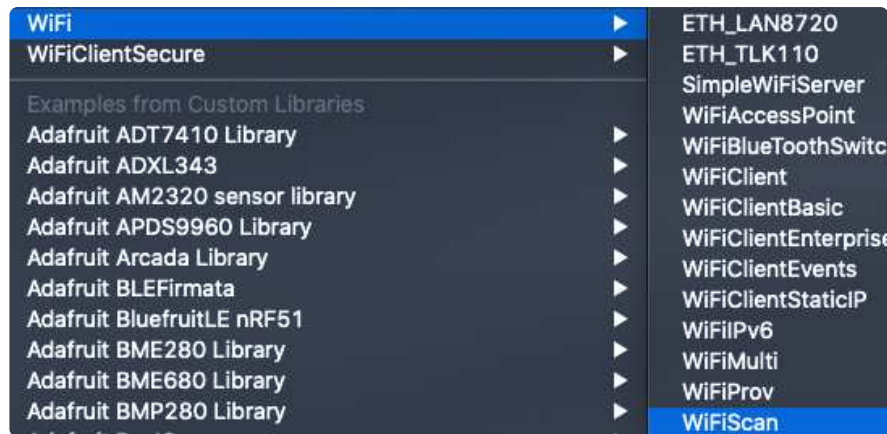
## WiFi Test

Thankfully if you have ESP32 sketches, they'll 'just work' with variations of ESP32. You can find a wide range of examples in the **File->Examples->Examples for Adafruit Metro ESP32-S2** subheading (the name of the board may vary so it could be "Examples for Adafruit Feather ESP32 V2" etc)



Let's start by scanning the local networks.

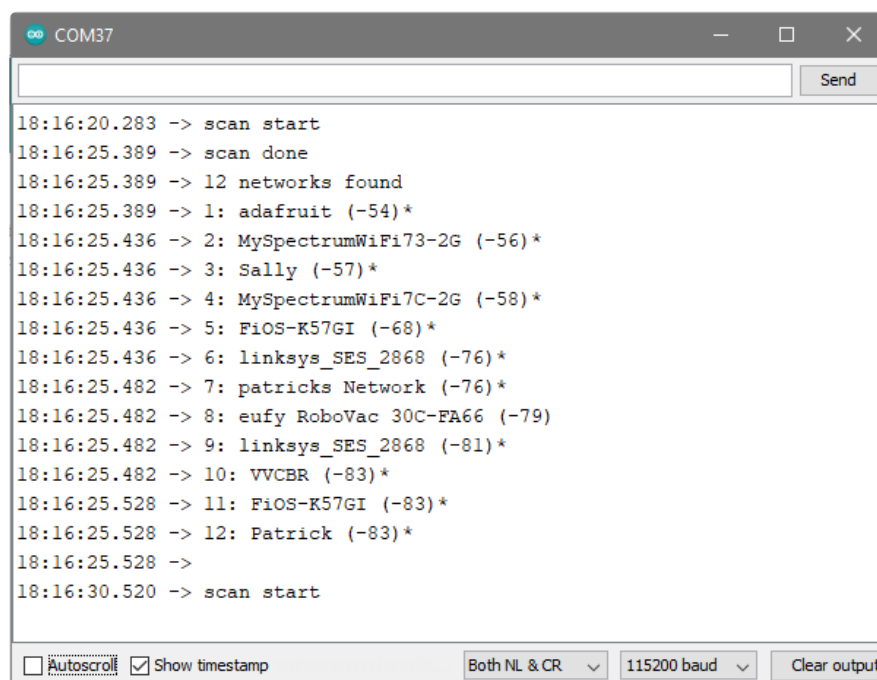
Load up the WiFiScan example under **Examples->Examples for YOUR BOARDNAME->WiFi->WiFiScan**



And **upload this example to your board**. The ESP32 should scan and find WiFi networks around you.

For ESP32, open the serial monitor, to see the scan begin.

For ESP32-S2, -S3 and -C3, don't forget you have to click Reset after uploading through the ROM bootloader. Then select the new USB Serial port created by the ESP32. It will take a few seconds for the board to complete the scan.



If you can not scan any networks, check your power supply. You need a solid power supply in order for the ESP32 to not brown out. A skinny USB cable or drained battery can cause issues.

## WiFi Connection Test

Now that you can scan networks around you, its time to connect to the Internet!

Copy the example below and paste it into the Arduino IDE:

```

// SPDX-FileCopyrightText: 2020 Brent Rubell for Adafruit Industries
//
// SPDX-License-Identifier: MIT

/*
  Web client

  This sketch connects to a website (wifitest.adafruit.com/testwifi/index.html)
  using the WiFi module.

  This example is written for a network using WPA encryption. For
  WEP or WPA, change the Wifi.begin() call accordingly.

  This example is written for a network using WPA encryption. For
  WEP or WPA, change the Wifi.begin() call accordingly.

  created 13 July 2010
  by dlf (Metodo2 srl)
  modified 31 May 2012
  by Tom Igoe
*/

#include <WiFi.h>

// Enter your WiFi SSID and password
char ssid[] = "YOUR_SSID";           // your network SSID (name)
char pass[] = "YOUR_SSID_PASSWORD";  // your network password (use for WPA, or use as key for WEP)
int keyIndex = 0;                    // your network key Index number (needed only for WEP)

int status = WL_IDLE_STATUS;
// if you don't want to use DNS (and reduce your sketch size)
// use the numeric IP instead of the name for the server:
//IPAddress server(74,125,232,128); // numeric IP for Google (no DNS)

char server[] = "wifitest.adafruit.com"; // name address for adafruit test
char path[] = "/testwifi/index.html";

// Initialize the Ethernet client library
// with the IP address and port of the server
// that you want to connect to (port 80 is default for HTTP):
WiFiClient client;

void setup() {
  //Initialize serial and wait for port to open:
  Serial.begin(115200);
  while (!Serial) {
    ; // wait for serial port to connect. Needed for native USB port only
  }

  // attempt to connect to Wifi network:
  Serial.print("Attempting to connect to SSID: ");
  Serial.println(ssid);

  WiFi.begin(ssid, pass);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("");
  Serial.println("Connected to WiFi");
  printWifiStatus();

  Serial.println("\nStarting connection to server...");
}

```

```

// if you get a connection, report back via serial:
if (client.connect(server, 80)) {
  Serial.println("connected to server");
  // Make a HTTP request:
  client.print("GET "); client.print(path); client.println(" HTTP/1.1");
  client.print("Host: "); client.println(server);
  client.println("Connection: close");
  client.println();
}
}

void loop() {
  // if there are incoming bytes available
  // from the server, read them and print them:
  while (client.available()) {
    char c = client.read();
    Serial.write(c);
  }

  // if the server's disconnected, stop the client:
  if (!client.connected()) {
    Serial.println();
    Serial.println("disconnecting from server.");
    client.stop();

    // do nothing forevermore:
    while (true) {
      delay(100);
    }
  }
}

void printWifiStatus() {
  // print the SSID of the network you're attached to:
  Serial.print("SSID: ");
  Serial.println(WiFi.SSID());

  // print your board's IP address:
  IPAddress ip = WiFi.localIP();
  Serial.print("IP Address: ");
  Serial.println(ip);

  // print the received signal strength:
  long rssi = WiFi.RSSI();
  Serial.print("signal strength (RSSI):");
  Serial.print(rssi);
  Serial.println(" dBm");
}

```

**NOTE:** You must change the **SECRET\_SSID** and **SECRET\_PASS** in the example code to your WiFi SSID and password before uploading this to your board.

```

// Enter your WiFi SSID and password
char ssid[] = "YOUR_SSID";           // your network SSID (name)
char pass[] = "YOUR_SSID_PASSWORD";  // your network password (use for WPA, or use as key for WEP)
int keyIndex = 0;                    // your network key Index number (needed only for WEP)

```

After you've set it correctly, upload and check the serial monitor. You should see the following. If not, go back, check wiring, power and your SSID/password

```

Attempting to connect to SSID: Transit
*****
Connected to WiFi
SSID: Transit
IP Address: 192.168.1.182
signal strength (RSSI):-57 dBm

Starting connection to server...
connected to server
HTTP/1.1 200 OK
Server: nginx/1.10.3 (Ubuntu)
Date: Wed, 11 Nov 2020 20:51:30 GMT
Content-Type: text/html
Content-Length: 70
Last-Modified: Thu, 16 May 2019 18:21:16 GMT
Connection: close
ETag: "5cddaa1c-46"
Accept-Ranges: bytes

This is a test of Adafruit WiFi!
If you can read this, its working :)

disconnecting from server.

```

## Secure Connection Example

Many servers today do not allow non-SSL connectivity. Lucky for you the ESP32 has a great TLS/SSL stack so you can have that all taken care of for you. Here's an example of a using a secure WiFi connection to connect to the Twitter API.

Copy and paste it into the Arduino IDE:

```

// SPDX-FileCopyrightText: 2015 Arturo Guadalupi
// SPDX-FileCopyrightText: 2020 Brent Rubell for Adafruit Industries
//
// SPDX-License-Identifier: MIT

/*
This example creates a client object that connects and transfers
data using always SSL.

It is compatible with the methods normally related to plain
connections, like client.connect(host, port).

Written by Arturo Guadalupi
last revision November 2015

*/

#include <WiFiClientSecure.h>
#include <WiFi.h>

// Enter your WiFi SSID and password
char ssid[] = "YOUR_SSID";           // your network SSID (name)

```

```

char pass[] =
"YOUR_SSID_PASSWORD";    // your network password (use for WPA, or use as key for
WEP)
int keyIndex =
0;                        // your network key Index number (needed only for WEP)

int status = WL_IDLE_STATUS;
// if you don't want to use DNS (and reduce your sketch size)
// use the numeric IP instead of the name for the server:
//IPAddress server(74,125,232,128); // numeric IP for Google (no DNS)

#define SERVER "cdn.syndication.twimg.com"
#define PATH   "/widgets/followbutton/info.json?screen_names=adafruit"

// Initialize the SSL client library
// with the IP address and port of the server
// that you want to connect to (port 443 is default for HTTPS):
WiFiClientSecure client;

void setup() {
  //Initialize serial and wait for port to open:
  Serial.begin(115200);
  while (!Serial) {
    ; // wait for serial port to connect. Needed for native USB port only
  }

  // attempt to connect to Wifi network:
  Serial.print("Attempting to connect to SSID: ");
  Serial.println(ssid);

  WiFi.begin(ssid, pass);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("");
  Serial.println("Connected to WiFi");
  printWifiStatus();

  client.setInsecure(); // don't use a root cert

  Serial.println("\nStarting connection to server...");
  // if you get a connection, report back via serial:
  if (client.connect(SERVER, 443)) {
    Serial.println("connected to server");
    // Make a HTTP request:
    client.println("GET " PATH " HTTP/1.1");
    client.println("Host: " SERVER);
    client.println("Connection: close");
    client.println();
  }
}

uint32_t bytes = 0;

void loop() {
  // if there are incoming bytes available
  // from the server, read them and print them:
  while (client.available()) {
    char c = client.read();
    Serial.write(c);
    bytes++;
  }

  // if the server's disconnected, stop the client:
  if (!client.connected()) {
    Serial.println();
    Serial.println("disconnecting from server.");
  }
}

```



```

    client.stop();
    Serial.print("Read "); Serial.print(bytes); Serial.println(" bytes");

    // do nothing forevermore:
    while (true);
  }
}

void printWifiStatus() {
  // print the SSID of the network you're attached to:
  Serial.print("SSID: ");
  Serial.println(WiFi.SSID());

  // print your board's IP address:
  IPAddress ip = WiFi.localIP();
  Serial.print("IP Address: ");
  Serial.println(ip);

  // print the received signal strength:
  long rssi = WiFi.RSSI();
  Serial.print("signal strength (RSSI):");
  Serial.print(rssi);
  Serial.println(" dBm");
}

```

As before, **update the ssid and password first**, then upload the example to your board.

Note we use **WiFiClientSecure client** instead of **WiFiClient client**; to require a SSL connection!

```

Attempting to connect to SSID: Transit
.....
Connected to WiFi
SSID: Transit
IP Address: 192.168.1.182
signal strength (RSSI):-52 dBm

Starting connection to server...
connected to server
HTTP/1.1 200 OK
Accept-Ranges: bytes
Access-Control-Allow-Origin: platform.twitter.com
Access-Control-Allow-Methods: GET
Age: 12
cache-control: must-revalidate, max-age=600
content-disposition: attachment; filename=json.json
Content-Type: application/json;charset=utf-8
Date: Wed, 11 Nov 2020 20:58:39 GMT
expires: Wed, 11 Nov 2020 21:08:39 GMT
Last-Modified: Wed, 11 Nov 2020 20:58:27 GMT
Server: ECS (agb/52BA)
strict-transport-security: max-age=631138519
timing-allow-origin: *
X-Cache: HIT
x-connection-hash: a50988a9020759ec70520caef6c38bcf
x-content-type-options: nosniff
x-frame-options: SAMEORIGIN
x-response-time: 12
x-transaction: 003d88570028acec
x-tw-cdn: VZ
x-tw-cdn: VZ
x-xss-protection: 0
Content-Length: 197
Connection: close

[{"following":false,"id":"20731304","screen_name":"adafruit","name":"adafruit industries",
disconnecting from server.
Read 966 bytes

```

---

# WipperSnapper Setup

The WipperSnapper firmware and ecosystem are in BETA and are actively being developed to add functionality, more boards, more sensors, and fix bugs. We encourage you to try out WipperSnapper with the understanding that it is not final release software and is still in development. If you encounter any bugs, glitches, or difficulties during the beta period, or with this guide, please contact us via <http://io.adafruit.com/support>

## What is WipperSnapper

WipperSnapper is a firmware designed to turn any WiFi-capable board into an Internet-of-Things device without programming a single line of code. WipperSnapper connects to [Adafruit IO \(https://adafru.it/fsU\)](https://adafru.it/fsU), a web platform designed ([by Adafruit! \(https://adafru.it/Bo5\)](https://adafru.it/Bo5)) to display, respond, and interact with your project's data.

Simply load the WipperSnapper firmware onto your board, add credentials, and plug it into power. Your board will automatically register itself with your Adafruit IO account.

From there, you can add components to your board such as buttons, switches, potentiometers, sensors, and more! Components are dynamically added to hardware, so you can immediately start interacting, logging, and streaming the data your projects produce without writing code.

## Sign up for Adafruit.io

You will need an Adafruit IO account to use WipperSnapper on your board. If you do not already have one, head over to [io.adafruit.com \(https://adafru.it/fsU\)](https://adafru.it/fsU) to create a free account.

## Install USB Driver

### Install CP2104 / CP2102N USB Driver

Many ESP32 boards have a USB-to-Serial converter that talks to the chip itself, and will need a driver on your computer's operating system. The driver is available for Mac, Windows, and Linux.

Click here to download the CP2104/  
CP2102N driver

<https://adafru.it/vrf>

## Install CH9102 / CH34X USB Driver

Newer ESP32 boards have a different USB-to-serial converter that talks to the chip itself, and will need a driver on your computer's operating system. The driver is available for Mac and Windows. It is already built into Linux.

If you would like more detail, check out [the guide on installing these drivers \(https://adafru.it/-f8\)](https://adafru.it/-f8).

Click here to download the  
Windows driver

<https://adafru.it/-f9>

Click here to download the Mac  
driver

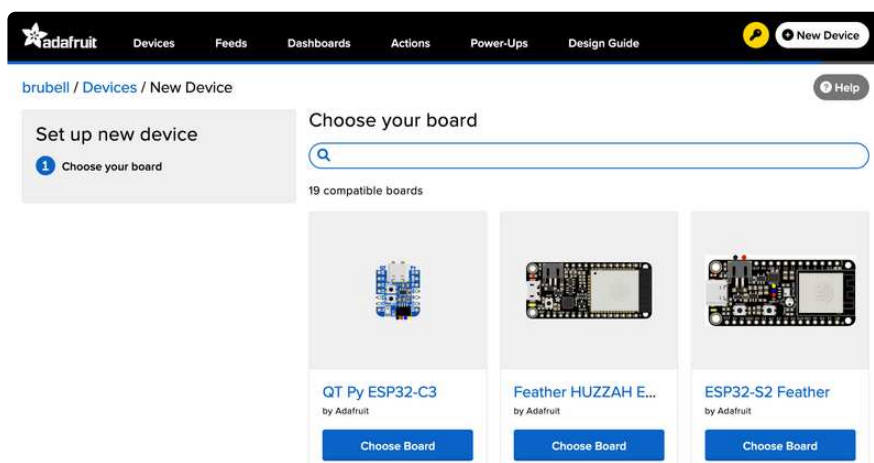
<https://adafru.it/-ed>

## Add a New Device to Adafruit IO

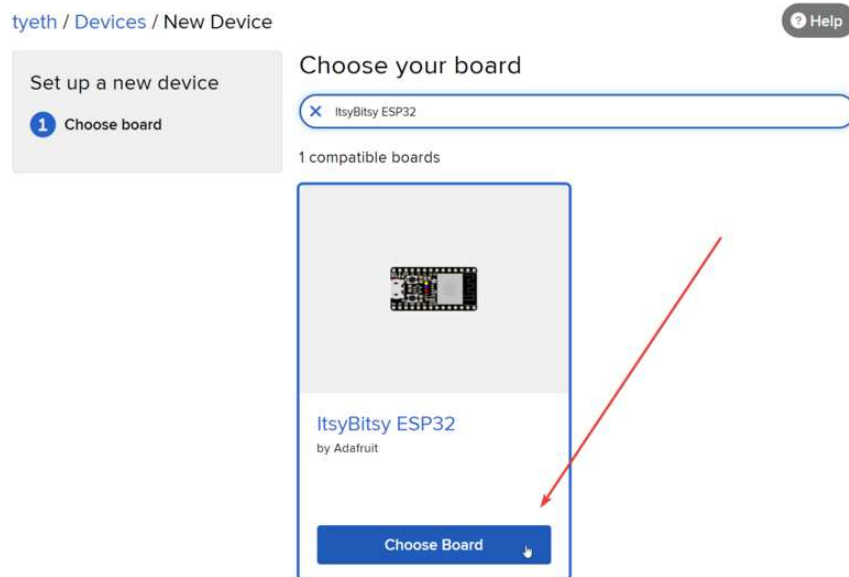
Log into your [Adafruit IO \(https://adafru.it/fsU\)](https://adafru.it/fsU) account. Click the New Device button at the top of the page.



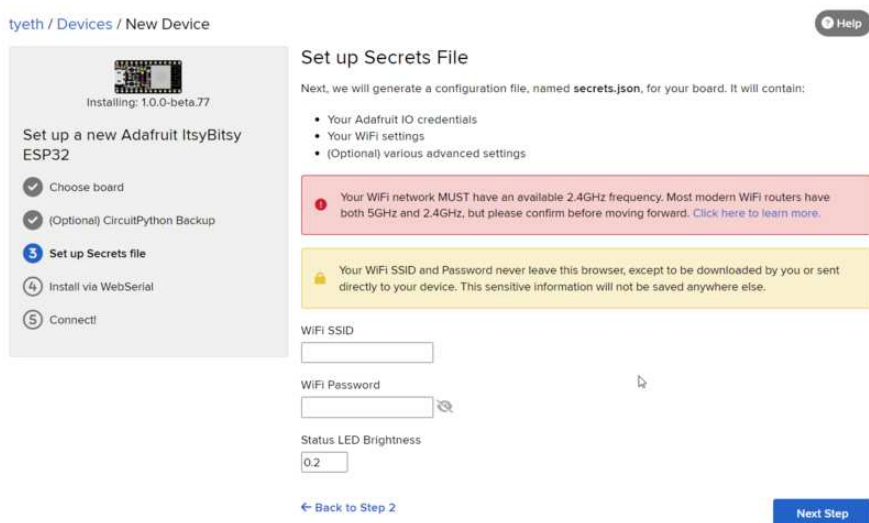
After clicking New Device, you should be on the board selector page. This page displays every board that is compatible with the WipperSnapper firmware.



In the board selector page's search bar, search for the ItsyBitsy ESP32. Once you've located the board you'd like to install WipperSnapper on, click the Choose Board button to bring you to the self-guided installation wizard.



Follow the step-by-step instructions on the page to install Wippersnapper on your device and connect it to Adafruit IO.



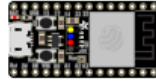
If the installation was successful, a popover should appear displaying that your board has successfully been detected by Adafruit IO.

Give your board a name and click "Continue to Device Page".

## New Device Detected!



You have successfully connected a new **itsybitsy-esp32** device to Adafruit IO. It is already set up and submitting data. You can name the device here, and set up components on the device page.



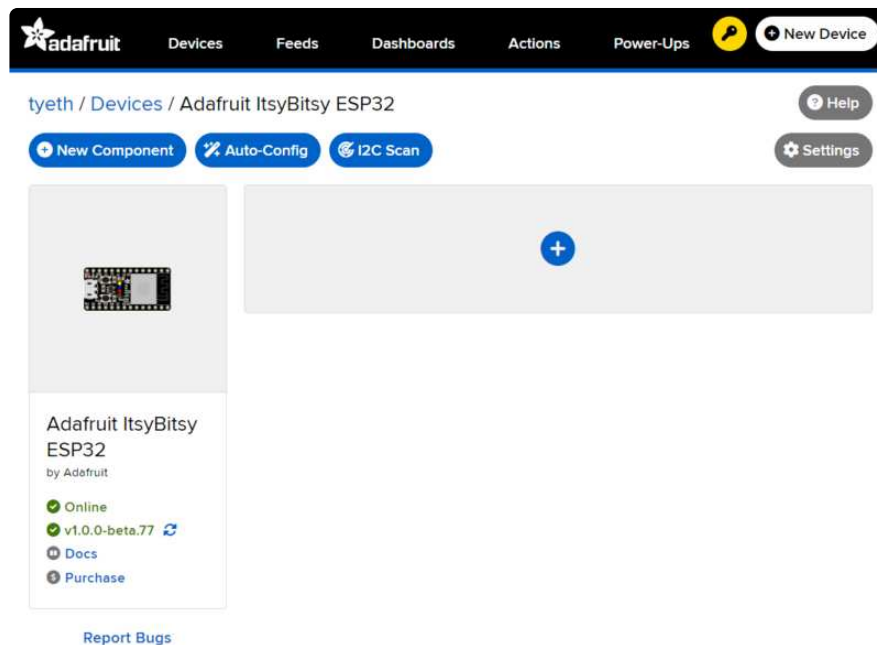
### Device Name

Adafruit ItsyBitsy ESP32

Firmware Version:  **v1.0.0-beta.77** 

[Continue to Device Page >](#)

You should be brought to your board's device page.



The screenshot shows the Adafruit IO web interface. At the top is a navigation bar with links: Devices, Feeds, Dashboards, Actions, Power-Ups, and a 'New Device' button. Below the navigation bar, the breadcrumb path 'tyeth / Devices / Adafruit ItsyBitsy ESP32' is visible. A row of buttons includes 'New Component', 'Auto-Config', 'I2C Scan', 'Help', and 'Settings'. The main content area features a large card for the 'Adafruit ItsyBitsy ESP32' device. The card displays the device's status as 'Online', the firmware version 'v1.0.0-beta.77', and links for 'Docs' and 'Purchase'. A 'Report Bugs' link is located at the bottom of the card. To the right of the device card is a large empty box with a blue plus sign, indicating where to add new components.

## Feedback

Adafruit.io WipperSnapper is in **beta** and you can help improve it!

If you have suggestions or general feedback about the installation process - visit <https://io.adafruit.com/support> (<https://adafru.it/Sgb>), click "Contact Adafruit IO Support" and select "I have feedback or suggestions for the WipperSnapper Beta".

## Troubleshooting

If you encountered an issue during installation, please try the steps below first.

If you're still unable to resolve the issue, or if your issue is not listed below, get in touch with us directly at <https://io.adafruit.com/support> (<https://adafru.it/Sgb>). Make sure to click "Contact Adafruit IO Support" and select "There is an issue with WipperSnapper. Something is broken!"

### I don't see my board on Adafruit IO, it is stuck connecting to WiFi

First, make sure that you selected the correct board on the board selector.

Next, please make sure that you entered your WiFi credentials properly, there are no spaces/special characters in either your network name (SSID) or password, and that you are connected to a 2.4GHz wireless network.

If you're still unable to connect your board to WiFi, please [make a new post on the WipperSnapper technical support forum with the error you're experiencing, the LED colors which are blinking, and the board you're using.](#) (<https://adafru.it/V6a>)

### I don't see my board on Adafruit IO, it is stuck "Registering with Adafruit IO"

Try hard-resetting your board by unplugging it from USB power and plugging it back in.

If the error is still occurring, please [make a new post on the WipperSnapper technical support forum with information about what you're experiencing, the LED colors which are blinking \(if applicable\), and the board you're using.](#) (<https://adafru.it/V6a>)



# "Uninstalling" WipperSnapper

WipperSnapper firmware is an application that is loaded onto your board. There is nothing to "uninstall". However, you may want to "move" your board from running WipperSnapper to running Arduino or CircuitPython. You also may need to restore your board to the state it was shipped to you from the Adafruit factory.

## Moving from WipperSnapper to CircuitPython

Follow the steps on the [Installing CircuitPython page \(https://adafru.it/Amd\)](https://adafru.it/Amd) to install CircuitPython on your board running WipperSnapper.

- If you are unable to double-tap the RST button to enter the UF2 bootloader, follow the "Factory Resetting a WipperSnapper Board" instructions below.

Uploading this sketch will overwrite WipperSnapper. If you want to re-install WipperSnapper, follow the instructions at the top of this page.

## Moving from WipperSnapper to Arduino

If you want to use your board with Arduino, you will use the Arduino IDE to load any sketch onto your board.

First, follow the page below to set up your Arduino IDE environment for use with your board.

**Setup Arduino IDE**

<https://adafru.it/19FK>

Then, follow the page below to upload the "Arduino Blink" sketch to your board.

**Upload Arduino Blink Sketch**

<https://adafru.it/19FL>

Uploading this sketch will overwrite WipperSnapper. If you want to re-install WipperSnapper, follow the instructions at the top of this page.

## Factory Resetting a WipperSnapper Board

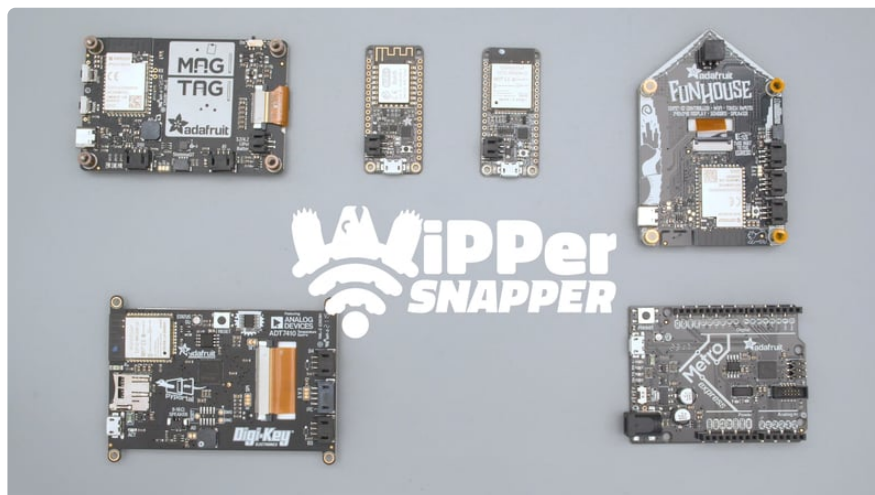
Sometimes, hardware gets into a state that requires it to be "restored" to the original state it shipped in. If you'd like to get your board back to its original factory state, follow the guide below.

**Factory Reset page**

<https://adafru.it/19FM>

---

# WipperSnapper Essentials



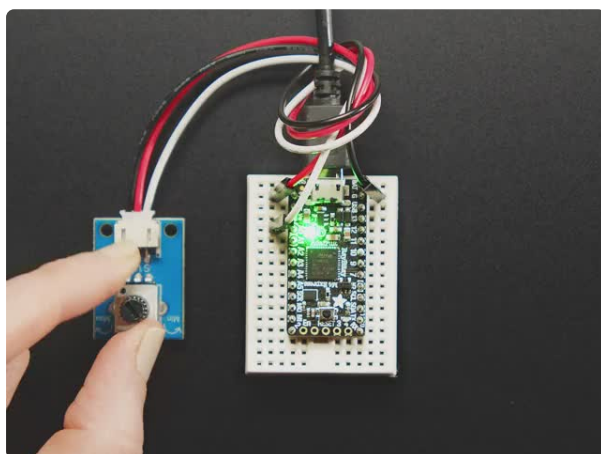
You've installed WipperSnapper firmware on your board and connected it to Adafruit IO. Next, let's learn how to use Adafruit IO!

The Adafruit IO supports a large number of components. Components are physical parts such as buttons, switches, sensors, servos, LEDs, RGB LEDs, and more.

The following pages will get you up and running with WipperSnapper as you interact with your board's LED, read the value of a push button, send the value of an I2C sensor to the internet, and wirelessly control colorful LEDs.

## Parts

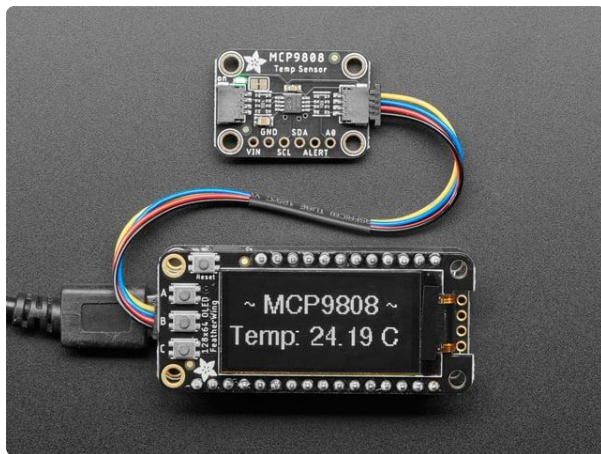
The following parts are **required** to complete the WipperSnapper essentials pages for this board:



### [STEMMA Wired Potentiometer Breakout Board - 10K ohm Linear](https://www.adafruit.com/product/4493)

For the easiest way possible to measure twists, turn to this STEMMA potentiometer breakout (ha!). This plug-n-play pot comes with a JST-PH 2mm connector and a matching

<https://www.adafruit.com/product/4493>



### Adafruit MCP9808 High Accuracy I2C Temperature Sensor Breakout

The MCP9808 digital temperature sensor is one of the more accurate/precise we've ever seen, with a typical accuracy of  $\pm 0.25^{\circ}\text{C}$  over the sensor's  $-40^{\circ}\text{C}$  to...

<https://www.adafruit.com/product/5027>



### STEMMA QT / Qwiic JST SH 4-pin Cable - 100mm Long

This 4-wire cable is a little over 100mm / 4" long and fitted with JST-SH female 4-pin connectors on both ends. Compared with the chunkier JST-PH these are 1mm pitch instead of...

<https://www.adafruit.com/product/4210>

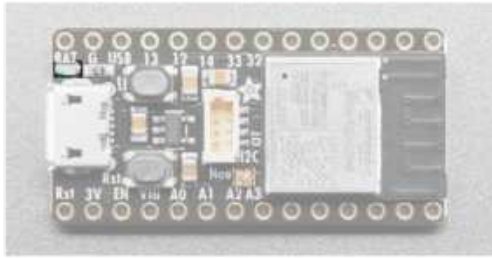
---

## LED Blink

This demo shows controlling an LED from Adafruit IO. The same kind of control can be used for relays, lights, motors, or solenoids.

One of the first programs you typically write to get used to embedded programming is a sketch that repeatably blinks an LED. IoT projects are wireless, so after completing this section, you'll be able to turn on (or off) the LED built into your board from anywhere in the world.

## Where is the LED on my board?

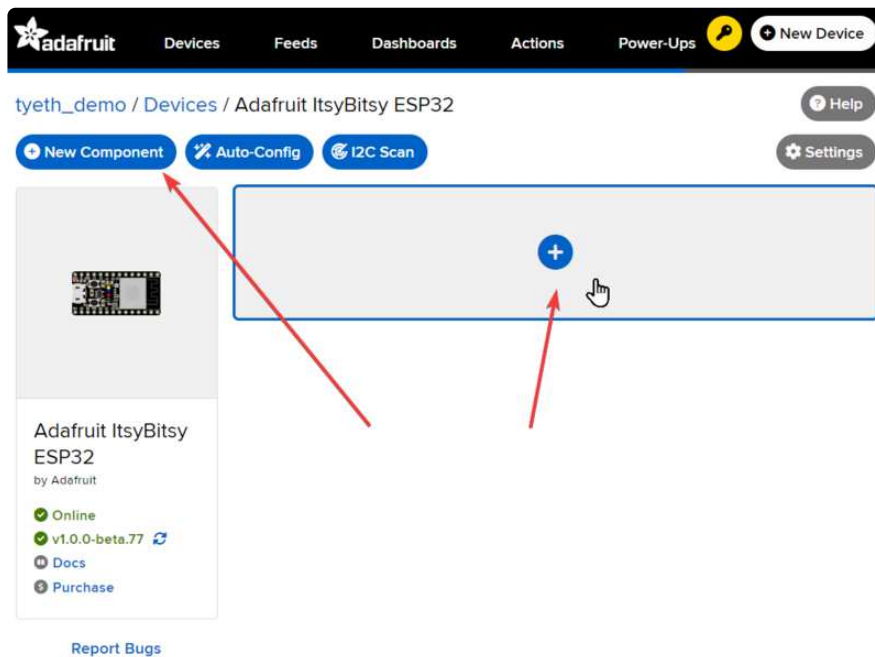


A little red LED is just beneath the pin labelled **BAT** on the board, above the USB connector (when on the left side).

It's connected to pin GPIO13 (**D13**)

## Create a LED Component on Adafruit IO

On the device page, click the New Component (or "+") button to open the component picker.



### New Component

Which component would you like to set up?

X LED

Displaying 3 matching Components.



Search for the component name by entering **LED** into the text box on the component picker, the list of components should update as soon as you stop typing.



## Filtering and searching for components

Since WipperSnapper supports such a large number of components, there is keyword filtering. Try searching for various keywords, like:

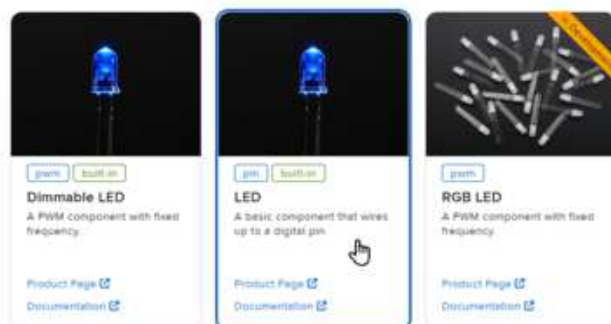
- component names: `aht20`, `servo`, `buzzer`, `button`, `led`, etc
- sensor types: `light`, `temperature`, `pressure`, `humidity`, etc
- interface: `i2c`, `uart`, `ds18x20`, `pin`, etc (also I2C addresses e.g. `0x44`)
- vendor: `Adafruit`, `ASAIR`, `Infineon`, `Bosch`, `Honeywell`, `Sensirion`, etc

There are also product and documentation links to every component. Follow the links beneath the component descriptions to be taken to the appropriate product page or Learn Guide

Which component would you like to set up?

X LED

Displaying 3 matching Components.



Select the **LED** from the list of components.

On the Create LED Component form, the board's LED pin is pre-selected.

Click Create Component.

Create LED Component ✕

Settings

LED Name

LED

LED Pin

D13 (LED BUILT-IN) ▾



[← Back to Component Type](#)

Create Component 

Behind the scenes, Adafruit IO sends a command to your board running WipperSnapper telling it to configure "LED Pin" as a digital output.

Your board's page on Adafruit IO shows a new LED component.

tyeth\_demo / Devices / Adafruit ItsyBitsy ESP32 Help

New Component Auto-Config I2C Scan

Settings



Adafruit ItsyBitsy ESP32  
by Adafruit  
Online  
v1.0.0-beta.77  
Docs  
Purchase  
Report Bugs

LED D13 led 

Create Action | Add to Dashboard 

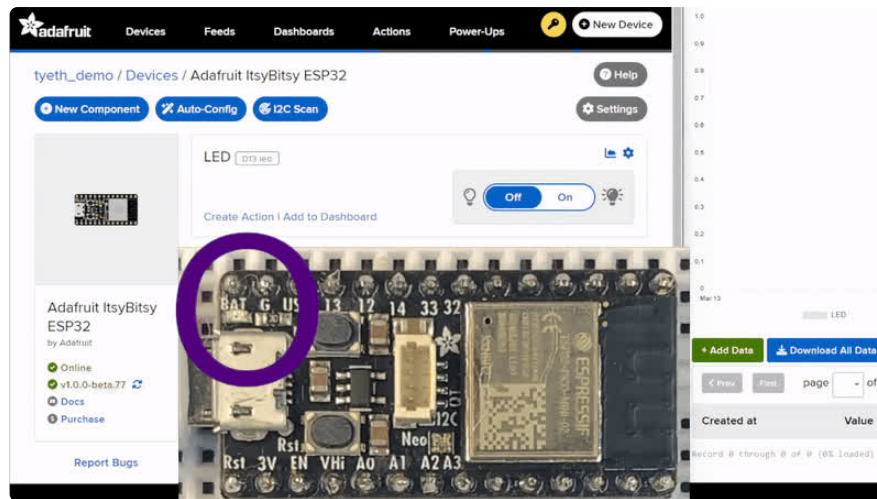
Off On



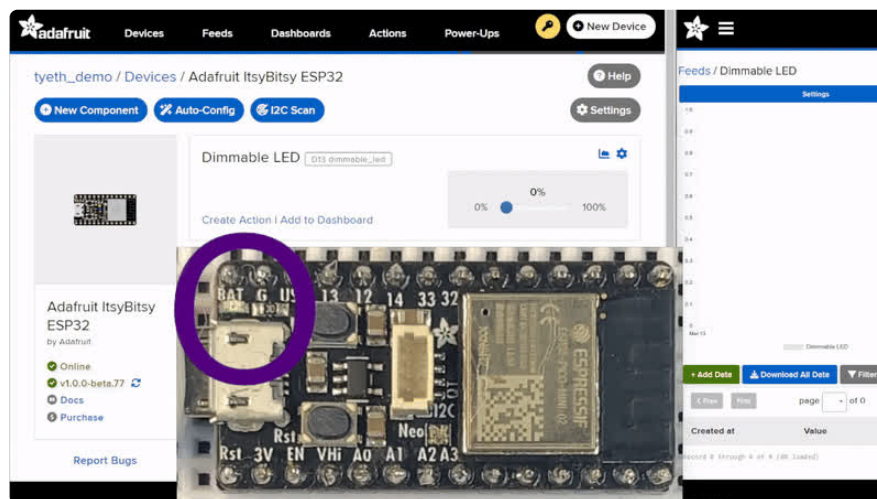
## Usage

On the board page, toggle the LED component by clicking the toggle switch. This should turn your board's built-in LED on or off.





The built-in LED pin (**D13**) also supports PWM, and can instead be used as a **Dimmable LED** component as shown here:



## Read a Push-button

This demo shows reading the state of a push-button from WipperSnapper. But the same kind of control can be used for reading switches, break beam sensors, and other digital sensors.

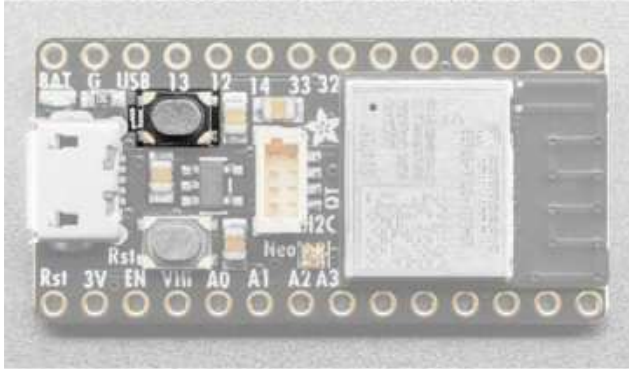
You can configure a board running WipperSnapper to read data from standard input buttons, switches, or digital sensors, and send the value to Adafruit IO.

From Adafruit IO, you will configure one of the pushbuttons on your board as a push button component. Then, when the button is pressed (or released), a value will be published to Adafruit IO.



## Button Location

This example uses the board's built-in push-button and internal pull-up resistor instead of wiring a push-button up.

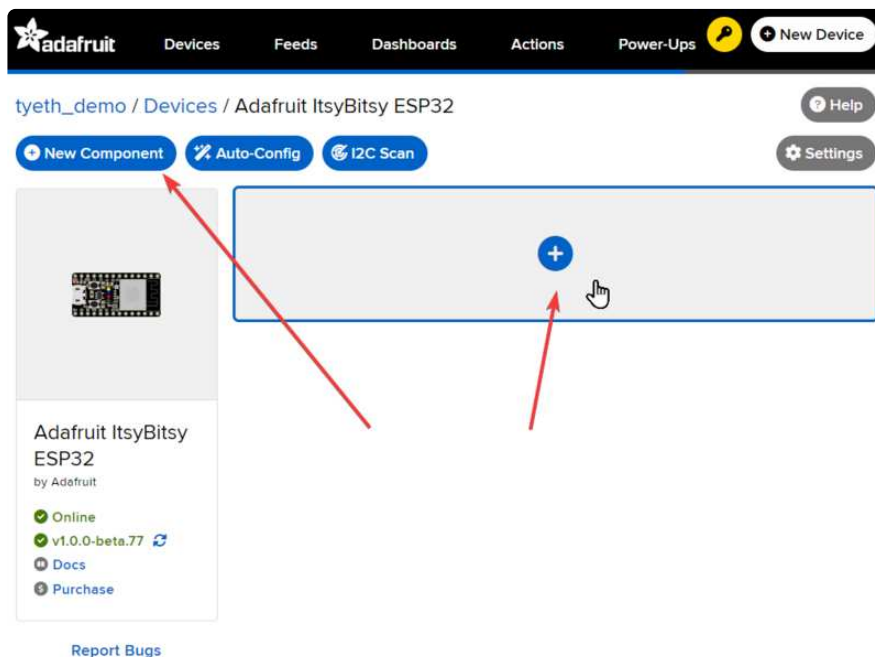


There is a user readable button switch to use as an input, labeled **U** on the silk-screen (There's a via in the middle of the U so it's hard to see until you know).

You can read it on pin GPIO 35 (D35).

## Create a Push-button Component on Adafruit IO

On the device page, click the New Component (or "+") button to open the component picker.



## New Component

Which component would you like to set up?

Displaying 1 matching Components.



Search for the component name by entering **push** into the text box on the component picker, the list of components should update as soon as you stop typing.



## Filtering and searching for components

Since WipperSnapper supports such a large number of components, you can use filtering. Try searching for various keywords, like:

- component names: **aht20**, **servo**, **buzzer**, **button**, **potentiometer**, etc
- sensor types: **light**, **temperature**, **pressure**, **humidity**, etc
- interface: **i2c**, **uart**, **ds18x20**, **pin**, etc (also I2C addresses e.g. **0x44**)
- vendor: **Adafruit**, **ASAIR**, **Infineon**, **Bosch**, **Honeywell**, **Sensirion**, etc

There are also added product and documentation links for every component. Follow the links beneath the component descriptions to be taken to the appropriate product page or Learn Guide.

Which component would you like to set up?

Displaying 1 matching Components.



Select the **Push Button** from the list of results to go to the component configuration page.

There will be a back button if you select the wrong component, and you can use the Edit component icon (⚙️) on the device page to update the component configuration in the future.

The "Create Push Button Component" form presents you with options for configuring the push button.

Start by selecting the board's pin connected to the push button.

Create Push Button Component

Settings

Push Button Name

Push Button

Push Button Pin

D35 (User BUTTON)

Return Interval

☐ On Change

☒ Periodically

Period

Every 30 seconds

☐ Specify Pin Pull Direction?

← Back to Component Type

Create Component



The Return Interval dictates how frequently the value of the push-button will be sent from the board to Adafruit IO.

For this example, you will configure the push button's value to be only sent when the value changes (i.e. when it's either pressed or depressed).

Create Push Button Component

Settings

Push Button Name

Push Button

Push Button Pin

D35 (User BUTTON)

Return Interval

☒ On Change

☐ Periodically

☐ Specify Pin Pull Direction?

← Back to Component Type

Create Component



Finally, check the **Specify Pin Pull Direction** checkbox and select the pull direction.

Create Push Button Component

×

Settings

Push Button Name

Push Button

Push Button Pin

D35 (User BUTTON)

Return Interval

☒ On Change
 ☐ Periodically

☒ Specify Pin Pull Direction?
 

☒ Pull Up
 ☐ Pull Down

← Back to Component Type

Create Component



Make sure the form's settings look like the following screenshot. Then, click **Create Component**.

Create Push Button Component

×

Settings

Push Button Name

Push Button

Push Button Pin

D35 (User BUTTON)

Return Interval

☒ On Change
 ☐ Periodically

☒ Specify Pin Pull Direction?
 

☒ Pull Up
 ☐ Pull Down

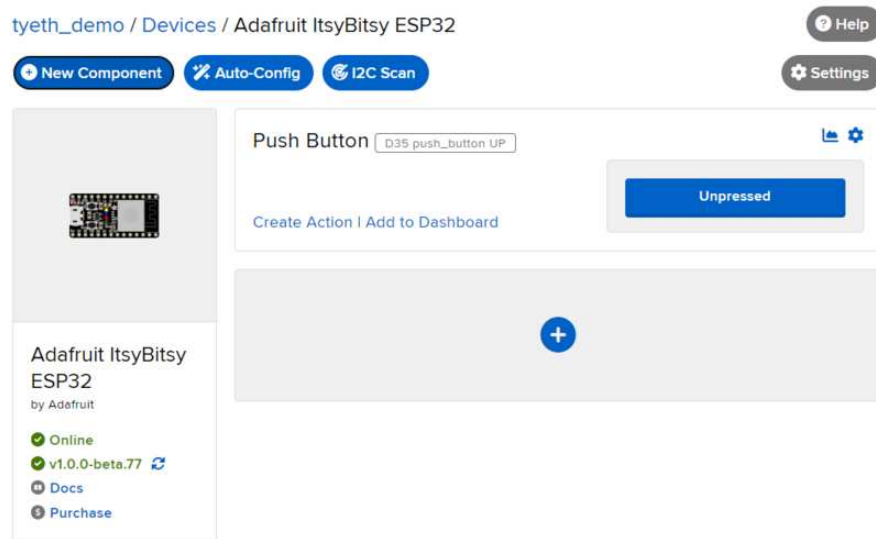
← Back to Component Type

Create Component

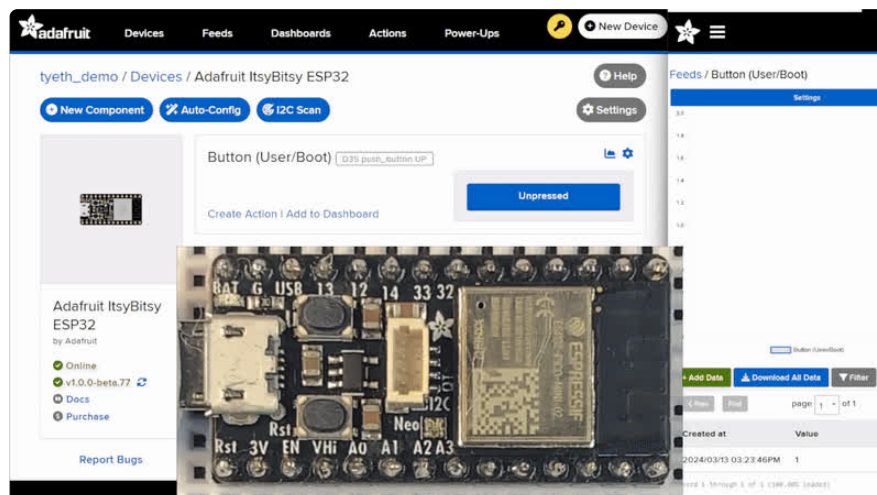


Adafruit IO sends a command to your WipperSnapper board, telling it to configure the GPIO pin you selected to behave as a digital input pin and to enable it to pull up the internal resistor.

Your board's page should also show the new push-button component.



Push the button on your board to change the value of the push-button component on Adafruit IO.

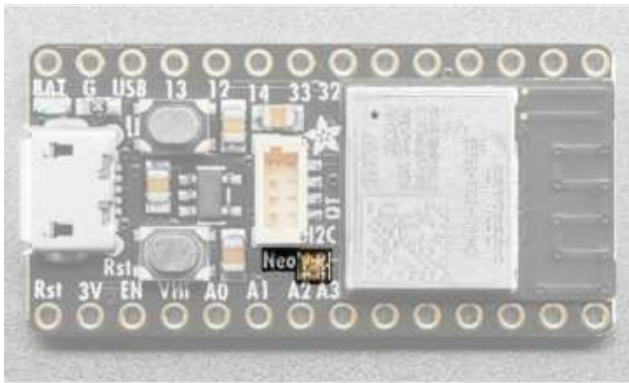


## NeoPixel LED

Your board has a WS281x RGB LED (NeoPixel, in Adafruit jargon) built in. Boards running the WipperSnapper firmware can be wirelessly controlled by Adafruit IO to interact with NeoPixels.

On this page, you'll learn how to change the color and brightness of the NeoPixel built into your board from Adafruit IO.

## Where is the NeoPixel on my board?

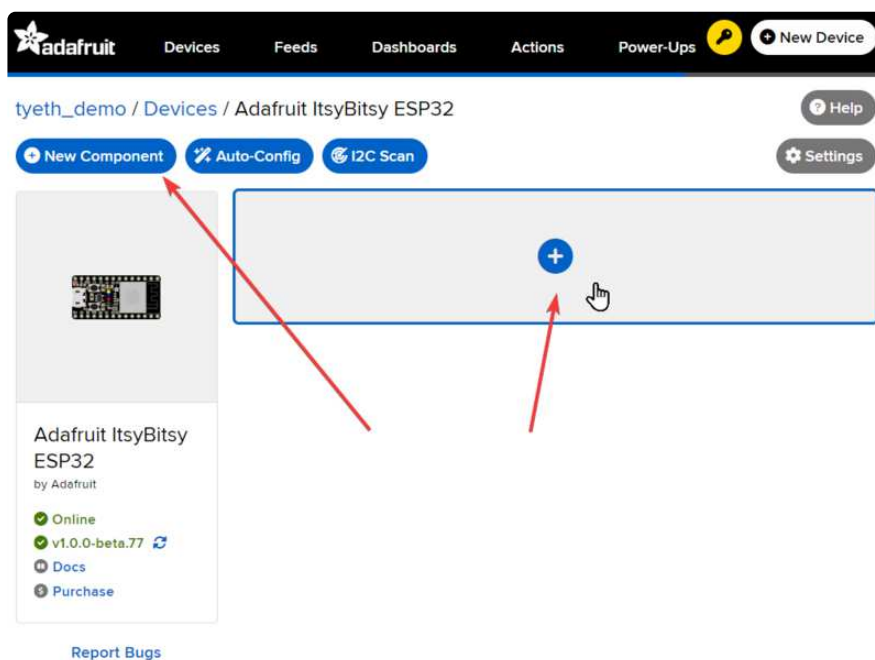


In the center of the board, to the left of the ESP32 module and just below the StemmaQT / Qwiic connector is the **NeoPixel**. This addressable RGB LED works both as a status LED, and can be controlled by the user (it's a status LED at boot time or if not in use).

It is connected to GPIO0 (D0).

## Create the NeoPixel Component

On the device page, click the New Component (or "+") button to open the component picker.



## New Component

Which component would you like to set up?

Displaying 1 matching Components.



Search for the component name by entering **neopixel** into the text box on the component picker, the list of components should update as soon as you stop typing

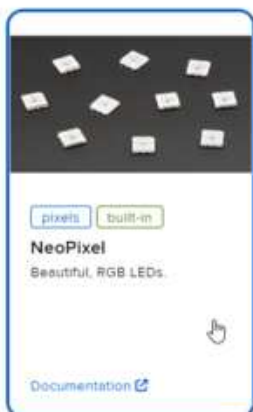
### ? Filtering and searching for components

Since WipperSnapper supports such a large number of components, you can use keyword filtering. Try searching for various keywords, like:

- component names: **aht20** , **servo** , **buzzer** , **button** , **neopixel** , etc
- sensor types: **light** , **temperature** , **pressure** , **humidity** , etc
- interface: **i2c** , **uart** , **ds18x20** , **pin** , etc (also I2C addresses e.g. **0x44** )
- vendor: **Adafruit** , **ASAIR** , **Infineon** , **Bosch** , **Honeywell** , **Sensirion** , etc

There is also added product and documentation links for every component, follow the links beneath the component descriptions to be taken to the appropriate product page or Learn Guide

Displaying 1 matching Components.



Select the **NeoPixel** from the list of results to go to the component configuration page.

There will be a back button if you select the wrong component, and you can use the Edit component icon (⚙️) on the device page to update the component configuration in the future.



The board NeoPixel pin is automatically found and selected.

Click **Create Component**

Create NeoPixel Component

Settings

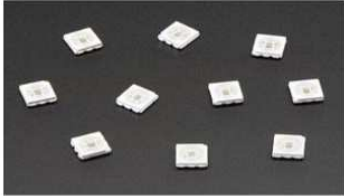
NeoPixel Name

NeoPixel Pin

Number of Pixels

Color Order

Brightness

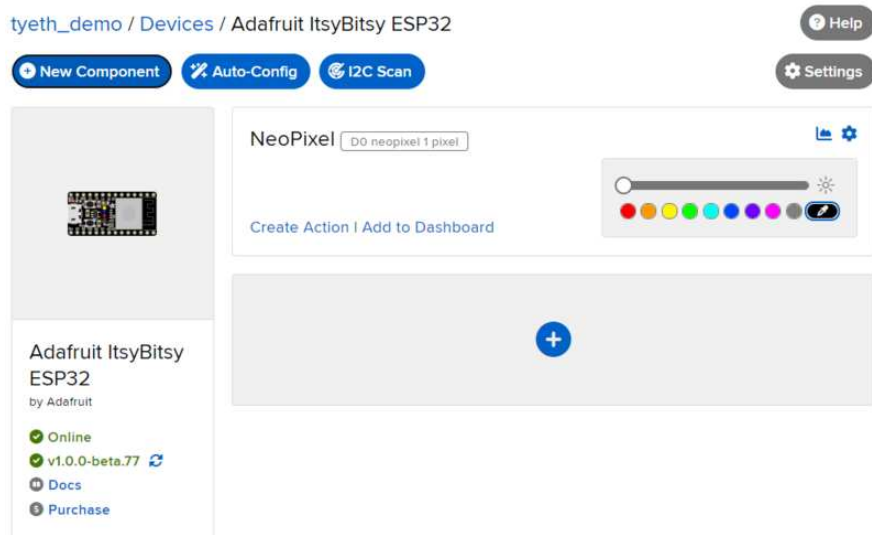


← Back to Component Type

Create Component

Behind the scenes, Adafruit IO sends a command to your board running WipperSnapper firmware telling it to configure the pin as a NeoPixel component with the settings from the form.

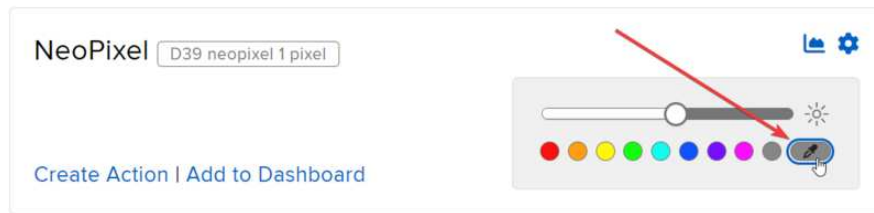
The Device page shows the NeoPixel component.



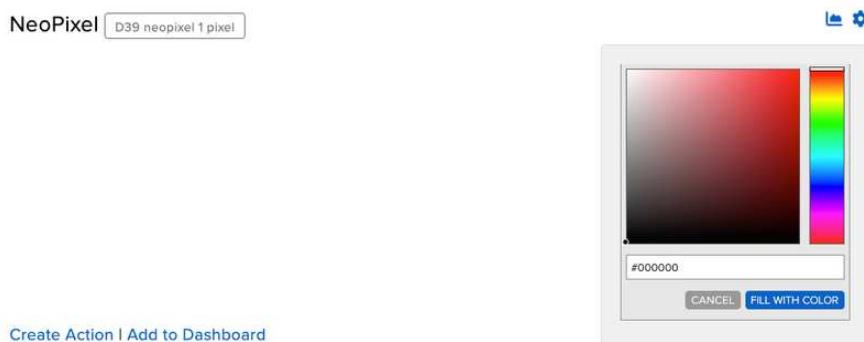
## Set the NeoPixel's RGB Color

Since no colors have been set yet, the color picker's default value is **#000000** (black in hex color code) and appears "off". You can change that to make the NeoPixel shine brightly!

On the NeoPixel component, click the color dropper at the end of the color swatch list.



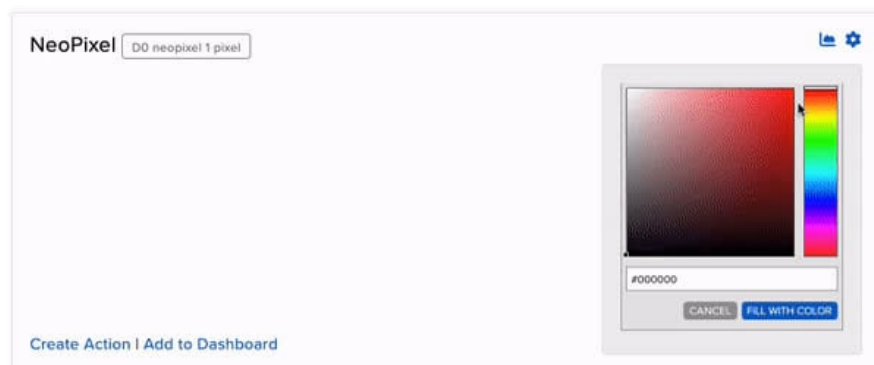
A color picker pops open! Next: learning how Adafruit IO uses hex color codes to represent the colors on your NeoPixel.



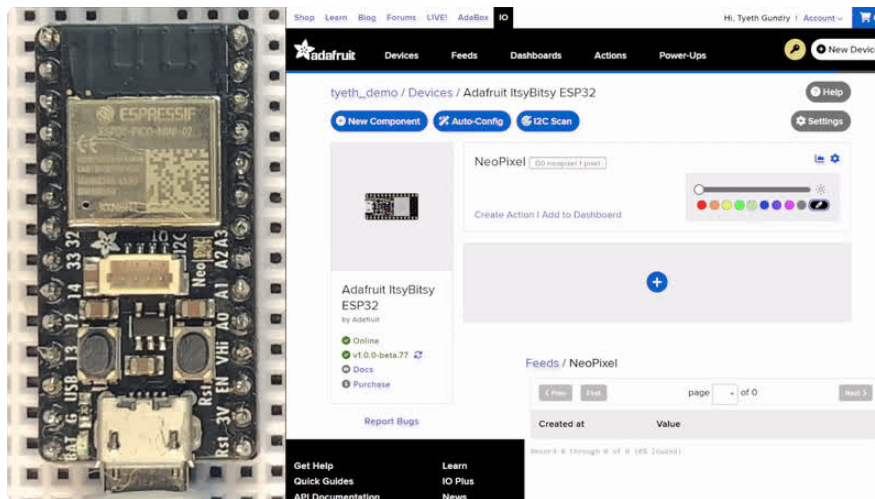
## Hex Colors 101

The color picker on Adafruit IO uses hex color codes to represent Red, Green, and Blue values. For example, `#FF0000` is the hex color code for the color red. The colors (`#FF0000`) red component is `FF` (255 translated to decimal), the green component is `00` and the blue component is `00`. Translated to RGB format, the color is `RGB (255, 0, 0)`.

Using the color picker, or by manually entering a hex color code, select a color.

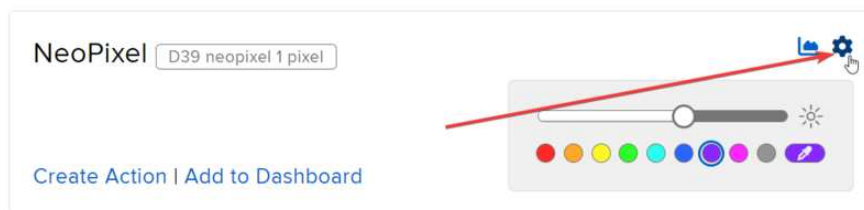


When you're ready to set the color of your device's NeoPixel, click **FILL WITH COLOR**. The NeoPixel will immediately glow!



## Set NeoPixel Brightness

If the NeoPixel is too bright (or too dim), you can change the overall brightness. Click the gear/cog icon on the NeoPixel component to open its settings.



On the NeoPixel component form, set Brightness to a value between 0 (fully off) and 255 (full brightness).

Click the **Update Component** button to send the updated configuration to your device.

Edit NeoPixel Component

Settings

NeoPixel Name

NeoPixel Pin

Number of Pixels

Color Order

Brightness

---

# I2C Sensor

While this page uses the "MCP9808 High Accuracy I2C Temperature Sensor Breakout", the process for adding an I2C sensor to your board running WipperSnapper is similar for all I2C sensors.

Inter-Integrated Circuit, aka **I2C**, is a two-wire protocol for connecting sensors and "devices" to a microcontroller. A large number of sensors, including the ones sold by Adafruit, use I2C to communicate.

Typically, using I2C with a microcontroller involves programming. Adafruit IO and WipperSnapper let you configure a microcontroller to read data from an I2C sensor and publish that data to the internet without writing code.

The WipperSnapper firmware supports a number of I2C sensors, [viewable in list format here \(https://adafru.it/Zbq\)](https://adafru.it/Zbq).

- If you do not see the I2C sensor you're attempting to use with WipperSnapper, [Adafruit has a guide on adding a component to Adafruit IO WipperSnapper here \(https://adafru.it/Zbr\)](https://adafru.it/Zbr).

On this page, you'll learn how to wire up an I2C sensor to your board. Then, you'll create a new component on Adafruit IO for your I2C sensor and send the sensor values to Adafruit IO. Finally, you'll learn how to locate, interpret, and download the data produced by your sensors.

## Parts

You will need the following parts to complete this page:



### [Adafruit MCP9808 High Accuracy I2C Temperature Sensor Breakout](https://www.adafruit.com/product/5027)

The MCP9808 digital temperature sensor is one of the more accurate/precise we've ever seen, with a typical accuracy of  $\pm 0.25^{\circ}\text{C}$  over the sensor's  $-40^{\circ}\text{C}$  to... <https://www.adafruit.com/product/5027>

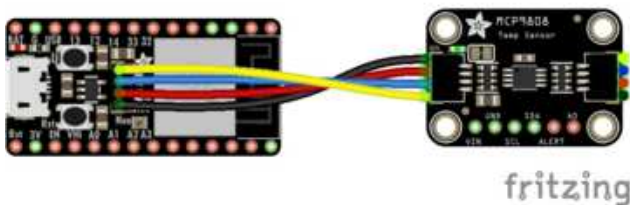


## STEMMA QT / Qwiic JST SH 4-pin Cable - 100mm Long

This 4-wire cable is a little over 100mm / 4" long and fitted with JST-SH female 4-pin connectors on both ends. Compared with the chunkier JST-PH these are 1mm pitch instead of...

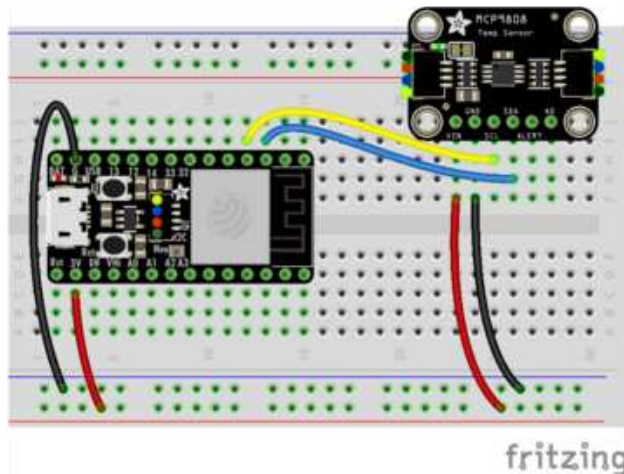
<https://www.adafruit.com/product/4210>

## Wiring



If you're using a STEMMA QT to STEMMA QT cable:

**Board STEMMA QT Port to MCP9808's STEMMA QT Port**

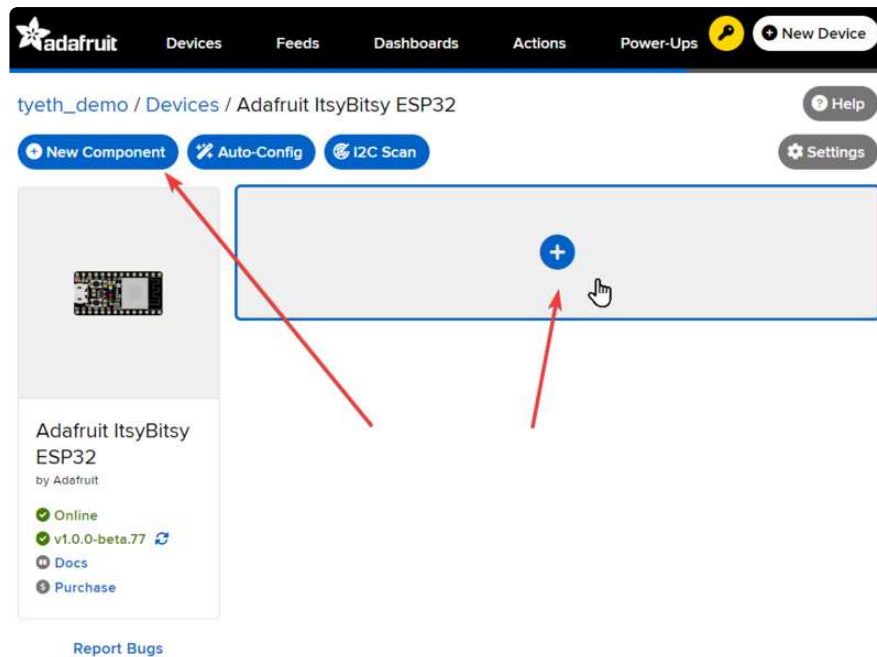


If you're using a breadboard:

**Board power to MCP9808 VIN**  
**Board ground to MCP9808 GND**  
**Board SCL to MCP9808 SCL**  
**Board SDA to MCP9808 SDA**

## Add an MCP9808 Component

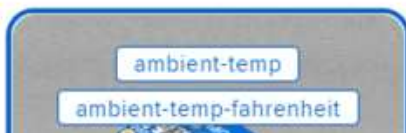
On the device page, click the New Component (or "+") button to open the component picker.



## New Component

Which component would you like to set up?

Displaying 1 matching Components.



Search for the component name by entering **MCP9808** into the text box on the component picker, the list of components should update as soon as you stop typing.

## ? Filtering and searching for components

Since WipperSnapper supports such a large number of components, there is keyword filtering. Try searching for various keywords, like:

- component names: **ah20**, **servo**, **buzzer**, **button**, **potentiometer**, etc
- sensor types: **light**, **temperature**, **pressure**, **humidity**, etc
- interface: **i2c**, **uart**, **ds18x20**, **pin**, etc (also I2C addresses e.g. **0x44**)
- vendor: **Adafruit**, **ASAIR**, **Infineon**, **Bosch**, **Honeywell**, **Sensirion**, etc



There are added product and documentation links for every component, follow the links beneath the component descriptions to be taken to the appropriate product page or Learn Guide.



Select the **MCP9808** from the list of results to go to the component configuration page.

There will be a back button if you select the wrong component, and you can use the Edit component icon (⚙️) on the device page to update the component configuration in the future.

On the component configuration page, the MCP9808's I2C sensor address should be listed along with the sensor's settings.

#### Create MCP9808 Component

Select I2C Address:

☒ Enable MCP9808: Temperature Sensor (°C)?  
Name:

Send Every:

☒ Enable MCP9808: Temperature Sensor (°F)?  
Name:

Send Every:



The MCP9808 sensor can measure ambient temperature. This page has individual options for reading the ambient temperature, in either Celsius or Fahrenheit. You may select the readings which are appropriate to your application and region.

The **Send Every** option is specific to each sensor measurement. This option will tell the board how often it should read from the sensor and send the data to Adafruit IO. Measurements can range from every 30 seconds to every 24 hours.



For this example, set the **Send Every** interval for both seconds to **Every 30 seconds**. Click **Create Component**.

### Create MCP9808 Component ✕

Select I2C Address:  

0x18

☐ Enable MCP9808: Temperature Sensor (°C)?  
☒ Enable MCP9808: Temperature Sensor (°F)?  
Name:  

MCP9808: Temperature Sensor (°F)

Send Every:  

Every 30 seconds



[← Back to Component Type](#)

Create Component

The board page should now show the MCP9808 component you created. After the interval you configured elapses, the WipperSnapper firmware running on your board automatically reads values from the sensor and sends them to Adafruit IO.

tyeth\_demo / Devices / Adafruit ItsyBitsy ESP32

New Component

Auto-Config

I2C Scan

Help

Settings



Adafruit ItsyBitsy ESP32  
by Adafruit  
Online  
v1.0.0-beta.77  
Docs  
Purchase  
Report Bugs

MCP9808: Temperature Sensor (°C)  
mcp9808:ambient-temp  
19.81°C  
Create Action | Add to Dashboard

MCP9808: Temperature Sensor (°F)  
mcp9808:ambient-temp-fahrenheit  
67.66°F  
Create Action | Add to Dashboard

+

## Read I2C Sensor Values

Now to look behind the scenes at a powerful element of using Adafruit IO and WipperSnapper. When a new component is created on Adafruit IO, an [Adafruit IO Feed](https://adafru.it/ioA) (<https://adafru.it/ioA>) is also created. This Feed holds your sensor component values for long-term storage (30 days of storage for Adafruit IO Free and 60 days for Adafruit IO Plus plans).

Aside from holding the **values** read by a sensor, the component's feed also holds **metadata** about the data pushed to Adafruit IO. This includes settings for

©Adafruit Industries

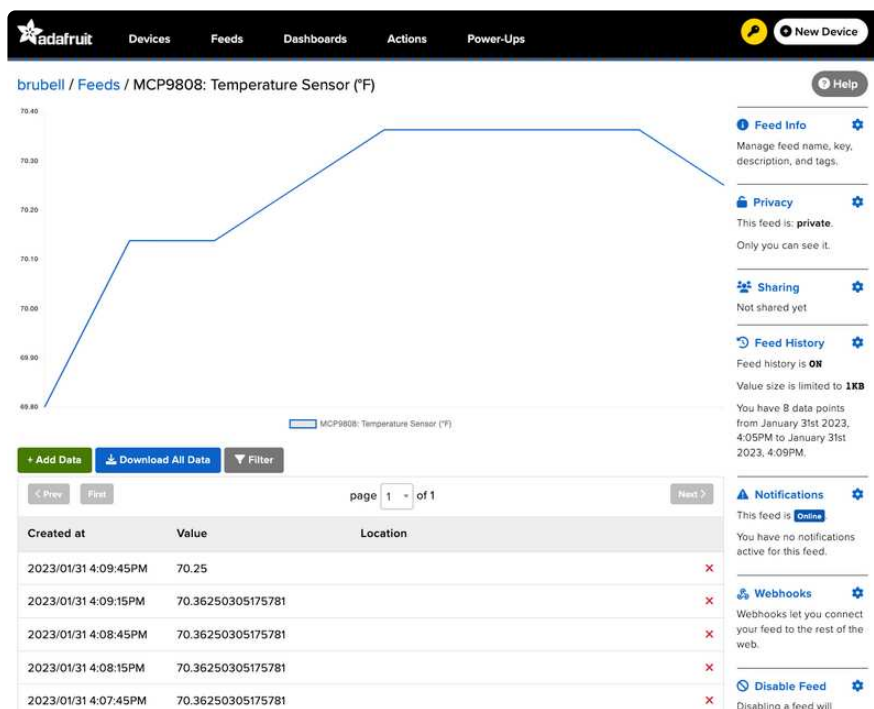
Page 95 of 113

whether the data is public or private, what license the stored sensor data falls under, and a general description of the data.

Next, to look at the sensor temperature feed. To navigate to a component's feed, click on the chart icon in the upper-right-hand corner of the component.



On the component's feed page, you'll each data point read by your sensor and when they were reported to Adafruit IO.



## Doing more with your sensor's Adafruit IO Feed

This only scratches the surface of what Adafruit IO Feeds can accomplish for your IoT projects. For a complete overview of Adafruit IO Feeds, including tasks like downloading feed data, sharing a feed, removing erroneous data points from a feed, and more, [head over to the "Adafruit IO Basics: Feed" learning guide \(https://adafru.it/ioA\)](https://adafru.it/ioA).

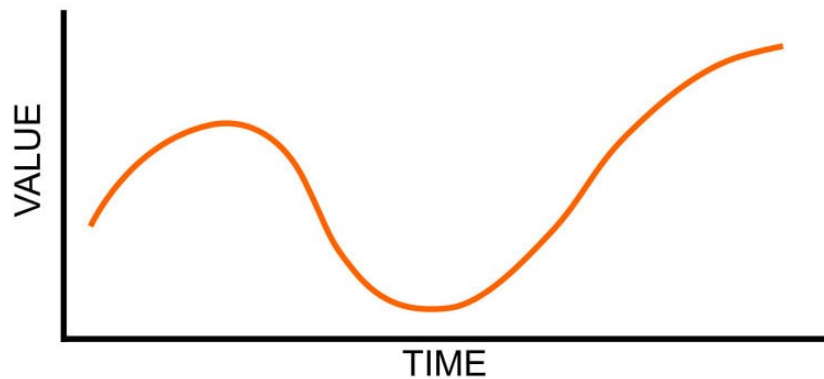
## Analog Input

Your microcontroller board has both digital and analog signal capabilities. Some pins are analog, some are digital, and some are capable of both. Check the **Pinouts** page in this guide for details about your board.

Analog signals are different from digital signals in that they can be any voltage and can vary continuously and smoothly between voltages. An analog signal is like a dimmer switch on a light, whereas a digital signal is like a simple on/off switch.

Digital signals only can ever have two states, they are either are **on** (high logic level voltage like 3.3V) or **off** (low logic level voltage like 0V / ground).

By contrast, analog signals can be any voltage in-between on and off, such as 1.8V or 0.001V or 2.98V and so on.



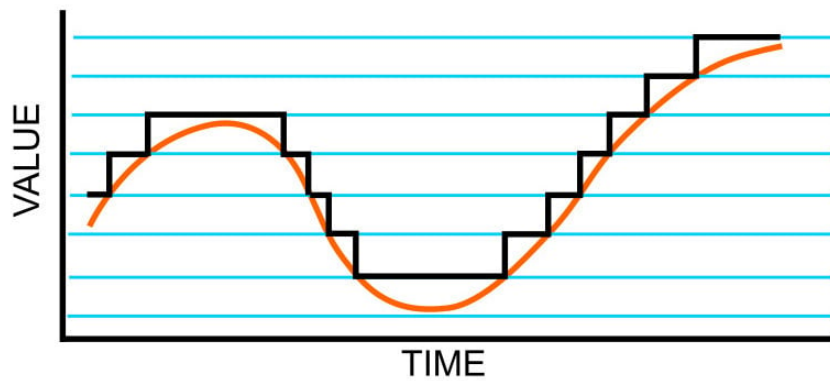
Analog signals are continuous values which means they can be an infinite number of different voltages. Think of analog signals like a floating point or fractional number, they can smoothly transiting to any in-between value like 1.8V, 1.81V, 1.801V, 1.8001V, 1.80001V and so forth to infinity.

Many devices use analog signals, in particular sensors typically output an analog signal or voltage that varies based on something being sensed like light, heat, humidity, etc.

## Analog to Digital Converter (ADC)

An analog-to-digital-converter, or ADC, is the key to reading analog signals and voltages with a microcontroller. An ADC is a device that reads the voltage of an analog signal and converts it into a digital, or numeric, value. The microcontroller can't read analog signals directly, so the analog signal is first converted into a numeric value by the ADC.

The black line below shows a digital signal over time, and the red line shows the converted analog signal over the same amount of time.

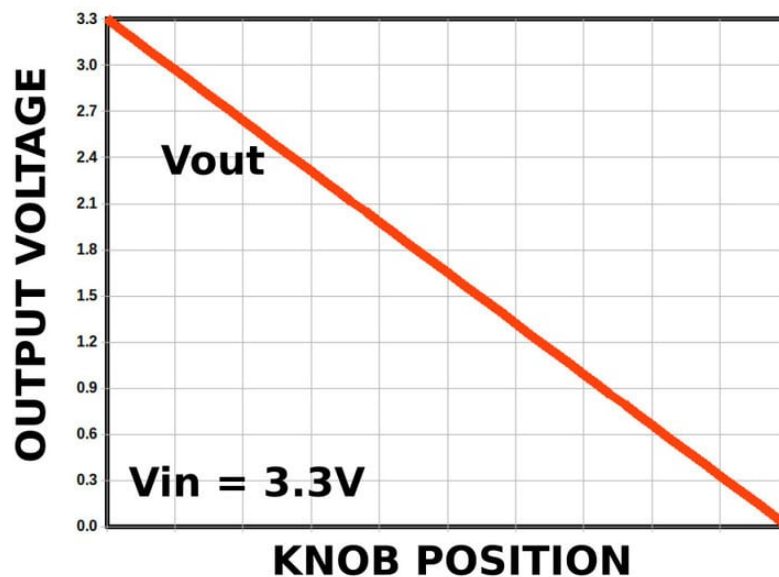


Once that analog signal has been converted by the ADC, the microcontroller can use those digital values any way you like!

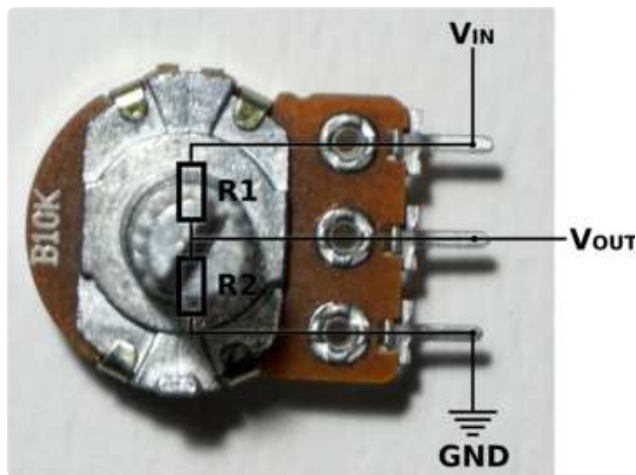
## Potentiometers

A potentiometer is a small variable resistor that you can twist a knob or shaft to change its resistance. It has three pins. By twisting the knob on the potentiometer you can change the resistance of the middle pin (called the wiper) to be anywhere within the range of resistance of the potentiometer.

By wiring the potentiometer to your board in a special way (called a voltage divider) you can turn the change in resistance into a change in voltage that your board's analog to digital converter can read.



To wire up a potentiometer as a voltage divider:

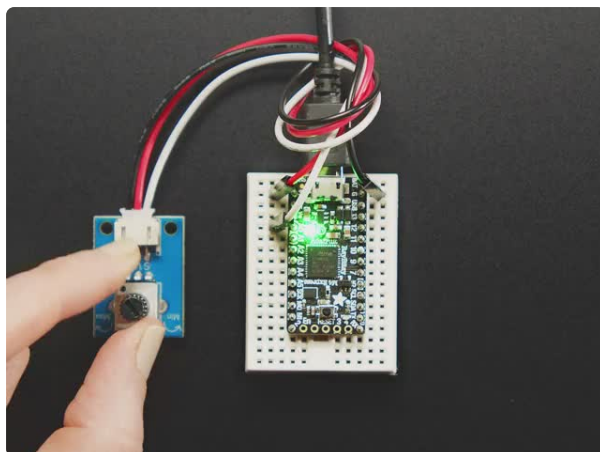


Connect one outside pin to ground  
 Connect the other outside pin to voltage in (e.g. 3.3V)  
 Connect the middle pin to an analog pin (e.g. A0)

## Hardware

In addition to your microcontroller board, you will need the following hardware to follow along with this example.

### Potentiometer



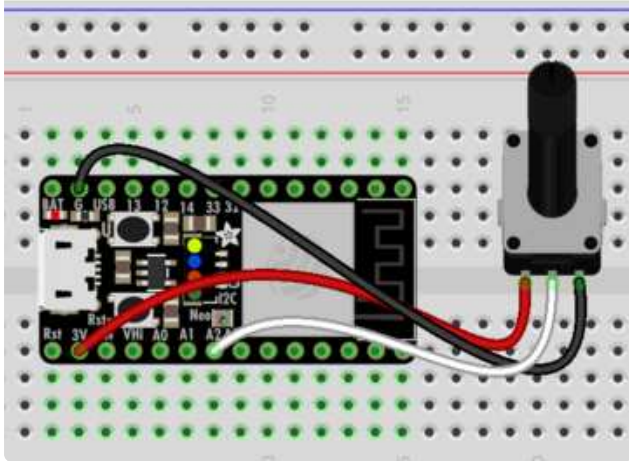
#### [STEMMA Wired Potentiometer Breakout Board - 10K ohm Linear](https://www.adafruit.com/product/4493)

For the easiest way possible to measure twists, turn to this STEMMA potentiometer breakout (ha!). This plug-n-play pot comes with a JST-PH 2mm connector and a matching

<https://www.adafruit.com/product/4493>

## Wire Up the Potentiometer

Connect the potentiometer to your board as follows.



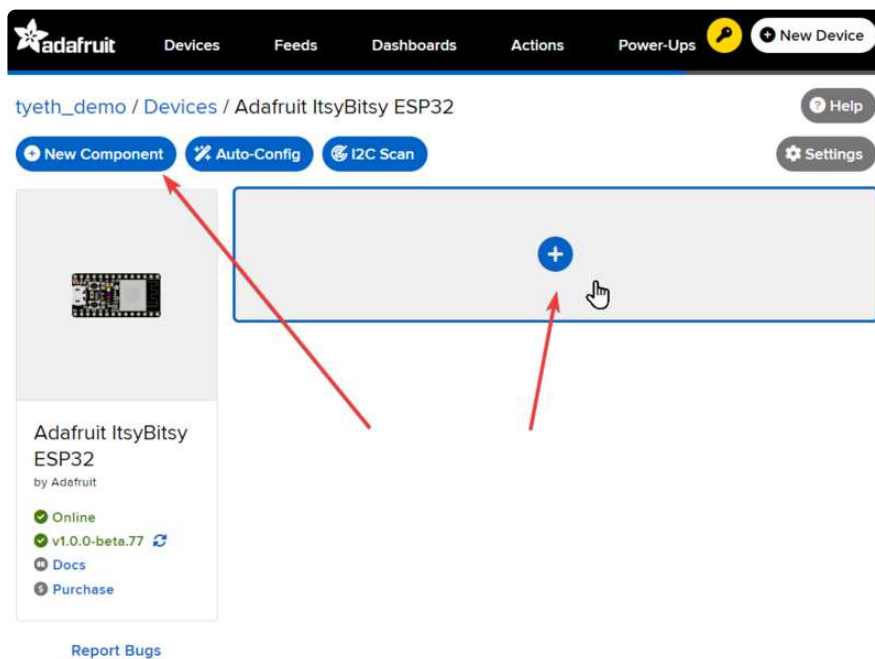
ItsyBitsy 3.3V to potentiometer left pin  
 ItsyBitsy A3 to potentiometer middle pin  
 ItsyBitsy GND to potentiometer right pin

**Note:** On WipperSnapper, the ESP32's ADC2 pins are unusable for analog input since WiFi is constantly running.

[For information on what Analog Input pins can \(and can't\) be used on this board, click here >>> \(https://adafru.it/19FN\)](https://adafru.it/19FN)

## Create a Potentiometer Component on Adafruit IO

On the device page, click the New Component (or "+") button to open the component picker.



## New Component

Which component would you like to set up?

A screenshot of the component picker search bar. The text 'potentiometer' is entered into the search box. A red arrow points to the search bar.

Displaying 1 matching Components.



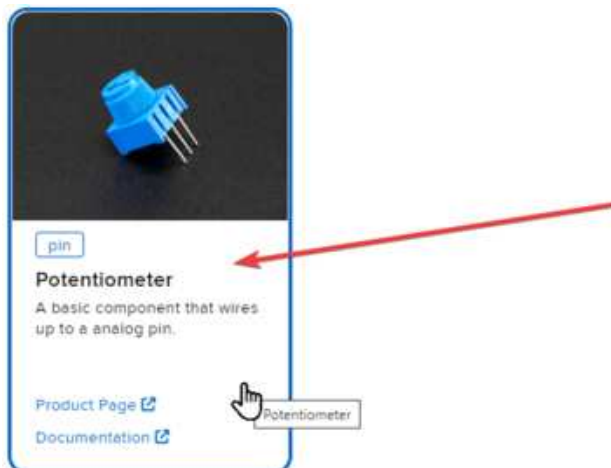
Search for the component name by entering **potentiometer** into the text box on the component picker, the list of components should update as soon as you stop typing.

### ? Filtering and searching for components

Since WipperSnapper supports such a large number of components, there is keyword filtering. Try searching for various keywords, like:

- component names: **aht20**, **servo**, **buzzer**, **button**, **potentiometer**, etc
- sensor types: **light**, **temperature**, **pressure**, **humidity**, etc
- interface: **i2c**, **uart**, **ds18x20**, **pin**, etc (also I2C addresses e.g. **0x44**)
- vendor: **Adafruit**, **ASAIR**, **Infineon**, **Bosch**, **Honeywell**, **Sensirion**, etc

There are also added product and documentation links for every component, follow the links beneath the component descriptions to be taken to the appropriate product page or Learn Guide.



Select the **Potentiometer** from the list of results to go to the component configuration page.

There will be a back button if you select the wrong component, and you can use the Edit component icon (⚙️) on the device page to update the component configuration in the future.



On the Create Potentiometer Component form:

- Set **Potentiometer Pin** to **A3**
- Select **"On Change"** as the **Return Interval**
- Select **Raw Analog Value** as the **Return Type**

Then, click Create Component

Create Potentiometer Component

Settings

Potentiometer Name

Potentiometer

Potentiometer Pin

A3

Return Interval

☒ On Change

☐ Periodically

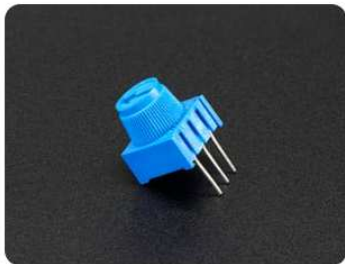
Return Type

☒ Raw Analog Value

☐ Voltage

[← Back to Component Type](#)

Create Component



The potentiometer component appears on your board page! Next, learning to read values from it.

tyeth\_demo / Devices / Adafruit ItsyBitsy ESP32

New Component

Auto-Config

I2C Scan

Help

Settings



Adafruit ItsyBitsy ESP32  
by Adafruit

Online

v1.0.0-beta.77

Docs

Purchase

Potentiometer

A38 potentiometer

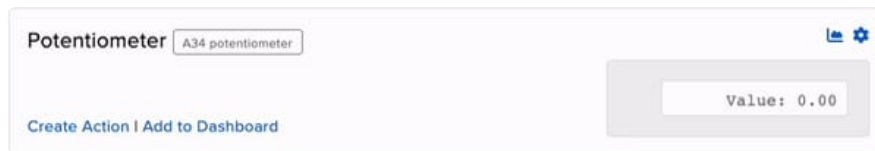
Value: -

Create Action | Add to Dashboard

+

## Read Analog Pin Values

Rotate the potentiometer to see the value change.

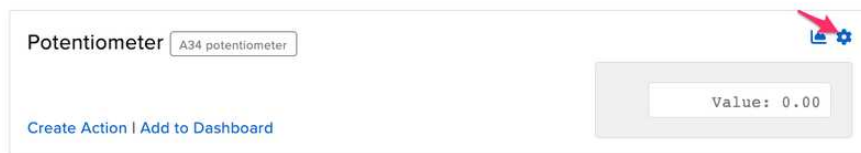


What do these values mean?

WipperSnapper reports ADC "raw values" as 16-bit unsigned integer values. Your potentiometer will read between 0 (twisting the pot to the leftmost position) and 65535 (twisting the pot to the rightmost position).

## Read Analog Pin Voltage Values

You can update the potentiometer component (or any analog pin component in WipperSnapper) to report values in Volts. To do this, on the right-hand side of the potentiometer component, click the cog button.

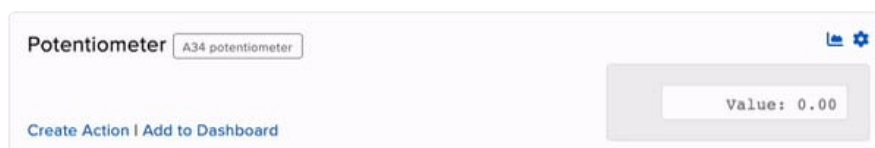


Under **Return Type**, click Voltage.

Click Update Component to send the updated component settings to your board running WipperSnapper.



Now, twist the potentiometer to see the value reported to Adafruit IO in Volts!



## Factory Reset

Your microcontroller ships running a factory demo. It's lovely, but you probably had other plans for the board. As you start working with your board, you may want to return to the original code to begin again, or you may find your board gets into a bad state. Either way, this page has you covered.

## Factory Reset Example Code

If you're still able to load Arduino sketches, you can load the following sketch onto your board to return it to its original state.

```
// SPDX-FileCopyrightText: 2024 ladyada for Adafruit Industries
//
// SPDX-License-Identifier: MIT

#include "WiFi.h"
#include <Adafruit_TestBed.h>
extern Adafruit_TestBed TB;

void setup() {
  Serial.begin(115200);
  Serial.println("ItsyBitsy ESP32 Factory Reset");

  TB.neopixelPin = PIN_NEOPIXEL;
  TB.neopixelNum = 1;
  TB.begin();
  TB.setColor(0x0);
}

uint8_t j = 0;
void loop() {
  pinMode(13, OUTPUT);
  if (j == 255) {
    TB.setColor(GREEN);
    Serial.println("scan start");
    // WiFi.scanNetworks will return the number of networks found
    int n = WiFi.scanNetworks();
    Serial.println("scan done");
    if (n == 0) {
      Serial.println("no networks found");
    } else {
      Serial.print(n);
      Serial.println(" networks found");
      for (int i = 0; i < n; ++i) {
        // Print SSID and RSSI for each network found
        Serial.print(i + 1);
        Serial.print(": ");
        Serial.print(WiFi.SSID(i));
        Serial.print(" (");
        Serial.print(WiFi.RSSI(i));
        Serial.print(")");
        Serial.println((WiFi.encryptionType(i) == WIFI_AUTH_OPEN)?" ":"*");
        delay(10);
      }
    }
    Serial.println("");
  }

  TB.setColor(TB.Wheel(j++));
  delay(10);
  if (j % 20 == 10) digitalWrite(13, HIGH);
  if (j % 20 == 0) digitalWrite(13, LOW);
  return;
}
```

Your board is now back to its factory-shipped state! You can now begin again with your plans for your board.

## Factory Reset .bin

If your board is in a state where Arduino isn't working, you may need to use these tools to flash a .bin file directly onto your board.

There are two ways to do a factory reset. The first is using WebSerial through a Chromium-based browser, and the second is using `esptool` via command line. **We highly recommend using WebSerial through Chrome/Chromium.**

First you'll need to download the factory-reset.bin file. Save the following file wherever is convenient for you. You'll need access to it for both tools.

Click to download  
Adafruit\_ItsyBitsy\_ESP32\_FactoryRes

<https://adafru.it/19je>

Now that you've downloaded the .bin file, you're ready to continue with the factory reset process. The next two sections walk you through using WebSerial and `esptool`.

## The WebSerial ESPTool Method

We highly recommend using WebSerial ESPTool method to perform a factory reset and bootloader repair. However, if you'd rather use `esptool` via command line, you can skip this section.

This method uses the WebSerial ESPTool through Chrome or a Chromium-based browser. The WebSerial ESPTool was designed to be a web-capable option for programming ESP32 boards. It allows you to erase the contents of the microcontroller and program up to four files at different offsets.

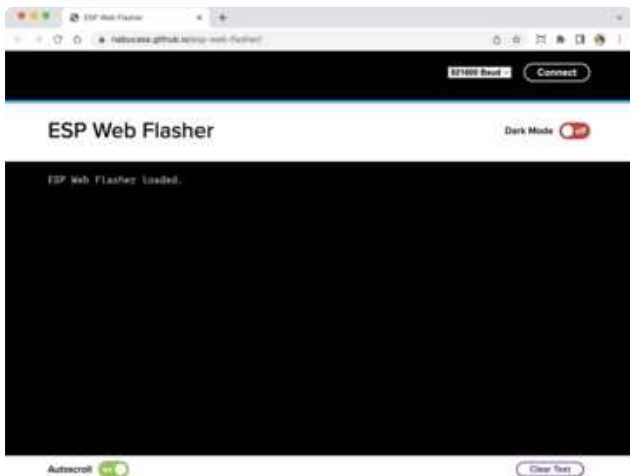
You will have to use a Chromium browser (like Chrome, Opera, Edge...) for this to work, Safari and Firefox, etc. are not supported because we need Web Serial and only Chromium is supporting it to the level needed.

Follow the steps to complete the factory reset.

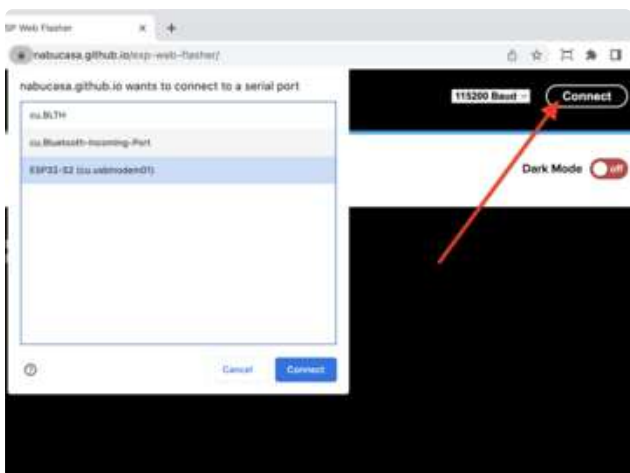
If you're using Chrome 88 or older, see the Older Versions of Chrome section at the end of this page for instructions on enabling Web Serial.

## Connect

You should have plugged in **only the ESP32 that you intend to flash**. That way there's no confusion in picking the proper port when it's time!



In the **Chrome browser** visit [https://adafruit.github.io/Adafruit\\_WebSerial\\_ESPTool/](https://adafruit.github.io/Adafruit_WebSerial_ESPTool/) (<https://adafru.it/PMB>). You should see something like the image shown.



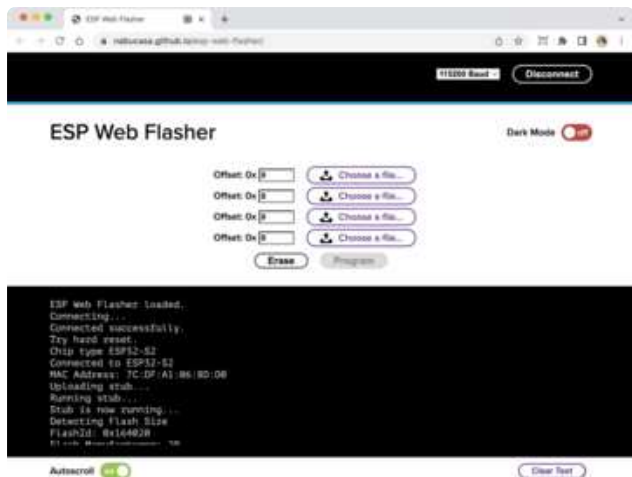
Press the **Connect** button in the top right of the web browser. You will get a pop up asking you to select the COM or Serial port.

Remember, you should remove all other **USB devices** so only the **ESP32 board** is attached, that way there's no confusion over multiple ports!

On some systems, such as **MacOS**, there may be additional system ports that appear in the list.

```
ESP Web Flasher loaded.
Connecting...
Connected successfully.
Try hard reset.
Chip type ESP32-S2
Connected to ESP32-S2
MAC Address: 7C:DF:A1:06:8D:D0
Uploading stub...
Running stub...
Stub is now running...
Detecting Flash Size
FlashId: 0x164020
Flash Manufacturer: 20
Flash Device: 4016
Auto-detected Flash size: 4MB
```

The Javascript code will now try to connect to the ROM bootloader. It may timeout for a bit until it succeeds. On success, you will see that it is **Connected** and will print out a unique **MAC address** identifying the board along with other information that was detected.



Once you have successfully connected, the command toolbar will appear.

## Erase the Contents



To erase the contents, click the Erase button. You will be prompted whether you want to continue. Click OK to continue or if you changed your mind, just click cancel.

```
Erasing flash memory. Please wait...
Finished. Took 15899ms to erase.
```

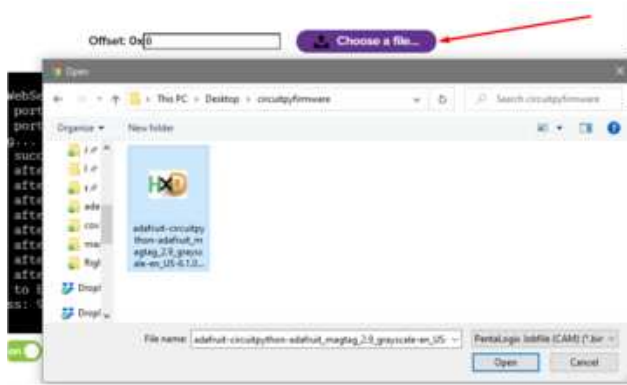
You'll see "Erasing flash memory. Please wait..." This will eventually be followed by "Finished." and the amount of time it took to erase.

**Do not disconnect!** Immediately continue on to programming the ESP32.

Do not disconnect after erasing! Immediately continue on to the next step!

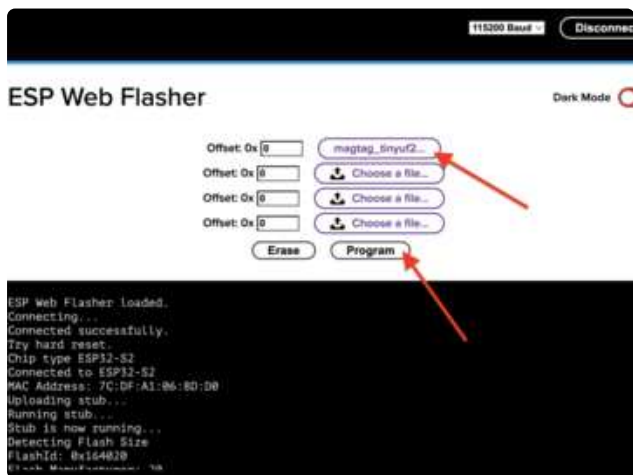
# Program the ESP32

Programming the microcontroller can be done with up to four files at different locations, but with the board-specific **factory-reset.bin** file, which you should have downloaded earlier, you only need to use one file.



Click on the first **Choose a file....** (The tool will only attempt to program buttons with a file and a unique location.) Then, select the **\*-factory-reset.bin** file you downloaded in Step 1 that matches your board.

Verify that the **Offset** box next to the file location you used is (0x) **0**.



Once you choose a file, the button text will change to match your filename. You can then select the **Program** button to begin flashing.



A progress bar will appear and after a minute or two, you will have written the firmware.

Once completed, you can skip down to the section titled **Reset the Board**.



## The `esptool` Method (for advanced users)

If you used WebSerial ESPTool, you do not need to complete the steps in this section!

Alternatively, you can [use Espressif's esptool program \(https://adafru.it/E9p\)](https://adafru.it/E9p) to communicate with the chip! `esptool` is the 'official' programming tool and is the most common/complete way to program an ESP chip.

### Install ESPTool.py

You will need to use the command line / Terminal to install and run `esptool`.

You will also need to have pip and Python installed (any version!).

Install the latest version using pip (you may be able to run `pip` without the `3` depending on your setup):

```
pip3 install --upgrade esptool
```

Then, you can run:

```
esptool.py
```

### Test the Installation

Run `esptool.py` in a new terminal/command line and verify you get something like the below:

```
9102 kattni@robocrepe:~ (esptool) $ esptool.py
esptool.py v3.0
usage: esptool [-h] [--chip {auto,esp8266,esp32,esp32s2,esp32s3beta2,esp32c3}]
               [--port PORT] [--baud BAUD]
               [--before {default_reset,no_reset,no_reset_no_sync}]
               [--after {hard_reset,soft_reset,no_reset}] [--no-stub]
               [--trace] [--override-vddsdio [{1.8V,1.9V,0FF}]]
               [--connect-attempts CONNECT_ATTEMPTS]
               {load_ram,dump_mem,read_mem,write_mem,write_flash,run,image_info,m
ake_image,elf2image,read_mac,chip_id,flash_id,read_flash_status,write_flash_statu
s,read_flash,verify_flash,erase_flash,erase_region,version,get_security_info}
...
```

### Connect

Run the following command, replacing the `COM88` identifier after `--port` with the `COMxx`, `/dev/cu.usbmodemxx` or `/dev/ttySxx` you found above.

```
esptool.py --port COM88 chip_id
```

You should get a notice that it connected over that port and found an ESP32.

```
9103 kattni@robocrepe:~ (esptool) $ esptool.py --port /dev/cu.usbserial-1144440 c
hip_id
esptool.py v3.0
Serial port /dev/cu.usbserial-1144440
Connecting....
Detecting chip type... ESP32
Chip is ESP32-PICO-V3-02 (revision 3)
Features: WiFi, BT, Dual Core, 240MHz, Embedded Flash, Embedded PSRAM, VRef calib
ration in efuse, Coding Scheme None
Crystal is 40MHz
MAC: 4c:75:25:be:19:98
Uploading stub...
Running stub...
Stub running...
Warning: ESP32 has no Chip ID. Reading MAC instead.
MAC: 4c:75:25:be:19:98
Hard resetting via RTS pin...
```

## Installing the Factory Test file

Run this command and replace the serial port name, `COM88`, with your matching port and `*-factory-reset.bin` with file you just downloaded

```
esptool.py --port COM88 write_flash 0x0 *-factory-reset.bin
```

Don't forget to change the `--port` name to match.

There might be a bit of a 'wait' when programming, where it doesn't seem like it's working. Give it a minute, it has to erase the old flash code which can cause it to seem like it's not running.

You'll finally get an output like this:

```
esptool.py v3.0
Serial port /dev/cu.usbserial-1144440
Connecting....
Detecting chip type... ESP32
Chip is ESP32-PICO-V3-02 (revision 3)
Features: WiFi, BT, Dual Core, 240MHz, Embedded Flash, Embedded PSRAM, VRef calib
ration in efuse, Coding Scheme None
Crystal is 40MHz
MAC: 4c:75:25:be:19:98
Uploading stub...
Running stub...
Stub running...
Configuring flash size...
Compressed 3084464 bytes to 241457...
Wrote 3084464 bytes (241457 compressed) at 0x00000000 in 26.3 seconds (effective
939.4 kbit/s)...
Hash of data verified.

Leaving...
Hard resetting via RTS pin...
```

Once completed, you can continue to the next section.

## Reset the board

Now that you've reprogrammed the board, you need to reset it to continue. Click the reset button to launch the new firmware.

In the event that pressing the reset button does not restart the board, unplug the board from USB and plug it back in to get the new firmware to start up.

The NeoPixel LED on the ItsyBitsy ESP32 will show a rainbow swirl while blinking the onboard LED. Then, the NeoPixel will turn green while a WiFi network scan occurs. If you open the Serial Monitor, you'll see a list of detected WiFi networks.

You've successfully returned your board to a factory reset state!

## Older Versions of Chrome

As of chrome 89, Web Serial is already enabled, so this step is only necessary on older browsers.

We suggest updating to Chrome 89 or newer, as Web Serial is enabled by default.

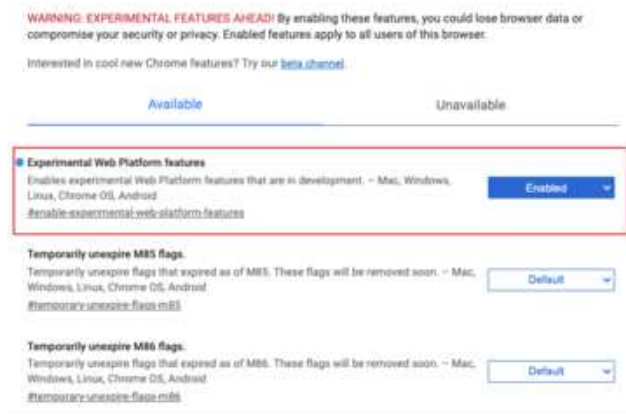
If you must continue using an older version of Chrome, follow these steps to enable Web Serial.



Sorry, Web Serial is not supported on this device, make sure you're running Chrome 78 or later and have enabled the enable-experimental-web-platform-features flag in chrome://flags

If you receive an error like the one shown when you visit the WebSerial ESPTool site, you're likely running an older version of Chrome.

**You must be using Chrome 78 or later to use Web Serial.**



To enable Web Serial in Chrome versions 78 through 88:

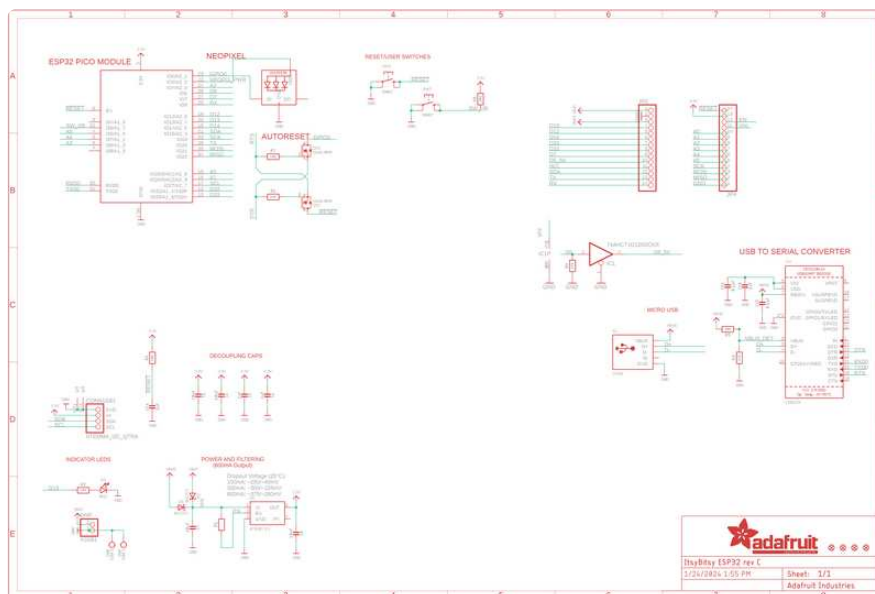
Visit **chrome://flags** from within Chrome.  
Find and enable the **Experimental Web Platform features**  
**Restart Chrome**

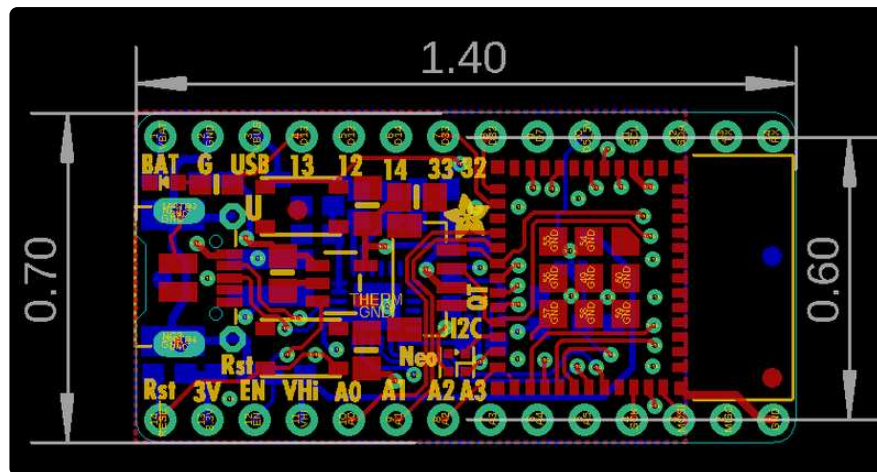
# Downloads

## Files

- [ESP32 datasheet \(https://adafru.it/YQA\)](https://adafru.it/YQA)
- [CH9102 datasheet \(https://adafru.it/-fb\)](https://adafru.it/-fb)
- [EagleCAD PCB files on GitHub \(https://adafru.it/19jf\)](https://adafru.it/19jf)
- [3D Models on GitHub \(https://adafru.it/19jA\)](https://adafru.it/19jA)
- [Fritzing object in the Adafruit Fritzing Library \(https://adafru.it/19jB\)](https://adafru.it/19jB)
- [PrettyPins pinout PDF on GitHub \(https://adafru.it/19jb\)](https://adafru.it/19jb)
- [PrettyPins pinout SVG \(https://adafru.it/19jC\)](https://adafru.it/19jC)

## Schematic and Fab Print





## 3D Model

